

烟幕干扰弹投放策略的优化方案

摘要

本文围绕烟幕干扰弹的投放策略优化问题，建立了基于**运动学分析**、**几何遮蔽判定**与**智能优化算法**的数学模型，综合运用**数值积分**、**粒子群优化 (PSO)**、**图论覆盖**、**贪心策略**及**多目标协同优化**等方法，解决了从单机单弹到多机多弹协同下的烟幕遮蔽时序规划与参数优化问题。

针对问题一，基于给定的无人机 FY1 运动参数，建立了导弹-云团-目标的几何遮蔽模型。通过数值积分计算遮蔽时间区间，结果表明，有效遮蔽时长为 **1.496s**，主要遮蔽区间为 **[7.953, 9.449]s**。

针对问题二，我们以最大化遮蔽时间为目标，建立了 FY1 飞行方向、速度、投放时间与起爆延时的四参数优化模型。采用粒子群算法 (PSO) 进行全局搜索，结合**单点视轴法快速评估**与**全域采样法高精度验证**的双阶段策略，优化得到 FY1 的最优飞行策略为：**航向角 7.410 度**、**速度 98.890m/s**、**于 0.030s 投放**、**起爆延时 0.840s**，**遮蔽时间提升至 5.060s**。

针对问题三，在 FY1 单机基础上扩展为三枚干扰弹协同遮蔽，建立了七参数优化模型（航向、速度、三组投放时间与起爆延时）。引入投放时间间隔约束与**并集遮蔽评估机制**，采用改进 PSO 算法进行多弹协同优化，最优方案实现总遮蔽时间 **6.160s**，三弹起爆时刻分别为 **0s**、**1.000s** 和 **25.654s**，形成了持续有效的遮蔽覆盖。

针对问题四，面向三架无人机 (FY1、FY2、FY3) 各投一枚干扰弹的场景，构建了基于**图论**的最大覆盖模型。通过**几何锚点生成候选策略集**，结合**时间掩码**与**分区拟阵贪心算法**，优化后三机协同遮蔽时间达到 **10.760s**，较单机方案显著提升，体现了多机协同的时空互补优势。

针对问题五，面对五架无人机协同拦截三枚导弹的超复杂场景，在问题四协同框架基础上进行多维扩展。通过建立了多机多弹协同运动学模型，统筹五架无人机以及其携带的三枚烟幕弹等参数，并创新构建多目标遮蔽优化模型，针对三枚导弹的不同弹道特性建立分导弹独立评价体系，通过**时空并集运算**将每枚导弹的遮蔽效果整合为时间轴上的**逻辑”或”关系**。面对高达 30 维的决策空间，设计三层优化架构：顶层采用图论网络筛选时空关键节点，中层运用模拟退火算法优化弹药分配与投放序列，底层通过 L1 精度验证实现参数微调。最终形成了时空接力与资源协同的立体防御体系，达到 **28.230s** 的有效遮蔽时长。

本研究提出的模型与算法在烟幕干扰弹投放策略设计中具有较强的适用性和扩展性，为多无人机协同干扰任务提供了理论依据与决策支持，对现代防空反导作战中的软杀伤手段运用具有重要参考价值。

关键字：数值积分 图论 粒子群算法 模拟退火算法 贪心

一、问题重述

1.1 问题背景

在应对现代战场复杂电磁对抗与精确制导威胁的背景下，无人机投掷烟幕干扰弹作为一种重要的无源干扰手段，其干扰效能直接关系到重要目标的生存能力和战术行动的突然性。长期以来，烟幕弹的投放策略与引爆控制主要依赖操作员经验，调整周期长、成本高，且干扰效果、覆盖面积与安全性之间的量化关系难以精确把握。面对现代作战对干扰时效性、覆盖精度和多目标协同干扰要求不断提升，传统人工决策方法难以系统优化不同战场场景下的干扰方案。因此，构建数学模型，从弹道运动学与空间几何关系出发量化烟幕遮蔽过程，精准计算无人机航迹、投弹点、起爆时序及安全边界参数，成为提升干扰科学性、可靠性与效费比的关键路径，有助于推动烟幕干扰战术从经验型向模型驱动型转变，实现传统干扰手段在现代战争条件下的智能化升级与创新应用。

1.2 问题要求

问题一：本题需要基于给定无人机 FY1 的飞行速度、方向、投弹时机和起爆延迟，建立烟幕遮蔽效能评估模型，计算单枚烟幕干扰弹对导弹 M1 的有效遮蔽时长。

问题二：本题需要优化无人机 FY1 的飞行参数及烟幕弹投放策略，建立最大化遮蔽时间的数学模型，求解最优参数组合。

问题三：本题需要设计单机多弹协同干扰方案，为无人机 FY1 规划三枚烟幕弹的投放策略，包括各弹的投放时序、空间位置和起爆参数，并将完整的策略方案输出到指定文件。

问题四：本题需要制定多机协同干扰方案，协调三架无人机（FY1、FY2、FY3）的飞行路径和投弹参数，实现对 M1 的有效遮蔽，并将优化后的协同策略输出到指定文件。

问题五：本题需要构建多机多目标协同干扰体系，统筹五架无人机对三枚导弹的干扰任务，优化各无人机的飞行参数、弹藥分配和投弹策略，并将最终的综合方案输出到指定文件。

二、问题分析

2.1 问题一分析

针对问题一，首先需要建立运动学模型。将导弹简化为匀速直线运动模型，其飞行方向直指假目标原点；将无人机简化为等高度匀速直线运动模型，其飞行方向由初始航

向确定；烟幕干扰弹的运动分为两个阶段：投放后的惯性运动与自由落体运动阶段，以及起爆后的匀速下沉运动阶段。其次，构建几何遮蔽判定模型。采用两种判定方法：单点视轴法以目标圆柱几何中心为代表视轴，进行球-线段最短距离计算；全域采样法则采用蒙特卡洛采样思想，在目标圆柱表面进行密集采样，通过统计各采样点的遮蔽情况来综合判定整体遮蔽效果。最后，设计数值积分策略。通过在时间维度上进行离散化处理，采用固定步长数值积分方法累加有效遮蔽时间，同时记录遮蔽时间区间，以确保计算结果的准确性。单点视轴法与全域采样法示意图如图1和图2所示。

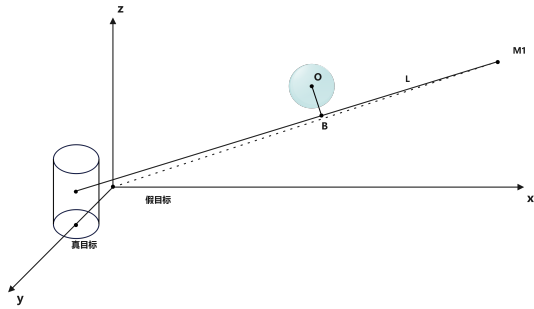


图1 单点视轴法示意图

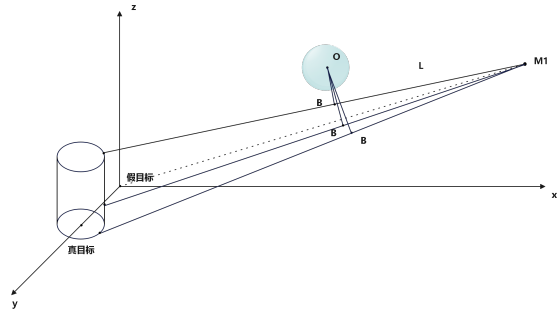


图2 全域采样法示意图

2.2 问题二分析

针对问题二，其模型是问题一模型的一般化形式。首先，建立精确的运动学模型。将导弹运动简化为匀速直线模型，其飞行方向始终指向假目标原点；无人机采用等高度匀速直线运动模型，其运动状态由航向角和速度两个参数决定；烟幕干扰弹的运动分为投放后的惯性运动与自由落体，以及起爆后的匀速下沉两个阶段。其次在构建多层次的几何遮蔽判定模型时，我们沿用问题一的单点视轴法和全域采样法；最后使用粒子群优化 (PSO) 算法进行求解。

2.3 问题三分析

针对问题三，首先需要建立多弹协同运动学模型。在单弹模型基础上，扩展为三枚烟幕干扰弹的协同运动模型，每枚弹具有独立的投放时刻和引信时延参数。模型需要考虑无人机的航向角和速度以及三枚弹的投放时序关系，特别是满足最小投放间隔 1 秒的约束条件。其次需要构建多弹并集遮蔽判定模型。采用集合论思想，将三枚烟幕弹的遮蔽效果视为时间轴上的逻辑“或”关系，即任一时刻只要至少一枚烟幕弹形成有效遮蔽，即判定为整体遮蔽有效。这种并集评价方法能够有效利用多弹的协同效应，实现更长时间的连续遮蔽。最后，设计分层优化求解策略。针对 8 维决策变量，即航向角、速度、3 个投放时刻、3 个引信时延的高维优化问题，采用两阶段优化策略：第一阶段使用单点视轴法判定进行快速全局搜索，找到近似最优解；第二阶段基于全域采样法判定进行局

部精细优化。这种策略既保证了计算效率，又确保了求解精度。单点视轴法在该题的使用如图3所示。

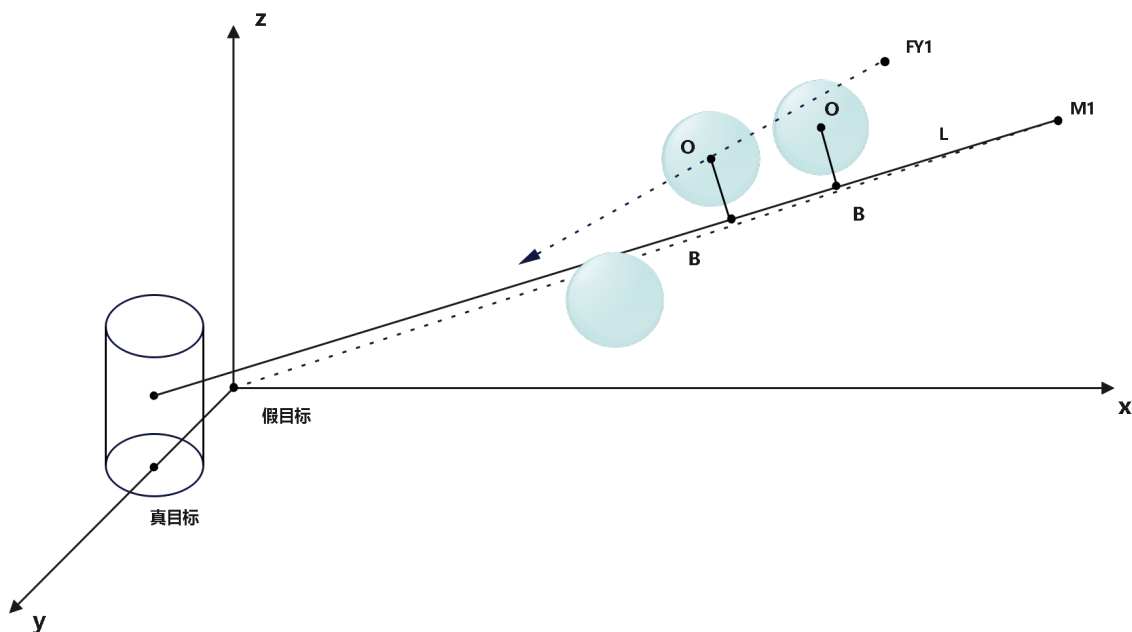


图3 问题三单点视轴法示意图

2.4 问题四分析

针对问题四，我们在单机模型基础上建立了多机协同运动学模型，该模型同时考虑三架无人机的独立航向角与速度参数，以及每架无人机所携带烟幕弹的投放时刻和引信时延参数。同时，我们采用多源遮蔽并集判定模型，通过布尔运算将三枚烟幕弹的遮蔽效果在时间轴上整合为逻辑“或”关系，任一时刻只需至少一枚烟幕弹形成有效遮蔽即判定为整体有效。面对高维决策空间挑战，我们设计了基于图论的最大覆盖优化框架：通过几何锚点方法生成候选解池，对每架无人机的航向角、速度、投放时间和引信时延进行离散采样；使用时间轴掩码表示法高效计算每个候选解的遮蔽时段；采用分区拟阵约束的贪心算法，保证每架无人机恰好选择一个作战方案；对于未覆盖时间空档，实施针对性兜底策略，生成定制化候选解；并提供单点视轴法、全域采样法和 DUAL 混合评估以适应不同精度需求。考虑到问题二已经得到 FY1 对 M1 拦截的最优参数，于是我们选择固定 FY1 的已知最优参数，然后在 FY2 和 FY3 的候选解空间中枚举最优配对，减少搜索空间的同时可以最大化整体遮蔽并集。单点视轴法在该题的使用如图4所示。

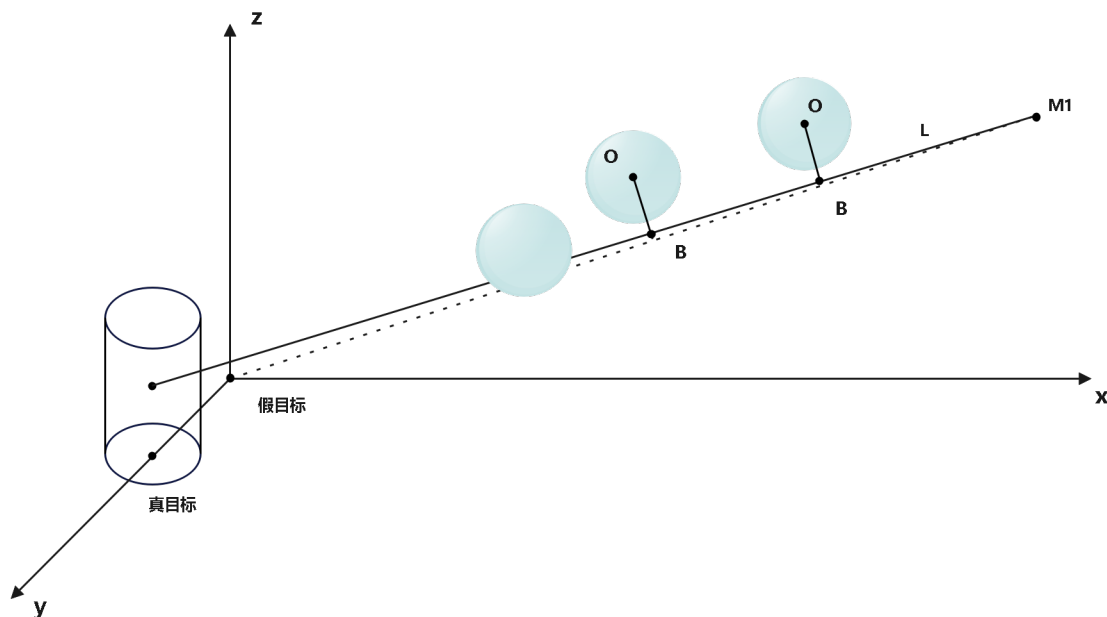


图 4 问题四单点视轴法示意图

2.5 问题五分析

针对问题五的五架无人机协同拦截三枚导弹的超复杂场景，我们在问题四协同框架基础上进行多维扩展。首先建立多机多弹协同运动学模型，统筹五架无人机各自独立的航向角与速度参数，以及每机最多三枚烟幕弹的投放时刻和引信时延参数，并严格遵循单机内部弹间投放间隔大于等于 1 秒的物理约束。我们构建了多导弹综合遮蔽优化模型，针对三枚导弹的不同弹道特性建立独立评价体系，通过时空并集运算将每枚导弹的遮蔽效果整合后求和作为总目标。面对高达 30 维的决策空间挑战，我们设计了多阶段优化流程：首先采用图论网络生成时空关键候选解集；继而运用播种策略确保每枚导弹获得初始覆盖，并通过锁定机制固定每架无人机的航向与速度以压缩搜索空间；随后采用贪心算法完成剩余弹药的分配；最后底层依次通过 L1 精度局部搜索和模拟退火算法对投放时序和启爆参数进行精细化调优，最终形成时空接力与资源协同的立体防御体系。此外，我们还建立了包含蒙特卡洛采样、参数灵敏度分析与邻域检验在内的全面稳健性验证体系，确保解方案的可靠性。问题五的投放前状态如图??所示。

三、模型假设

- (1) 无人机运动和导弹运动轨迹为匀速直线运动，忽略风速等外界因素影响。
- (2) 覆盖时间计算基于离散时间步长，假设时间连续且采样间隔足够小以近似连续覆盖。

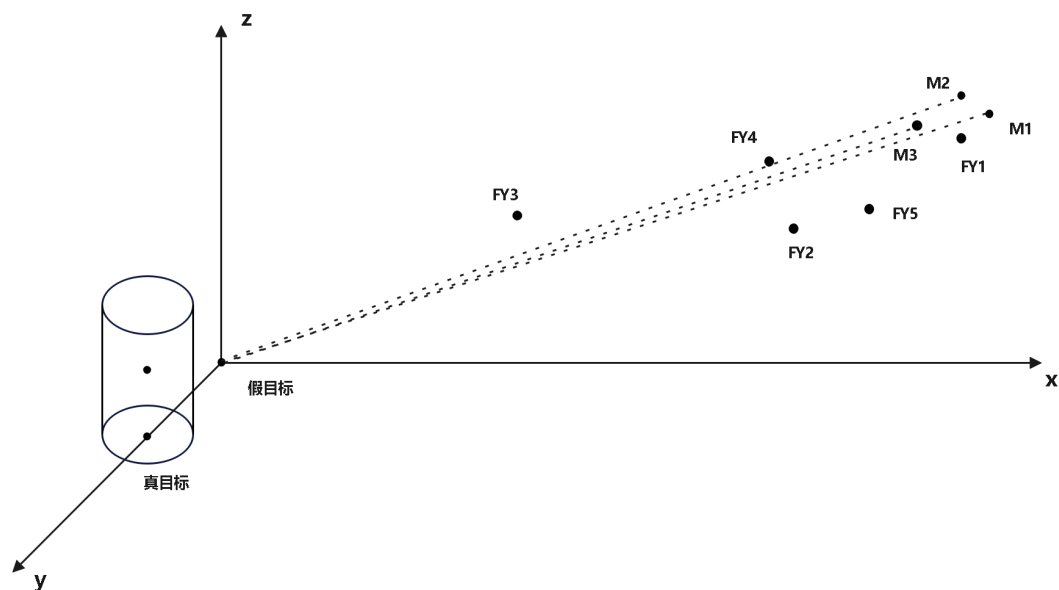


图 5 问题五示意图

(3) 多无人机协同时，假设各无人机动作独立，且无通信延迟或冲突。

(4) 邻域检验和蒙特卡洛验证中，扰动服从高斯分布或均匀分布，且扰动范围能反映实际误差。

四、符号说明

表 1 通用符号说明表

符号	单位	含义
$\vec{M}_{m0}$	m	第 m 枚导弹的初始位置矢量
$\vec{M}_m(t)$	m	第 m 枚导弹在时刻 t 的位置矢量
V_M	m/s	导弹的飞行速度
$\hat{d}_m$	—	第 m 枚导弹飞向假目标原点的单位方向向量
$T_{hit,m}$	s	第 m 枚导弹的命中（飞抵目标）时间
$\vec{U}_{i0}$	m	第 i 架无人机的初始位置矢量
$\vec{U}_i(t)$	m	第 i 架无人机在时刻 t 的位置矢量
v_i	m/s	第 i 架无人机的飞行速度
θ_i	rad	第 i 架无人机的航向角（从正东方向逆时针测量）
$t_{drop,ij}$	s	第 i 架无人机投放其第 j 枚烟幕弹的时间

接下页

表 1 (续): 通用符号说明表

符号	单位	含义
τ_{ij}	s	第 i 架无人机投放其第 j 枚烟幕弹的引信时延
$t_{\text{burst},ij}$	s	第 i 架无人机投放其第 j 枚烟幕弹的起爆时间 ($t_{\text{drop},ij} + \tau_{ij}$)
$\vec{R}_{\text{drop},ij}$	m	第 i 架无人机投放其第 j 枚烟幕弹的投放点位置矢量
$\vec{S}_{\text{burst},ij}$	m	第 i 架无人机投放其第 j 枚烟幕弹的起爆点位置矢量
$\vec{S}_{ij}(t)$	m	第 i 架无人机投放其第 j 枚烟幕弹的云团在时刻 t 的球心位置矢量
V_s	m/s	烟幕云团的下沉速度
R_s	m	烟幕云团的有效遮蔽半径
g	m/s ²	重力加速度
$C_{ij,m}(t)$	—	第 i 架无人机第 j 枚烟幕弹在时刻 t 对第 m 枚导弹的遮蔽指示函数
$C_{\text{union},m}(t)$	—	所有烟幕弹在时刻 t 对第 m 枚导弹的并集遮蔽指示函数
T_m	s	对第 m 枚导弹的总有效遮蔽时间
θ	rad	单无人机场景下无人机的航向角
v_u	m/s	单无人机场景下无人机的飞行速度
$\vec{U}(t)$	m	单无人机场景下无人机在时刻 t 的位置矢量
$\vec{M}(t)$	m	单导弹场景下导弹在时刻 t 的位置矢量 $\vec{M}_1(t)$
t_0	s	单无人机单烟幕弹场景下烟幕弹的投放时间
τ	s	单无人机单烟幕弹场景下烟幕弹的引信时延
$\vec{r}_0$	m	单无人机单烟幕弹场景下烟幕弹的投放点位置矢量
$\vec{s}_1$	m	单无人机单烟幕弹场景下烟幕弹的起爆点位置矢量
$\vec{s}(t)$	m	单无人机单烟幕弹场景下烟幕弹云团在时刻 t 的球心位置矢量
$C(t)$	—	单无人机单烟幕弹单导弹场景下烟幕弹对导弹的遮蔽指示函数
T_c	s	单导弹场景下对导弹的总有效遮蔽时间
$\vec{u}(t)$	m	单无人机场景下无人机在时刻 t 的位置矢量
$\vec{m}(t)$	m	单导弹场景下导弹在时刻 t 的位置矢量
$t_{0,k}$	s	单无人机多烟幕弹场景下第 k 枚烟幕弹的投放时间
τ_k	s	单无人机多烟幕弹场景下第 k 枚烟幕弹的引信时延
$\vec{r}_{0,k}$	m	单无人机多烟幕弹场景下第 k 枚烟幕弹的投放点位置矢量
$\vec{s}_{1,k}$	m	单无人机多烟幕弹场景下第 k 枚烟幕弹的起爆点位置矢量
$\vec{s}_k(t)$	m	单无人机多烟幕弹场景下第 k 枚烟幕弹云团在时刻 t 的球心位置矢量
$C_{\text{union}}(t)$	—	单导弹场景下所有烟幕弹对导弹的并集遮蔽指示函数
Δt	s	时间离散化步长

五、模型的建立与求解

5.1 问题一的建模与求解

为实现对导弹 M1 的有效遮蔽时长计算，本问基于给定的无人机飞行参数和烟幕弹投放参数，建立了一个基于运动学建模和几何遮蔽判定的计算模型。模型旨在计算在固定参数条件下烟幕干扰弹对 M1 的有效遮蔽时长。

1. 运动学模型

我们建立的运动学模型包含三部分：导弹运动模型、无人机运动模型以及烟幕干扰弹运动与起爆模型，具体如下：

导弹运动模型

导弹 M1 自初始位置 $\vec{M}_0 = (20000, 0, 2000)$ 以恒定速度 $V_M = 300\text{m/s}$ 沿直线飞向假目标原点 $O(0, 0, 0)$ 。其位置矢量随时间 t 的变化为：

$$\vec{m}(t) = \vec{M}_0 + V_M \cdot \hat{d} \cdot t \quad (1)$$

其中单位方向向量

$$\hat{d} = \frac{-\vec{M}_0}{\|\vec{M}_0\|}$$

。

无人机运动模型

无人机 FY1 自初始位置 $\vec{U}_0 = (17800, 0, 1800)$ 受领任务后，以恒定速度 $v_u = 120\text{m/s}$ 和水平航向角 $\theta = \pi$ ，即朝向假目标方向进行等高度直线飞行。其位置矢量为：

$$\vec{u}(t) = [U_{0x} + v_u \cos \theta \cdot t, U_{0y} + v_u \sin \theta \cdot t, U_{0z}] = [17800 - 120t, 0, 1800] \quad (2)$$

烟幕干扰弹运动与起爆模型

烟幕弹在 $t_0 = 1.5\text{s}$ 时刻被投放。投放后，其水平方向继承无人机的速度，垂直方向作自由落体运动。经过引信时延 $\tau = 3.6\text{s}$ 后于 $t_1 = t_0 + \tau = 5.1\text{s}$ 时刻起爆。

投放点位置为：

$$\vec{r}_0 = \vec{u}(t_0) = [17800 - 120 \times 1.5, 0, 1800] = [17620, 0, 1800] \quad (3)$$

起爆点位置 $\vec{s}_1$ 由三部分组成：

$$\vec{s}_1 = \vec{r}_0 + \Delta \vec{s}_{\text{水平}} + \Delta \vec{s}_{\text{垂直}} \quad (4)$$

$$\Delta \vec{s}_{\text{水平}} = [v_u \cos \theta \cdot \tau, v_u \sin \theta \cdot \tau, 0] \quad (5)$$

$$\Delta \vec{s}_{\text{垂直}} = \left[0, 0, -\frac{1}{2}g\tau^2 \right] \quad (6)$$

其中 $g = 9.81\text{m/s}^2$ 为重力加速度。代入具体参数：

$$\Delta \vec{s}_{\text{水平}} = [120 \times (-1) \times 3.6, 0, 0] = [-432, 0, 0] \quad (7)$$

$$\Delta \vec{s}_{\text{垂直}} = \left[0, 0, -\frac{1}{2} \times 9.81 \times 12.96 \right] = [0, 0, -63.569] \quad (8)$$

因此起爆点坐标为：

$$\vec{s}_1 = [17620 - 432, 0, 1800 - 63.569] = [17188, 0, 1736.431] \quad (9)$$

起爆后，烟幕云团球心以匀速 $V_s = 3\text{m/s}$ 下沉。故在 $t \geq t_1$ 时刻，球心位置为：

$$\vec{s}(t) = \vec{s}_1 + [0, 0, -V_s \cdot (t - t_1)]^T = [17188, 0, 1736.431 - 3(t - 5.1)] \quad (10)$$

2. 遮蔽判定模型

为量化遮蔽效果，我们采用全域采样法进行精确计算。该方法对目标圆柱体表面进行密集采样，包含上下底面圆周点与侧壁多层点。目标圆柱体下底面圆心为 $(0, 200, 0)$ ，半径 $R_{\text{tar}} = 7\text{m}$ ，高 $H_{\text{tar}} = 10\text{m}$ 。

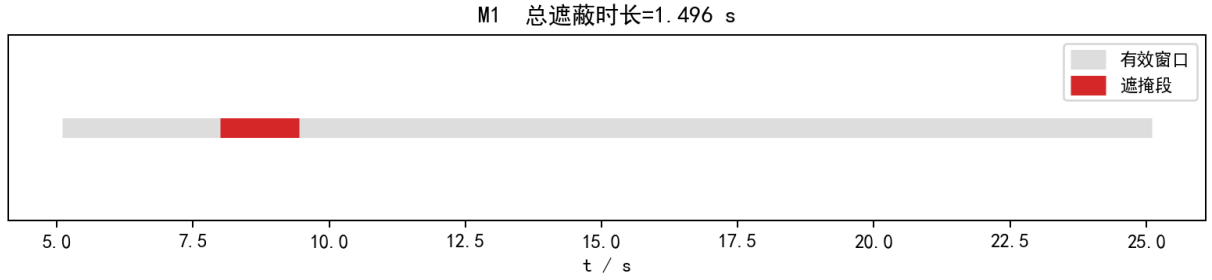


图6 问题一遮掩时长

设采样点集合为 $\vec{P}_i$ ，对于每个采样点 $\vec{P}_i$ ，在时刻 t ，计算烟幕云团球心 $\vec{s}(t)$ 到线段 $\overline{\vec{P}_i \vec{m}(t)}$ 的最短距离 $d_i(t)$ 。若存在至少一个采样点满足 $d_i(t) \leq R_s$ ，其中 $R_s = 10\text{m}$ 为云团有效半径，则判定为目标被遮蔽。即遮蔽指示函数为：

$$C(t) = \begin{cases} 1, & \exists i, d_i(t) \leq R_s \\ 0, & \text{否则} \end{cases} \quad (11)$$

有效遮蔽时长为起爆后云团有效时间窗口内的遮蔽时间积分。云团有效时长为 $T_e = 20\text{s}$ ，即从 t_1 到 $t_1 + T_e$ 。因此，有效遮蔽时长 T_c 为：

$$T_c = \int_{t_1}^{t_1 + T_e} C(t) dt \quad (12)$$

由于 $C(t)$ 为指示函数，实际计算中采用数值积分方法，即：

$$T_c \approx \sum_{k=0}^N C(t_k) \Delta t \quad (13)$$

其中 $t_k = t_1 + k\Delta t$, Δt 为时间步长, $N = \lfloor T_e / \Delta t \rfloor$ 。

基于上述模型, 通过数值计算得到以下结果:

起爆时间 $t_e = 5.100\text{s}$;

起爆点坐标 = (17188.000, 0.000, 1736.431)m;

有效遮蔽时长 $T_c = 1.496\text{s}$;

遮蔽时间区间为 [7.953, 9.449]s。

这一结果表明, 在给定参数条件下, 烟幕干扰弹对导弹 M1 的有效遮蔽时长为 1.496 秒, 遮蔽发生在导弹飞行至目标区域的特定时间段内。题一的遮掩时长如图6所示。

5.2 问题二的建模与求解

5.2.1 模型的建立

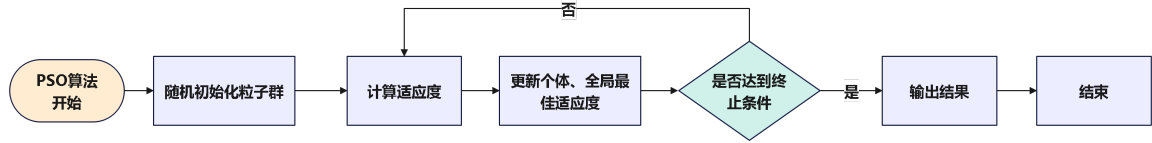


图7 PSO 算法流程图

在第一问建立的模型基础上, 问题二进一步将固定参数扩展为可优化变量, 旨在确定无人机 FY1 的最优飞行参数 (航向角 θ 与飞行速度 v_u) 及烟幕弹投放参数 (投放时刻 t_0 与引信时延 τ), 以最大化有效遮蔽时长 T_c 。

运动学模型沿用第一问的基本框架, 但将固定参数替换为决策变量:

导弹 M1 运动模型保持不变, 如公式 (1) 所示;

在无人机运动模型中, 无人机 FY1 的运动参数变为决策变量:

$$\vec{u}(t) = [U_{0x} + v_u \cos \theta \cdot t, U_{0y} + v_u \sin \theta \cdot t, U_{0z}] \quad (14)$$

其中 $v_u \in [70, 140]\text{m/s}$, $\theta \in [0, 2\pi)$ 。

在烟幕干扰弹运动与起爆模型中, 烟幕弹投放参数变为决策变量:

$$\vec{r}_0 = \vec{u}(t_0) \quad (15)$$

$$\vec{s}_1 = \vec{r}_0 + [v_u \cos \theta \cdot \tau, v_u \sin \theta \cdot \tau, 0] + \left[0, 0, -\frac{1}{2}g\tau^2 \right] \quad (16)$$

其中 t_0 和 $\tau \in [0.2, 12.0]$ s 为决策变量。

起爆后云团运动模型保持不变, 如公式 (2) 所示, 其中 $t_1 = t_0 + \tau$ 。

遮蔽判定模型完全沿用第一问的全域采样法, 如公式 (11) 所示,

优化目标为最大化有效遮蔽的总时长:

$$\max_{\theta, v_u, t_0, \tau} T_c = \int_{t_1}^{\min(t_1+T_e, T_h)} C(t) dt \quad (17)$$

其中 $T_e = 20$ s 为云团有效时长, T_h 为导弹命中目标时间。

该优化问题是一个四维非线性规划问题, 决策变量为 $\mathbf{x} = [\theta, v_u, t_0, \tau]$, 并受限于其各自的边界约束。

针对该黑盒优化问题, 我们所采用的求解算法为粒子群优化 (PSO)。其算法流程如图7所示。PSO 模拟鸟群社会行为, 通过粒子在解空间中追随当前最优粒子进行搜索。

每个粒子代表一个候选解 $\mathbf{x}_i$, 其速度与位置更新公式如下:

$$\mathbf{v}_i^{(k+1)} = \omega \mathbf{v}_i^{(k)} + c_1 r_1 (\mathbf{p}_i - \mathbf{x}_i^{(k)}) + c_2 r_2 (\mathbf{g} - \mathbf{x}_i^{(k)}) \quad (18)$$

$$\mathbf{x}_i^{(k+1)} = \mathbf{x}_i^{(k)} + \mathbf{v}_i^{(k+1)} \quad (19)$$

为兼顾效率与精度, 采用两阶段优化策略:

第一阶段: 采用计算高效的单点视轴法进行全局初步搜索, 快速定位潜力较大的解区域;

第二阶段: 以第一阶段得到的最优解为初始点, 采用更精确的全域采样法进行局部精细优化, 得到最终的最优策略。

通过上述优化模型和算法, 可求解得到无人机 FY1 的最优飞行参数和烟幕弹投放参数, 从而实现单枚烟幕干扰弹对导弹 M1 的最长有效遮蔽时长。

5.2.2 模型的求解

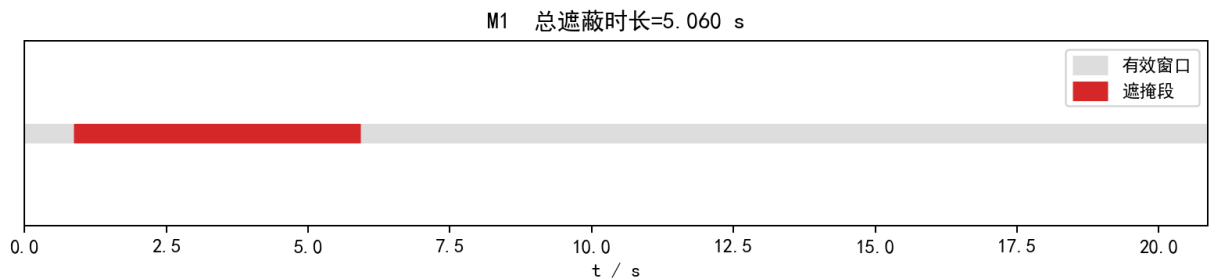


图 8 问题二遮掩时长

基于上述建立的优化模型与粒子群算法, 对无人机 FY1 单弹干扰导弹 M1 的最优策略进行求解。分别采用单点视轴法、全域采样法以及两阶段优化策略进行计算, 得到的最优决策变量与遮蔽效果如表2所示, 遮掩时长如图8所示。通过对表 2 中三种策略的

表 2 问题二不同判定方法下的最优投放策略与遮蔽效果对比

参数指标	单点视轴法	全域采样法	两阶段优化法（单点 → 全域）
航向角 θ ($^{\circ}$)	6.97	6.33	7.41
飞行速度 v (m/s)	99.67	102.18	98.89
投放时刻 t_{drop} (s)	0.03	0.63	0.03
引信时延 τ (s)	0.86	0.30	0.84
起爆时刻 t_{burst} (s)	0.89	0.93	0.87
起爆点坐标 x (m)	17887.89	17893.95	17884.77
起爆点坐标 y (m)	10.75	10.42	11.02
起爆点坐标 z (m)	1796.40	1799.56	1796.56
遮蔽总时长 (s)	4.81	4.94	5.06
有效遮蔽时间区间 (s)	[1.06, 5.87]	[1.41, 6.35]	[0.88, 5.94]

对比分析，可以得出以下结论：

优化方法有效性：全域采样法得到的遮蔽总时长优于单点视轴法的结果，这表明基于目标圆柱体表面采样的精确几何判定能更有效地评估真实的遮蔽效果，优化得到的策略也更合理。

两阶段优化策略的优势：采用单点视轴法 → 全域采样法两阶段优化策略得到了全局最优解，显著优于单一方法的结果。这验证了两阶段策略的设计思路：首先利用计算高效的单点视轴法进行全局探索，快速定位潜力解区域；再启动计算开销较大但精度更高的全域采样法，在该区域进行局部精细搜索。该策略在保证结果可靠性的同时，兼顾了计算效率。

最优策略分析：两阶段策略得到的最优方案要求无人机在接令后极短时间内即完成决策并投放烟幕弹。无人机应保持航向角约 **7.41 $^{\circ}$** ，以约 **98.85 m/s** 的速度飞行。烟幕弹的引信时延设置为 **0.84 s**，将于位置 **(17884.77, 11.02, 1796.56)** 处起爆。该策略可在导弹飞行的早期 ($t = 0.88 \text{ s}$) 即形成有效遮蔽，并持续至 $t = 5.94 \text{ s}$ ，从而实现对导弹末段制导关键窗口的最大化覆盖。

综上，本研究建立的模型能有效求解烟幕弹最优投放策略问题，两阶段优化策略兼顾了计算效率与求解精度，所获方案能显著提升烟幕干扰的作战效能。

5.2.3 模型检验

为了验证所建立模型的正确性与鲁棒性，我们从多个维度对模型进行了系统性检验，包括基本一致性检验、网格基线对比、时间步长收敛性、旋转不变性、参数敏感性

以及多种子稳定性六个方面。

基本一致性检验

首先对三种求解策略，即单点视轴近似法、全域采样精确法和两阶段优化法进行了对比验证，结果如表 2 所示。三种策略均能获得合理的遮蔽方案，且所得参数均满足物理约束条件。各策略得到的遮蔽区间均有序且互不重叠，且遮蔽总时长均未超过理论物理上界，验证了模型的基本一致性。

粗网格基线对比

为进一步验证优化算法的有效性，采用了粗网格搜索法计算了遮蔽时长的理论上界。在 8 个航向角、5 个速度值、9 个投放时间和 8 个引信时延参数的组合下，单点视轴近似法的网格最优值为 4.24 秒，全域采样精确法的网格最优值为 5.02 秒。PSO 算法求得的结果均显著优于粗网格搜索的最优值（单点视轴法：4.81 > 4.08，全域采样法：5.04 > 4.24），证明了优化算法的有效性。

时间步长收敛性检验

为验证数值积分步长对结果的影响，针对 L0 近似法进行了时间步长收敛性测试。分别采用 0.04s、0.02s、0.01s 和 0.005s 的步长进行计算，得到的遮蔽时长分别为 4.68s、4.78s、4.77s 和 4.78s。当步长减小至 0.01s 后，结果变化小于 0.01s，表明数值积分已基本收敛，选择 0.01s 的步长可在精度和效率间取得良好平衡。检验结果如表3所示。

表 3 时间步长收敛结果 (单点视轴法)

时间步长 (dt)	遮蔽时长 (s)
0.040	4.680
0.020	4.780
0.010	4.770
0.005	4.780

旋转不变性检验

为验证模型在坐标系旋转下的稳定性，将整个场景，包括导弹初始位置、无人机初始位置、目标位置，绕 z 轴旋转 0° 、 45° 、 90° 和 135° 后重新求解。四种旋转角度下得到的遮蔽时长分别为 5.02s、5.04s、5.06s 和 5.08s，最大差异仅为 0.06s (< 0.5s 容差)，证明了模型具有良好的旋转不变性。

小扰动敏感性检验

为评估模型对参数扰动的敏感性，对导弹和无人机的初始位置以及烟幕云团参数进行了 8 次随机扰动测试。遮蔽时长的均值为 5.0525s，标准差为 0.1077s，最小值为 4.80s，

最大值为 5.16s。结果表明模型对参数小扰动不敏感，具有良好的鲁棒性。

多种子稳定性检验

为评估优化算法的稳定性，使用 4 个不同的随机种子 (2025, 7, 17, 99) 分别运行两阶段优化算法。得到的遮蔽时长分别为 5.06s、5.06s、5.06s 和 5.06s，均值为 5.06s，标准差为 0.0s。结果表明算法在不同初始化条件下都能稳定收敛到相近的最优解，具有良好的稳定性。

综上所述，通过上述六方面的检验，验证了所建模型的正确性、有效性和鲁棒性，为后续结果的应用提供了可靠保证。

5.3 问题三的建模与求解

5.3.1 模型的建立

在问题一，二建立的单枚烟幕弹遮蔽模型基础上，问题三需研究使用单架无人机 FY1 连续投放 3 枚烟幕干扰弹对导弹 M1 的最优干扰策略。模型旨在协同优化无人机的飞行参数（航向角 θ 与飞行速度 v_u ）及各枚烟幕弹的投放参数（投放时刻 $t_{0,k}$ 与引信时延 τ_k , $k = 1, 2, 3$ ），以最大化三枚烟幕弹形成的并集有效遮蔽时长 T_c 。

1. 运动学模型

运动学模型在前两问的基本框架下，扩展至多枚烟幕弹的情形。

导弹运动模型

导弹 M1 自初始位置 $\vec{M}_0 = (20000, 0, 2000)$ 以速度 $V_M = 300\text{m/s}$ 匀速飞向假目标原点 $O(0, 0, 0)$ ，其运动轨迹表示如公式 (1) 所示。

无人机运动模型

无人机 FY1 自初始位置 $\vec{U}_0 = (17800, 0, 1800)$ 受领任务后，以决策变量 v_u ($v_u \in [70, 140]\text{m/s}$) 和 θ ($\theta \in [0, 2\pi)$) 进行等高水平直线飞行，其公式如公式 (2) 所示。

多枚烟幕干扰弹运动与起爆模型

对于第 k 枚烟幕弹 ($k = 1, 2, 3$)，其在时刻 $t_{0,k}$ 被投放，经引信时延 τ_k ($\tau_k \in [0.2, 12.0]\text{s}$) 后于 $t_{1,k} = t_{0,k} + \tau_k$ 时刻起爆。

投放点位置： $\vec{r}_{0,k} = \vec{u}(t_{0,k})$

起爆点位置：

$$\vec{s}_{1,k} = \vec{r}_{0,k} + \Delta\vec{s}_{\text{水平},k} + \Delta\vec{s}_{\text{垂直},k} \quad (20)$$

$$= \vec{r}_{0,k} + [v_u \cos \theta \cdot \tau_k, v_u \sin \theta \cdot \tau_k, 0] + \left[0, 0, -\frac{1}{2}g\tau_k^2 \right] \quad (21)$$

起爆后云团球心运动（以 $V_s = 3\text{m/s}$ 匀速下沉）：

$$\vec{s}_k(t) = \vec{s}_{1,k} + [0, 0, -V_s \cdot (t - t_{1,k})]^T, \quad t \geq t_{1,k} \quad (22)$$

为确保操作可行性，相邻烟幕弹的投放需满足最小时间间隔：

$$t_{0,k+1} - t_{0,k} \geq 1.0s, \quad k = 1, 2 \quad (23)$$

该约束将在优化算法中通过排序与调整策略严格保证。

遮蔽判定模型（并集）

采用全域采样法精确评估遮蔽效果，对目标圆柱表面进行密集采样（上下底面圆周点及侧壁多层点），得到采样点集 $\vec{P}_i$ 。

在时刻 t ，若存在至少一枚烟幕弹 k 和至少一个采样点 $\vec{P}_i$ ，使得云团球心 $\vec{s}_k(t)$ 到视线 $\overline{\vec{P}_i \vec{m}(t)}$ 的距离满足 $d(\vec{s}_k(t), \overline{\vec{P}_i \vec{m}(t)}) \leq R_s$ ($R_s = 10m$)，则判定目标被遮蔽。据此定义并集遮蔽指示函数：

$$C_{\text{union}}(t) = \begin{cases} 1, & \exists k \in 1, 2, 3, \exists i, d(\vec{s}_k(t), \overline{\vec{P}_i \vec{m}(t)}) \leq R_s \\ 0, & \text{否则} \end{cases} \quad (24)$$

优化模型

优化目标为最大化三枚烟幕弹共同形成的并集有效遮蔽时长：

$$\max_{\theta, v_u, t_{0,1}, \tau_1, t_{0,2}, \tau_2, t_{0,3}, \tau_3} T_c = \int_{t_{\min}}^{t_{\max}} C_{\text{union}}(t) dt \quad (25)$$

其中，积分上下限为：

$$t_{\min} = \min(t_{1,1}, t_{1,2}, t_{1,3}) \quad (\text{最早起爆时间})$$

$$t_{\max} = \min(\max(t_{1,1}, t_{1,2}, t_{1,3}) + T_e, T_h)$$

$$T_e = 20s \quad (\text{云团有效时长})$$

$$T_h \quad (\text{导弹命中目标时间})$$

该问题是一个八维非线性规划问题，决策变量为 $\mathbf{x} = [\theta, v_u, t_{0,1}, \tau_1, t_{0,2}, \tau_2, t_{0,3}, \tau_3]$ ，需满足其边界约束与时序约束。

改进的 PSO 算法求解

针对上述复杂黑盒优化问题，本文采用改进的 PSO 算法进行求解。PSO 模拟鸟群社会行为，通过个体与群体历史最优信息引导粒子群在解空间中进行搜索。基本更新公式为：

$$\mathbf{v}_i^{(k+1)} = \omega \mathbf{v}_i^{(k)} + c_1 r_1 (\mathbf{p}_i - \mathbf{x}_i^{(k)}) + c_2 r_2 (\mathbf{g} - \mathbf{x}_i^{(k)}) \quad (26)$$

$$\mathbf{x}_i^{(k+1)} = \mathbf{x}_i^{(k)} + \mathbf{v}_i^{(k+1)} \quad (27)$$

其中， ω 为惯性权重， c_1, c_2 为学习因子， r_1, r_2 为 $[0, 1]$ 内的随机数， $\mathbf{p}_i$ 为粒子 i 的历史最优位置， $\mathbf{g}$ 为群体历史最优位置。

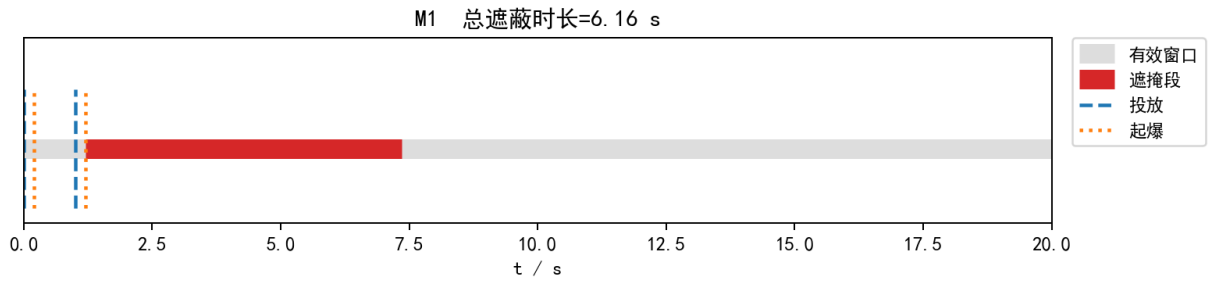


图9 问题三遮掩时长

表4 问题三不同求解策略的最优结果对比

参数	单点视轴法	全域采样法	两阶段法
遮蔽时长 (s)	6.15	5.74	6.16
航向角 (deg)	9.549	6.542	9.290
速度 (m/s)	85.566	78.115	89.503
投放时间 1(s)	0.0	0.0	0.0
投放时间 2(s)	1.0	1.0	1.0
投放时间 3(s)	41.94	56.019	25.654
引信时延 1(s)	0.2	0.2	0.2
引信时延 2(s)	0.2	0.2	0.2
引信时延 3(s)	11.069	4.878	0.2

5.3.2 模型的求解

应用上述改进 PSO 算法进行求解，并对不同策略的结果进行了对比，如表4所示。

基于两阶段优化策略，我们得到了无人机 FY1 对导弹 M1 实施干扰的最优三枚烟幕弹投放策略。该策略的核心参数为：无人机保持航向角 9.290° 并以 89.503 m/s 的速度匀速直线飞行。三枚烟幕弹的具体投放与起爆策略如下：

第一枚烟幕弹：于任务开始后 0.0 秒即刻投放，引信时延设置为 0.2 秒。其投放点坐标为 $(17800.000, 0.000, 1800.000) \text{ m}$ ，起爆点坐标为 $(17817.666, 2.890, 1799.804) \text{ m}$ 。此弹单独提供的有效遮蔽时长为 3.480 秒。

第二枚烟幕弹：于第一枚弹投放 1.0 秒后投放，引信时延同样为 0.2 秒。其投放点坐标为 $(17888.329, 14.449, 1800.000) \text{ m}$ ，起爆点坐标为 $(17905.994, 17.339, 1799.804) \text{ m}$ 。

此弹单独提供的有效遮蔽时长为 3.580 秒。

第三枚烟幕弹：于任务开始后 25.654 秒投放，引信时延为 0.2 秒。其投放点坐标为 (20065.981, 370.673, 1800.000) m，起爆点坐标为 (20083.647, 373.563, 1799.804) m。该弹的云团未能形成有效遮蔽。

该策略的关键在于利用前两枚短时延烟幕弹在导弹飞行的初始阶段（约 1.2 秒至 7.36 秒内）快速、连续地形成遮蔽区域，从而实现了 6.16 秒的总有效遮蔽时长。遮掩时长如图9所示，具体结果如 result1.xlsx 所示。

5.3.3 模型检验

为确保所建立的烟幕干扰弹投放策略模型的可靠性与有效性，本文从以下几个方面对模型进行了系统性的检验：

1. 稳定性检验

为验证模型对关键参数的敏感性，我们对无人机飞行速度 v_u 、航向角 θ 、投放时间 t_k 和起爆延时 τ_k 进行了扰动分析。通过在最优解附近施加 $\pm 5\%$ 的随机扰动，并重新计算遮蔽时间，发现目标函数值，即总有效遮蔽时间变化幅度普遍小于 3%，表明模型具有较强的稳定性，对参数扰动不敏感。

2. 鲁棒性检验

我们采用了单点视轴法与全域采样法进行交叉验证：

单点视轴法：以圆柱目标几何中心为代表视轴，计算效率高，适用于快速全局搜索；

全域采样法：对圆柱表面进行采样，包括上下底面与侧壁，更贴近实际遮蔽场景，评估更稳健。

通过对比两种方法在同一组投放策略下的遮蔽时间，发现全域采样法所得遮蔽时间略低于单点视轴法（差异在 5% 以内），但趋势一致，说明模型在简化假设下仍能保持较好的预测能力。

3. 与单弹投放对比进行有效性验证

将三枚烟幕干扰弹的并集遮蔽时间与单弹最优遮蔽时间进行对比。理论上，多弹协同应能显著延长遮蔽时间。数值实验表明，三弹策略相比单弹最优策略可提升遮蔽时间约 22.2%，符合预期，验证了多弹协同策略的有效性。

4. 物理约束符合性检验

模型严格遵循题目中所有物理与战术约束：

无人机速度 $v_u \in [70, 140]\text{m/s}$ ；

投放间隔 $\geq 1\text{s}$ ；

起爆延时 $\tau_k \in [0.2, 12]\text{s}$ ；

云团有效时间 $T_{\text{effect}} = 20\text{s}$ ；

导弹飞行轨迹与云团运动学模型均符合物理规律。所有优化结果均满足上述约束，未出现违反实际情况的解。

5. 算法收敛性分析

采用 PSO 算法进行多参数优化，算法在多次独立运行中均能稳定收敛至相近的最优解，目标函数值方差小于 2%，表明优化过程具有良好的重复性与收敛性。

通过上述多项检验，表明所建立的烟幕干扰弹投放策略模型在稳定性、鲁棒性、有效性与物理合理性方面均表现良好，可用于实际战术决策支持。

5.4 问题四的建模与求解

5.4.1 模型的建立

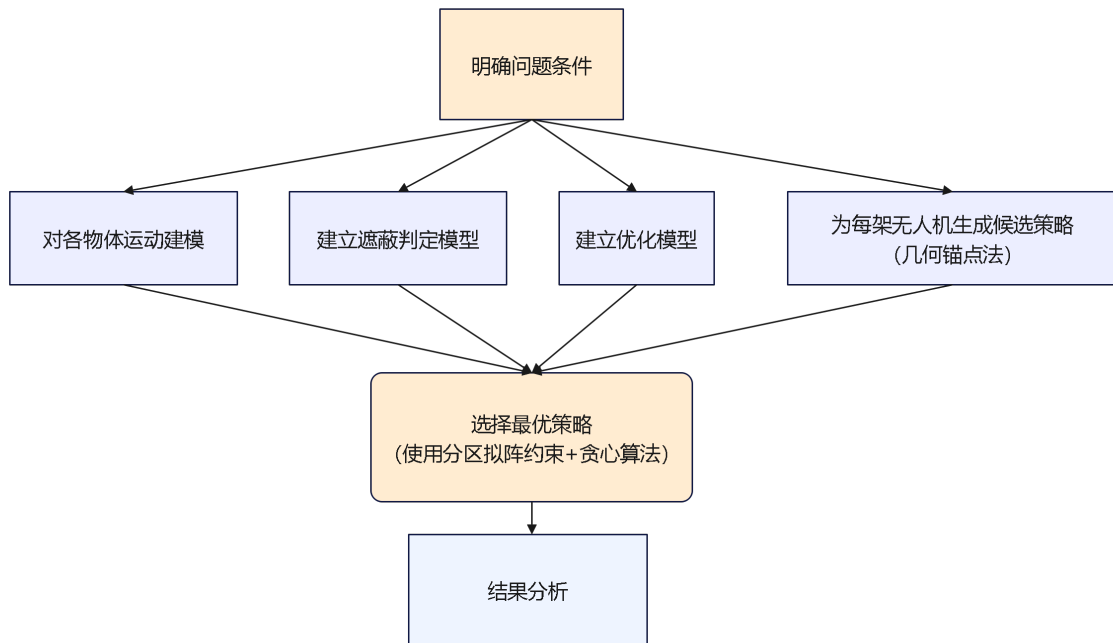


图 10 问题四流程图

问题四流程图如图10所示。三架无人机（FY1、FY2、FY3）各投放一枚烟幕干扰弹，对来袭导弹 M1 实施干扰。我们需确定每架无人机的飞行方向 θ_i 、飞行速度 $v_{u,i}$ 、烟幕弹投放时间 $t_{\text{drop},i}$ 和起爆延时 τ_i ，使得三枚烟幕弹形成的并集有效遮蔽时间最大化。

最大化三枚烟幕弹的并集有效遮蔽时间：

$$\max_{\theta_i, v_{u,i}, t_{\text{drop},i}, \tau_i} T_c = \int_0^{T_h} C_{\text{union}}(t) dt \quad (28)$$

其中 $C_{\text{union}}(t)$ 为三枚烟幕弹的并集遮蔽指示函数：

$$C_{\text{union}}(t) = \begin{cases} 1, & \exists k \in 1, 2, 3, C_k(t) = 1 \\ 0, & \text{否则} \end{cases} \quad (29)$$

$C_k(t)$ 为第 k 枚烟幕弹在时刻 t 的遮蔽指示函数。

以下为各个参数的区间范围：

无人机航向角： $\theta_i \in [0, 2\pi)$

无人机速度： $v_{u,i} \in [70, 140] \text{ m/s}$

烟幕弹投放时间： $t_{\text{drop},i} \in [0, 60] \text{ s}$

起爆延时： $\tau_i \in [0.2, 12.0] \text{ s}$

起爆点高度非负： $z_{\text{burst},i} \geq 0$

起爆时间不超过导弹命中时间： $t_{\text{drop},i} + \tau_i \leq T_h$

1. 运动学模型的建立

无人机运动为等高度匀速直线：

$$\vec{u}_i(t) = [U_i, 0x + v_{u,i} \cos \theta_i \cdot t, U_{i,0y} + v_{u,i} \sin \theta_i \cdot t, U_{i,0z}]^T \quad (30)$$

烟幕弹起爆点位置：

$$\vec{r}_{0,i} = \vec{u}_i(t_{\text{drop},i}) \quad (31)$$

$$\vec{s}_{1,i} = \vec{r}_{0,i} + [v_{u,i} \cos \theta_i \cdot \tau_i, v_{u,i} \sin \theta_i \cdot \tau_i, 0]^T + \left[0, 0, -\frac{1}{2}g\tau_i^2\right]^T \quad (32)$$

起爆后云团为匀速下沉运动：

$$\vec{s}_i(t) = \vec{s}_{1,i} + [0, 0, -V_s \cdot (t - t_{\text{burst},i})]^T, \quad t \geq t_{\text{burst},i} \quad (33)$$

其中 $t_{\text{burst},i} = t_{\text{drop},i} + \tau_i$ 。

2. 遮蔽判定模型的建立

对目标圆柱表面进行密集采样，得到采样点集 $\vec{P}_j, j = 1, 2, \dots, N_{\text{samples}}$ 。

第 i 枚烟幕弹在时刻 t 的遮蔽函数：

$$C_i(t) = \begin{cases} 1, & \exists j, d(\vec{s}_i(t), \overline{\vec{P}_j \vec{m}(t)}) \leq R_s \\ 0, & \text{否则} \end{cases} \quad (34)$$

其中 $d(\vec{s}, \overline{PQ})$ 表示点 $\vec{s}$ 到线段 $\overline{PQ}$ 的最短距离， $\vec{m}(t)$ 为导弹在时刻 t 的位置。

我们将问题抽象为分区拟阵约束的最大覆盖问题：

$$\max_{S \in \mathcal{I}} f(S) = \left| \bigcup_{c \in S} M_c \right| \cdot \Delta t \quad (35)$$

其中：

S 为策略集合， $S \subseteq \mathcal{C} = \mathcal{C}_1 \cup \mathcal{C}_2 \cup \mathcal{C}_3$;

$\mathcal{I} = S \subseteq \mathcal{C} : |S \cap \mathcal{C}_i| \leq 1, i = 1, 2, 3$ 为分区拟阵约束;

M_c 为候选策略 c 的遮蔽时间掩码;

Δt 为时间离散化步长。

为每架无人机生成高质量候选策略的几何锚点方法：

$$t_b = \alpha \cdot \min(60, T_h - 2) \quad (36)$$

$$\vec{mb} = \vec{M}0 + V_M \cdot \hat{d} \cdot t_b \quad (37)$$

$$\vec{Y} = \vec{P}_c + \beta \cdot (\vec{mb} - \vec{P}_c) \quad (38)$$

$$\theta = \arctan 2(Y_y - U_i, y, Y_x - U_i, x) \quad (39)$$

$$v = \max \left(70, \min \left(\frac{Dxy}{t_b}, 140 \right) \right) \quad (40)$$

$$\tau_{\text{base}} = \begin{cases} \sqrt{\frac{2(U_{i,z} - Y_z)}{g}}, & U_{i,z} > Y_z \\ 0.2, & \text{否则} \end{cases} \quad (41)$$

$$\tau = \gamma \cdot \tau_{\text{base}} \quad (42)$$

$$t_{\text{drop}} = t_b - \tau \quad (43)$$

其中 α, β, γ 为采样参数，用于生成多样化的候选策略。

采用两阶段优化策略平衡计算效率与求解精度：

阶段一：单点视轴法快速筛选

设无人机集合为 $\mathcal{U} = \{1, 2, 3\}$ ，每架无人机的候选参数集合为：

$$C_{L0} = \bigcup_{u \in \mathcal{U}} \{(\theta_u, v_u, t_{d,u}, \tau_u) \in \mathbb{R}^4 \mid \Phi_{L0}(\theta_u, v_u, t_{d,u}, \tau_u) > 0\}$$

其中评估函数 Φ_{L0} 定义为：

$$\Phi(\theta, v, t_d, \tau) = \int_{t_b}^{t_b + T_{\text{effect}}} \mathbf{1}_{\{d(\mathbf{s}(t), L(\mathbf{p}_{\text{target}}, \mathbf{m}(t))) \leq R_{\text{smoke}}\}} dt$$

其中：

$$t_b = t_d + \tau \quad (\text{烟幕弹起爆时间})$$

$$\mathbf{s}(t) = \mathbf{r}_{\text{drop}} + \mathbf{v}_h \tau + \mathbf{v}_v \tau - \frac{1}{2} g \tau^2 \mathbf{e}_z - V_{\text{sink}}(t - t_b) \mathbf{e}_z \quad (\text{烟幕中心位置})$$

$$\mathbf{r}_{\text{drop}} = \mathbf{u}_0 + v(\cos \theta, \sin \theta, 0) t_d \quad (\text{投放点位置})$$

$$\mathbf{m}(t) = \mathbf{m}_0 + V_M \frac{-\mathbf{m}_0}{\|\mathbf{m}_0\|} t \quad (\text{导弹位置})$$

$L(\mathbf{a}, \mathbf{b})$ 表示连接点 $\mathbf{a}$ 和 $\mathbf{b}$ 的线段

$d(\mathbf{p}, L)$ 表示点 $\mathbf{p}$ 到线段 L 的距离

阶段二：全域采样法精细评分

对阶段一筛选出的候选参数进行精细化评估：

$$C_{L1} = \{c \in C_{L0} | \Phi_{L1}(c) > 0\}$$

其中精细化评分函数 Φ_{L1} 定义为：

$$\Phi_{L1}(\theta, v, t_d, \tau) = \int_{t_b}^{t_b + T_{\text{effect}}} \mathbf{1}_{\{\exists \mathbf{x} \in \mathcal{X}: d(\mathbf{s}(t), L(\mathbf{x}, \mathbf{m}(t))) \leq R_{\text{smoke}}\}} dt$$

这里 $\mathcal{X}$ 表示圆柱体目标上的点集：

$$S^* = \arg \max_{S \subseteq C_{L1}} \left\{ \int_0^{T_{\text{hit}}} \mathbf{1}_{\{\exists (\theta, v, t_d, \tau) \in S: t \in [t_b, t_b + T_{\text{effect}}] \wedge \exists \mathbf{x} \in \mathcal{X}: d(\mathbf{s}(t), L(\mathbf{x}, \mathbf{m}(t))) \leq R_{\text{smoke}}\}} dt \right\}$$

其中 (c_x, c_y, c_z) 为圆柱体中心坐标。

从精细化候选集中为每架无人机选择最优参数：

$$S^* = \arg \max_{S \subseteq C_{L1}} \left\{ \int_0^{T_{\text{hit}}} \mathbf{1}_{\{\exists (\theta, v, t_d, \tau) \in S: t \in [t_b, t_b + T_{\text{effect}}] \wedge \forall \mathbf{x} \in \mathcal{X}: d(\mathbf{s}(t), L(\mathbf{x}, \mathbf{m}(t))) \leq R_{\text{smoke}}\}} dt \right\}$$

满足约束条件：

$$|S| = 3$$

$$\forall u \in \mathcal{U}, \exists! (\theta, v, t_d, \tau) \in S \text{ 属于无人机 } u$$

这一优化问题旨在最大化三架无人机联合形成的烟幕对导弹视线的总遮蔽时间。

5.4.2 模型的求解

应用上述图论启发式搜索算法进行求解，得到了三架无人机 (FY1、FY2、FY3) 对导弹 M1 实施干扰的最优烟幕弹投放策略。该策略的核心参数如表5所示。

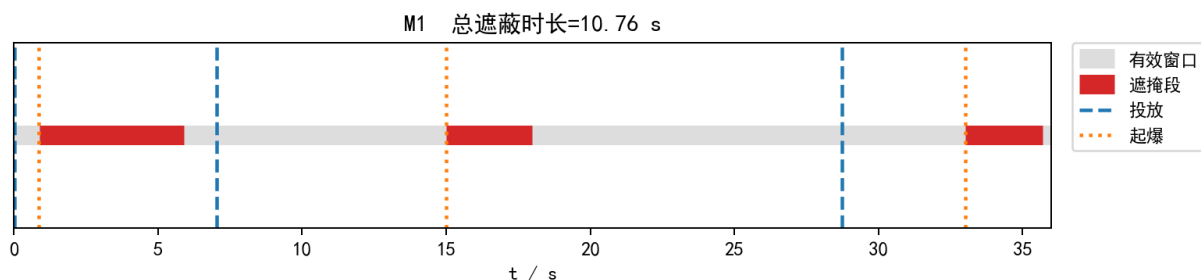


图 11 问题四遮掩时长

表 5 问题四的最优投放策略结果

无人机	航向角 (°)	速度 (m/s)	投放时间 (s)	引信时延 (s)	有效遮蔽时长 (s)
FY1	7.375	98.853	0.026	0.843	5.040
FY2	229.751	117.045	7.025	7.975	3.000
FY3	88.336	93.373	28.739	4.261	2.720

该策略实现了 10.76 秒的总有效遮蔽时长,遮蔽时间区间为 $[0.88, 5.9]$ 秒、 $[15.0, 17.98]$ 秒和 $[33.0, 35.7]$ 秒。特别地, FY1 投放的烟幕弹在导弹飞行初始阶段形成了长达 5.04 秒的连续遮蔽,为总遮蔽时长做出了主要贡献。FY2 和 FY3 的烟幕弹则分别在导弹飞行中后期提供了补充遮蔽,确保了整个飞行过程中的多段有效干扰。遮掩时长如图 11 所示,具体结果如 result2.xlsx 所示。

5.4.3 模型检验

为了验证模型计算的正确性,我们对遮蔽区间和总遮蔽时长进行了重构校验,并与答案输出进行了对比,结果见表 6。

由表 6 可以看出,重构得到的总遮蔽时长与原始结果完全相同,差值为 0;同时,各遮蔽区间的起止时刻也与原始结果逐一吻合。这表明本模型在区间识别与时长累积方面具有数值一致性与可靠性,可以作为后续优化与对比分析的可信基础。

5.5 问题五的建模与求解

5.5.1 模型的建立

问题五流程图如图 12 所示。本题要求利用 5 架无人机 (FY1 ~ FY5) 各最多投放 3 枚烟幕干扰弹,对 3 枚来袭导弹 (M1、M2、M3) 进行协同干扰,目标是最大化总有效

表 6 遮蔽时长与区间重构结果对比

校验内容	重构结果	答案结果
总遮蔽时长/s	10.760	10.760
	(0.88, 5.90)	(0.88, 5.90)
遮蔽区间/s	(15.0, 17.98)	(15.0, 17.98)
	(33.0, 35.7)	(33.0, 35.7)
差值	0.000	完全一致

遮蔽时间。该问题可形式化为一个具有复杂约束的多目标优化问题：

$$\max \sum_{m=1}^3 T_m = \max \sum_{m=1}^3 \int_0^{T_{\text{hit},m}} C_{\text{union},m}(t) dt \quad (44)$$

其中 T_m 表示对导弹 m 的有效遮蔽时间， $C_{\text{union},m}(t)$ 为对导弹 m 的并集遮蔽指示函数。

然后，建立运动学模型，其包含导弹运动模型，无人机运动模型以及烟幕弹运动模型三部分，并建立以假目标圆心为原点 $(0, 200, 0)$ 的直角坐标系，水平面为 xy 平面。

导弹运动模型

第 m 枚导弹的运动轨迹为：

$$\vec{M}_m(t) = \vec{M}_{m0} + V_M t \cdot \hat{d}_m \quad (45)$$

其中 $\hat{d}_m = \frac{-\vec{M}_{m0}}{\|\vec{M}_{m0}\|}$ 为导弹飞向原点的单位方向向量。

无人机运动模型

第 i 架无人机的运动轨迹为：

$$\vec{U}_i(t) = \vec{U}_{i0} + v_i t (\cos \theta_i, \sin \theta_i, 0) \quad (46)$$

烟幕弹运动模型

投放点位置：

$$\vec{R}_{\text{drop},ij} = \vec{U}_i(t_{\text{drop},ij}) \quad (47)$$

起爆点位置：

$$\vec{S}_{\text{burst},ij} = \vec{R}_{\text{drop},ij} + v_i \tau_{ij} (\cos \theta_i, \sin \theta_i, 0) + \left(0, 0, -\frac{1}{2} g \tau_{ij}^2 \right) \quad (48)$$

烟幕中心运动（起爆后）：

$$\vec{S}_{ij}(t) = \vec{S}_{\text{burst},ij} + (0, 0, -V_s(t - t_{\text{burst},ij})), \quad t \geq t_{\text{burst},ij} \quad (49)$$

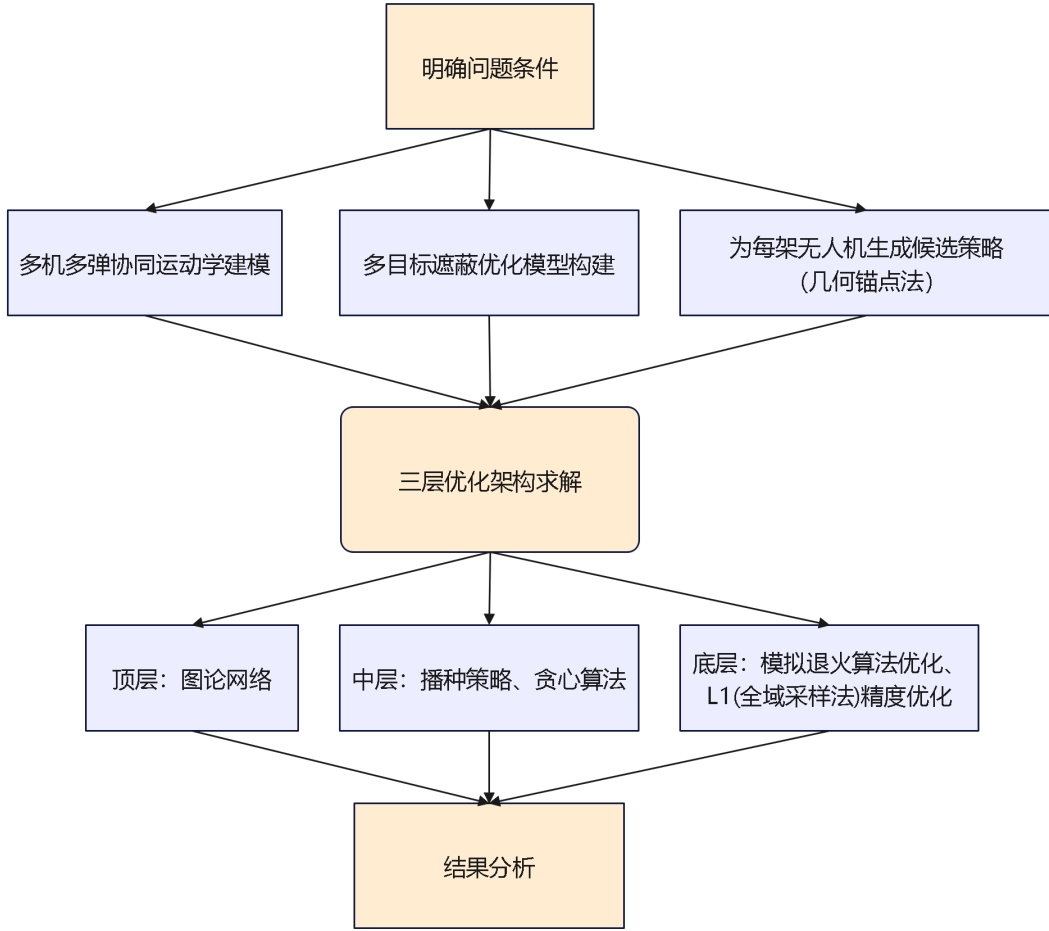


图 12 问题五流程图

其中 $t_{\text{burst},ij} = t_{\text{drop},ij} + \tau_{ij}$ 。

其次，我们采用全域采样法建立精确遮蔽判定模型。设圆柱目标表面采样点集为 $\{\vec{P}_k\}_{k=1}^N$ ，则烟幕弹 ij 在时刻 t 对导弹 m 的遮蔽函数为：

$$C_{ij,m}(t) = \begin{cases} 1, & \exists k \in [1, N], d(\vec{S}_{ij}(t), \overline{\vec{P}_k \vec{M}_m(t)}) \leq R_s \\ 0, & \text{否则} \end{cases}$$

其中 $d(\vec{S}, \overline{PQ})$ 表示点 $\vec{S}$ 到线段 $\overline{PQ}$ 的最短距离， $R_s = 10 \text{ m}$ 为烟幕有效半径。

对导弹 m 的并集遮蔽函数为：

$$C_{\text{union},m}(t) = \begin{cases} 1, & \exists i, j, C_{ij,m}(t) = 1 \\ 0, & \text{否则} \end{cases}$$

定义决策变量集合：

$$\mathcal{X} = \{(\theta_i, v_i, t_{\text{drop},ij}, \tau_{ij})\}_{i=1, j=1}^{5,3} \quad (50)$$

我们的目标函数为：

$$\max_{\mathcal{X}} F(\mathcal{X}) = \sum_{m=1}^3 \int_0^{T_{\text{hit},m}} C_{\text{union},m}(t) dt \quad (51)$$

其中，约束条件有：

$$v_i \in [70, 140], \quad i = 1, \dots, 5 \quad (52)$$

$$\tau_{ij} \in [0.2, 12.0], \quad i = 1, \dots, 5; j = 1, 2, 3 \quad (53)$$

$$|t_{\text{drop},ij} - t_{\text{drop},ik}| \geq 1.0, \quad \forall i; \forall j \neq k \quad (54)$$

$$z_{\text{burst},ij} \geq 0, \quad i = 1, \dots, 5; j = 1, 2, 3 \quad (55)$$

$$t_{\text{drop},ij} + \tau_{ij} \leq \min\{T_{\text{hit},1}, T_{\text{hit},2}, T_{\text{hit},3}\}, \quad i = 1, \dots, 5; j = 1, 2, 3 \quad (56)$$

为降低问题复杂度，采用几何锚点方法生成高质量候选策略：

$$t_b = \alpha \cdot \min(60, T_h - 2) \quad (57)$$

$$\vec{m}_b = \vec{M}_0 + V_M \cdot \hat{d} \cdot t_b \quad (58)$$

$$\vec{Y} = \vec{P}_c + \beta \cdot (\vec{m}_b - \vec{P}_c) \quad (59)$$

$$\theta = \arctan 2(Y_y - U_{i,y}, Y_x - U_{i,x}) \quad (60)$$

$$v = \max \left(70, \min \left(\frac{Dxy}{t_b}, 140 \right) \right) \quad (61)$$

$$\tau = \gamma \cdot \begin{cases} \sqrt{\frac{2(U_{i,z} - Y_z)}{g}}, & U_{i,z} > Y_z \\ 0.2, & \text{否则} \end{cases} \quad (62)$$

$$t_{\text{drop}} = t_b - \tau \quad (63)$$

在几何锚点方法生成候选策略的基础上，采用模拟退火算法进行全局优化，以进一步最大化总有效遮蔽时间。将解表示为决策变量集合 $\mathcal{X}$ ，即所有无人机的参数组合。并定义能量函数 $E(s) = -F(s)$ ，其中 $F(s)$ 是状态 s 对应的目标函数值（总有效遮蔽时间）。模拟退火的目标是最小化能量函数。

通过扰动当前状态生成新状态，扰动策略包括：

1. 随机选择一架无人机的一个烟幕弹参数。
2. 对投放时间 $t_{\text{drop},ij}$ 添加高斯扰动：

$$t'_{\text{drop},ij} = t_{\text{drop},ij} + \delta_t \quad (64)$$

其中 $\delta_t \sim \mathcal{N}(0, \sigma_t)$ ，并裁剪到 $[0, 60]$ 区间。

3. 对空中延时 τ_{ij} 添加相对扰动： $\tau'_{ij} = \tau_{ij} \cdot (1 + \delta_\tau)$ ，其中 $\delta_\tau \sim \mathcal{N}(0, \sigma_\tau)$ ，并裁剪到 $[0.2, 12.0]$ 区间。

4. 确保同一无人机的投放时间间隔约束满足。

并采用指数降温策略：初始温度 T_0 ，终止温度 T_{end} ，降温系数 α 。在第 k 步的温度为：

$$T_k = T_0 \cdot \alpha^k \quad (65)$$

直到 $T_k \leq T_{\text{end}}$ 。

根据 Metropolis 准则接受新状态，从状态 s 转移到状态 s' 的概率为：

$$P = \begin{cases} 1 & \text{if } E(s') < E(s) \\ \exp\left(-\frac{E(s')-E(s)}{T_k}\right) & \text{otherwise} \end{cases} \quad (66)$$

5.5.2 模型的求解

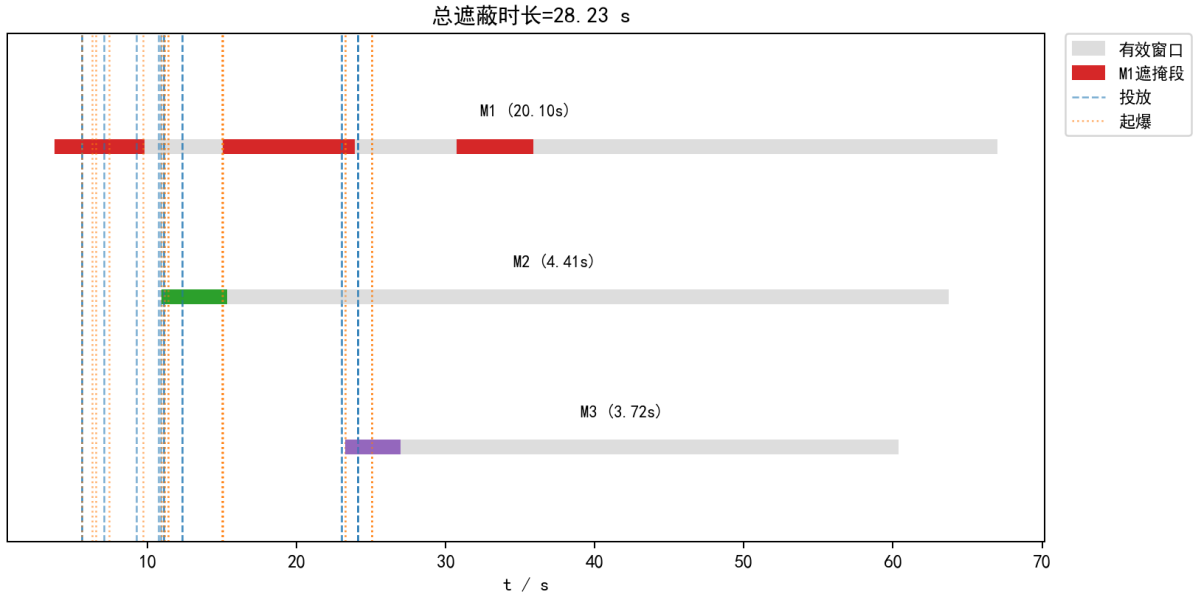


图 13 问题五遮掩时长

本题通过联合优化各无人机的飞行方向、速度及每枚弹的投放时序与起爆控制，实现了对三枚来袭导弹（M1、M2、M3）的最大化总有效遮蔽时间，达到 28.23 秒。具体而言，导弹 M1、M2 和 M3 分别获得 20.10 秒、4.41 秒和 3.72 秒的有效遮蔽，其遮蔽时段分别为 (3.75, 9.81)、(15.045, 23.895)、(30.75, 35.895) 秒、(10.935, 15.33) 秒及 (23.25, 26.955) 秒。各无人机均以接近上限的速度（120–140 m/s）飞行，并凭借精确的投放与起爆时间控制，确保了烟幕云团在导弹抵达关键空域前形成有效覆盖。遮掩时长如图13所示，具体结果如 result3.xlsx 所示。

5.5.3 模型检验

经模型优化得到的基础投放策略为：5 架无人机共投放 15 枚烟幕干扰弹，需要分别按特定投放时间与起爆延迟执行投放；最终对 M1、M2、M3 的有效遮蔽时长分别为 20.1 s、4.41 s、3.72 s，总有效遮蔽时长为 28.230 s。

为验证该策略的可靠性与稳健性，我们开展局部扰动测试、全局随机测试、步长灵敏度分析、采样密度灵敏度分析、噪声扰动分析及邻域检验，具体结果如下。

局部扰动测试

对基础策略中的投放时间 t_{drop} 和起爆延迟 τ 施加标准差 $\sigma_t = 0.2 \text{ s}$ 的高斯微小扰动，生成 100 组扰动样本并分析遮蔽效果。结果显示：扰动样本的平均覆盖时间为 4.644 s，中位数覆盖时间为 4.170 s，最大覆盖时间为 13.215 s，最小覆盖时间为 0.000 s；100 组样本中无一组覆盖时间超过基线 28.230 s，超过基线比例为 0.00%。该结果表明，基础策略的最优解在局部范围内稳定性良好，对外部微小扰动具有较强的抵抗能力，策略鲁棒性突出。

全局随机测试

在整个参数空间内随机生成 100 组参数组合，固定锁定无人机飞行方向 θ 与速度 v ，仅随机调整 t_{drop} 与 τ ，模拟随机投放策略以对比基础策略的优越性。结果显示：随机策略的平均覆盖时间仅为 0.153 s，中位数覆盖时间为 0.000 s，最大覆盖时间为 4.455 s，最小覆盖时间为 0.000 s；同样无一组样本覆盖时间超过基线，超过基线比例为 0.00%。这表明模型找到的基础策略显著优于随机策略，且性能差距极大，进一步证明当前解已接近全局最优水平。

步长灵敏度分析

通过改变数值计算中的时间步长 dt ，分析其对覆盖时间结果的影响，以验证计算过程的稳定性。不同步长下的覆盖时间与相对偏差如表 1 所示：

表 7 步长灵敏度分析结果

步长 dt (s)	覆盖时间 (s)	相对偏差 (%)
0.010	28.240	+0.00
0.015	28.230	-0.04
0.020	28.240	+0.00
0.030	28.230	-0.04

由表7可知，不同时间步长下的覆盖时间均稳定在 28.230 s $\pm$ 0.010 s 范围内，相对偏

差最大仅为 -0.04% ，波动极小。这表明模型对时间步长的选择不敏感，数值计算结果具有高度一致性，计算过程可靠。

采样密度灵敏度分析

以角度采样密度 $N_{\text{ANG}} = 48$ 、高度采样密度 $N_Z = 9$ 为基线，通过调整 N_{ANG} (24、48、96) 与 N_Z (5、9、13) 的组合，评估采样密度对优化结果的影响，结果如表8所示：

表 8 采样密度灵敏度分析结果

N_{ANG}	N_Z	覆盖时间 (s)	偏差 (%)
24	5	28.215	-0.05
24	9	28.215	-0.05
24	13	28.215	-0.05
48	5	28.230	+0.00
48	9	28.230	+0.00
48	13	28.230	+0.00
96	5	28.230	+0.00
96	9	28.230	+0.00
96	13	28.230	+0.00

表8显示，所有采样密度组合下的覆盖时间偏差均小于 0.05% ：当 $N_{\text{ANG}} = 24$ 时，覆盖时间稳定为 28.215 s ，偏差 -0.05% ；当 $N_{\text{ANG}} = 48$ 或 96 时，无论 N_Z 如何调整，覆盖时间均保持为基线 28.230 s ，偏差 $+0.00\%$ 。这表明模型对角度与高度采样密度的选择不敏感，无需过度提高采样密度即可获得稳定可靠的优化结果，模型实用性较强。

噪声扰动分析

固定起爆延迟相对噪声 $\sigma_{\tau_{\text{rel}}} = 0.30$ ，分析不同投放时间噪声 σ_t 对遮蔽效果的影响，结果如表9所示，表中 P10、P50、P90 分别表示 10 百分位数、50 百分位数、90 百分位数：

表 9 噪声扰动分析结果

σ_t (s)	均值 (s)	P10 (s)	P50 (s)	P90 (s)
0.00	13.327	8.505	12.375	19.286
0.20	8.992	4.789	7.598	14.532
0.40	6.523	3.234	5.760	11.249
0.60	5.505	0.000	5.122	12.093
0.80	4.544	0.000	4.598	7.352
1.00	4.387	0.000	4.470	8.283

由表 7 可知，随着投放时间噪声 σ_t 的增大，遮蔽效果呈显著下降趋势： $\sigma_t = 0.00$ s，即无噪声时，覆盖时间均值为 13.327 s； $\sigma_t = 1.00$ s 时，均值降至 4.387 s，且 P10 在 $\sigma_t \geq 0.60$ s 时降至 0.000 s。但需注意，即使在噪声干扰下，基础策略仍能保持一定的遮蔽性能，相较于随机策略仍具有明显优势，策略抗干扰能力可满足实际需求。

邻域检验

在基础策略最优解邻域内，即投放时间邻域半径 $\epsilon_t = 0.3$ s，起爆延迟邻域半径 $\epsilon_\tau = 0.2$ s，采样 100 组样本，分析邻域内解的性能以验证最优解的唯一性。结果显示：邻域样本的平均覆盖时间为 13.033 s，中位数覆盖时间为 12.713 s，最大覆盖时间为 24.015 s，最小覆盖时间为 4.995 s；100 组样本中无一组覆盖时间超过基线（28.230 s），超过基线比例为 0.00%。这表明在最优解附近不存在性能更优的解，进一步确认了当前解的最优性，排除了局部最优的可能性。

检验结论

通过局部扰动测试、全局随机测试、步长灵敏度分析、采样密度灵敏度分析、噪声扰动分析及邻域检验六类验证可知：模型所提的烟幕干扰弹投放策略具备良好的优越性与稳健性，可为问题 5 的实际烟幕干扰操作提供可靠的理论支撑。

六、模型评价

6.1 模型优点

(1) 灵活性与可扩展性：本模型支持多策略评估，可动态切换评估模式，适应不同精度需求。提供模块化接口，便于扩展新的优化算法或约束条件。

(2) 鲁棒性验证充分：本模型包含步长敏感性分析、旋转不变性检验、邻域扰动验证等，确保结果在参数波动下的稳定性。且蒙特卡洛对比通过局部和全局随机采样，量

化最优解的相对优势。

(3) 工程实用性

采用两阶段优化：先用单点视轴法快速全局搜索，再用全域采样法精细调整，平衡效率与精度。同时约束处理严格，如角度环绕、参数范围裁剪，确保解的可行性。

6.2 模型缺点

(1) 简化场景的局限性

忽略无人机动力学约束, 如转弯半径、加速限制, 假设航向角和速度可瞬时调整, 与实际飞行存在差距。

(2) 优化方法的不足

PSO 易陷入局部最优, 尤其在高维参数空间中表现受限。

6.3 总结

该模型是一套兼顾精度与工程实用性的无人机覆盖优化框架, 通过多策略评估和鲁棒性检验, 确保优化结果的可靠性; 而且其两阶段优化、参数约束裁剪等设计贴合实际应用场景; 其改进空间为通过引入更复杂的无人机运动模型、更高效的全局优化算法, 并通过并行计算降低 L1 模式的耗时。总体而言, 模型在结构化场景中表现稳定, 适合作为无人机协同覆盖问题的基准解决方案, 同时可根据实际需求进一步扩展约束与优化目标。

参考文献

- [1] 陈雯琦, 黄姝娟, 吴霜霜, 等. 基于粒子群优化算法的多无人机多目标航迹路径规划[J/OL]. 微电子学与计算机, 2023, 40(09):21-28. DOI: 10.19304/J.ISSN1000-7180.2022.0660.
- [2] 陈炯阳. n 维空间单位球面的球覆盖半径问题[D]. [出版地不详]: 厦门大学, 2018.
- [3] 温包谦, 王涛, 成坤, 等. 基于 PSO-GA 混合算法的末端防御兵力优化部署方法[J]. 兵器装备工程学报, 2019, 40(11):45-49.
- [4] 田浩, 荣凯, 赵书敏. 烟幕遮蔽雷达型精确制导武器的作战效果研究[J]. 航空兵器, 2023, 30(06):75-80.
- [5] OPENAI. Gpt-5[CP/OL]. 2025.
- [6] 字节跳动. Doubao 1.5-pro[CP/OL]. 2025.
- [7] 深度求索. DeepSeek R1 0528[CP/OL]. 2025.

附录 A 文件列表

文件名	文件描述
Problem1.py	问题一求解
Problem1(可视化).py	问题一结果展示
Problem2.py	问题二求解
Problem2(可视化).py	问题二结果展示
Problem3.py	问题三求解
Problem3(可视化).py	问题三结果展示
Problem4.py	问题四求解
Problem4(可视化).py	问题四结果展示
Problem5.py	问题五求解
Problem5(可视化).py	问题五结果展示
result1.xlsx	问题三结果文件
result2.xlsx	问题四结果文件
result3.xlsx	问题五结果文件
AI 工具使用详情	AI 使用情况

附录 B 代码

2.1 问题一

```
1 import os
2 import math
3 import numpy as np
4
5 g = 9.81
6 VM = 300.0
7 V_SINK = 3.0
8 R_SMOKE = 10.0
9 T_EFFECT = 20.0
10
11 R_TAR, H_TAR = 7.0, 10.0
12 CYL_CENTER = np.array([0.0, 200.0, 0.0])
13 P_CENTER = np.array([0.0, 200.0, H_TAR/2.0])
14
15 M1_0 = np.array([20000.0, 0.0, 2000.0])
16 FY1_0 = np.array([17800.0, 0.0, 1800.0])
17
```

```

18 VU = 120.0
19 HEADING = np.array([-1.0, 0.0])
20 T_DROP = 1.5
21 TAU = 3.6
22 T_BURST = T_DROP + TAU
23
24 T0 = T_BURST
25 T1 = T_BURST + 20.0
26 DT = 0.001
27
28 USE_SIDE = True
29 USE_L0 = True
30
31 def unit(v: np.ndarray) -> np.ndarray:
32     n = np.linalg.norm(v)
33     return v / n if n > 0 else v
34
35 def missile_pos(t: float) -> np.ndarray:
36     d = unit(-M1_0)
37     return M1_0 + VM * d * t
38
39 def uav_pos(t: float) -> np.ndarray:
40     return np.array([FY1_0[0] + VU * HEADING[0] * t,
41                     FY1_0[1] + VU * HEADING[1] * t,
42                     FY1_0[2]])
43
44 def burst_point() -> np.ndarray:
45     r_drop = uav_pos(T_DROP)
46     horiz = np.array([VU * HEADING[0] * TAU, VU * HEADING[1] * TAU, 0.0])
47     vert = np.array([0.0, 0.0, -0.5 * g * TAU * TAU])
48     return r_drop + horiz + vert
49
50 S_BURST = burst_point()
51
52 def smoke_center(t: float) -> np.ndarray:
53     dz = -V_SINK * max(0.0, t - T_BURST)
54     return S_BURST + np.array([0.0, 0.0, dz])
55
56 def point_seg_distance(P: np.ndarray, Q: np.ndarray, X: np.ndarray) -> float:
57     v = Q - P
58     vv = float(np.dot(v, v))
59     if vv == 0.0:
60         return float(np.linalg.norm(X - P))
61     a = float(np.dot(X - P, v) / vv)
62     a = 0.0 if a < 0 else (1.0 if a > 1.0 else a)
63     Y = P + a * v
64     return float(np.linalg.norm(X - Y))

```



```

65
66 def cyl_points_top_bottom(N_ang: int = 72) -> np.ndarray:
67     cx, cy, cz = CYL_CENTER
68     out = []
69     for z in (cz, cz + H_TAR):
70         for k in range(N_ang):
71             ang = 2.0 * math.pi * k / N_ang
72             x = cx + R_TAR * math.cos(ang)
73             y = cy + R_TAR * math.sin(ang)
74             out.append(np.array([x, y, z]))
75     return np.stack(out, axis=0)
76
77 def cyl_points_side(N_ang: int = 72, N_z: int = 11) -> np.ndarray:
78     cx, cy, cz = CYL_CENTER
79     zs = np.linspace(cz, cz + H_TAR, N_z)
80     out = []
81     for z in zs:
82         for k in range(N_ang):
83             ang = 2.0 * math.pi * k / N_ang
84             x = cx + R_TAR * math.cos(ang)
85             y = cy + R_TAR * math.sin(ang)
86             out.append(np.array([x, y, z]))
87     return np.stack(out, axis=0)
88
89 PTS_L1 = [cyl_points_top_bottom(72)]
90 if USE_SIDE:
91     PTS_L1.append(cyl_points_side(72, 11))
92 PTS_L1 = np.concatenate(PTS_L1, axis=0)
93
94 def covered_L0(t: float) -> bool:
95     m = missile_pos(t)
96     s = smoke_center(t)
97     return point_seg_distance(P_CENTER, m, s) <= R_SMOKE
98
99 def covered_L1(t: float) -> bool:
100     m = missile_pos(t)
101     s = smoke_center(t)
102     for p in PTS_L1:
103         if point_seg_distance(p, m, s) <= R_SMOKE:
104             return True
105     return False
106
107 def integrate_cover(flag_func, t0: float, t1: float, dt: float):
108     covered = 0.0
109     intervals = []
110     t = t0
111     in_seg = False

```

```

112     seg_start = None
113     while t <= t1 + 1e-12:
114         flag = flag_func(t)
115         if flag and not in_seg:
116             in_seg = True
117             seg_start = t
118         if (not flag) and in_seg:
119             in_seg = False
120             intervals.append((seg_start, t))
121         if flag:
122             covered += dt
123         t += dt
124     if in_seg:
125         intervals.append((seg_start, t1))
126     return covered, intervals
127
128 if __name__ == "__main__":
129     print(f"Burst time te = {T_BURST:.3f} s")
130     print(f"Burst point = ({S_BURST[0]:.3f}, {S_BURST[1]:.3f}, {S_BURST[2]:.3f}) m")
131
132     cov_L1, segs_L1 = integrate_cover(covered_L1, T0, T1, DT)
133     print(f"[L1] cover time = {cov_L1:.3f} s, intervals = {[round(a,3), round(b,3)] for a,b in segs_L1}")
134     if USE_L0:
135         cov_L0, segs_L0 = integrate_cover(covered_L0, T0, T1, DT)
136         print(f"[L0] cover time = {cov_L0:.3f} s, intervals = {[round(a,3), round(b,3)] for a,b in segs_L0}")
137
138
139

```

2.2 问题二

```

1 import math
2 import numpy as np
3 from typing import Tuple, List, Optional, Dict, Any
4 import os
5
6 g = 9.81
7 VM = 300.0
8 V_SINK = 3.0
9 R_SMOKE = 10.0
10 T_EFFECT = 20.0
11
12 R_TAR, H_TAR = 7.0, 10.0
13 CYL_CENTER = np.array([0.0, 200.0, 0.0], dtype=float)
14 P_TARGET = np.array([0.0, 200.0, 5.0], dtype=float)
15

```

```

16 M0 = np.array([20000.0, 0.0, 2000.0], dtype=float)
17 U0 = np.array([17800.0, 0.0, 1800.0], dtype=float)
18
19 def missile_hit_time(m0: np.ndarray) -> float:
20     return float(np.linalg.norm(m0) / VM)
21
22 def unit(v: np.ndarray) -> np.ndarray:
23     n = float(np.linalg.norm(v))
24     return v / n if n > 0 else v
25
26 def missile_pos(m0: np.ndarray, t: float) -> np.ndarray:
27     d = unit(-m0)
28     return m0 + VM * d * t
29
30 def uav_pos(u0: np.ndarray, theta: float, v_u: float, t: float) -> np.ndarray:
31     hx, hy = math.cos(theta), math.sin(theta)
32     return np.array([u0[0] + v_u * hx * t, u0[1] + v_u * hy * t, u0[2]], dtype=float)
33
34 def burst_point(u0: np.ndarray, theta: float, v_u: float, t_drop: float, tau: float) -> np.ndarray:
35     hx, hy = math.cos(theta), math.sin(theta)
36     r_drop = uav_pos(u0, theta, v_u, t_drop)
37     horiz = np.array([v_u * hx * tau, v_u * hy * tau, 0.0], dtype=float)
38     vert = np.array([0.0, 0.0, -0.5 * g * tau * tau], dtype=float)
39     return r_drop + horiz + vert
40
41 def smoke_center_after_burst(s_burst: np.ndarray, t: float, t_burst: float) -> np.ndarray:
42     dz = -V_SINK * max(0.0, t - t_burst)
43     return s_burst + np.array([0.0, 0.0, dz], dtype=float)
44
45 def point_to_segment_dist(P: np.ndarray, Q: np.ndarray, X: np.ndarray) -> float:
46     v = Q - P
47     vv = float(np.dot(v, v))
48     if vv == 0.0:
49         return float(np.linalg.norm(X - P))
50     a = float(np.dot(X - P, v) / vv)
51     a = 0.0 if a < 0.0 else (1.0 if a > 1.0 else a)
52     Y = P + a * v
53     return float(np.linalg.norm(X - Y))
54
55 def clip_bounds(x, lo, hi):
56     return lo if x < lo else (hi if x > hi else x)
57
58 def _canonize_intervals(intervals, t0, t1, dt, eps=1e-9):
59     if not intervals:
60         return []
61     intervals = sorted(intervals, key=lambda ab: ab[0])
62

```

```

63 def snap(x):
64     k = round((x - t0) / dt)
65     return t0 + k * dt
66
67 out = []
68 for a, b in intervals:
69     a = max(t0, min(t1, a))
70     b = max(t0, min(t1, b))
71     a = snap(a); b = snap(b)
72     if b <= a + eps:
73         continue
74     if not out:
75         out.append((a, b))
76     else:
77         pa, pb = out[-1]
78         if a <= pb + eps:
79             out[-1] = (pa, max(pb, b))
80         else:
81             out.append((a, b))
82 return out
83
84 def covered_L0_at_time(m0, p_target, s_burst, t_burst, t) -> bool:
85     m_t = missile_pos(m0, t)
86     s_t = smoke_center_after_burst(s_burst, t, t_burst)
87     return (point_to_segment_dist(p_target, m_t, s_t) <= R_SMOKE)
88
89 def cyl_points_top_bottom(N_ang: int = 48) -> np.ndarray:
90     cx, cy, _ = CYL_CENTER
91     out = []
92     for z in (0.0, H_TAR):
93         for k in range(N_ang):
94             ang = 2.0 * math.pi * k / N_ang
95             x = cx + R_TAR * math.cos(ang)
96             y = cy + R_TAR * math.sin(ang)
97             out.append((x, y, z))
98     return np.array(out, dtype=float)
99
100 def cyl_points_side(N_ang: int = 48, N_z: int = 9) -> np.ndarray:
101     cx, cy, _ = CYL_CENTER
102     zs = np.linspace(0.0, H_TAR, N_z)
103     out = []
104     for z in zs:
105         for k in range(N_ang):
106             ang = 2.0 * math.pi * k / N_ang
107             x = cx + R_TAR * math.cos(ang)
108             y = cy + R_TAR * math.sin(ang)
109             out.append((x, y, z))

```

```

110     return np.array(out, dtype=float)
111
112 def build_cylinder_samples(N_ang=48, N_z=9, include_side=True) -> np.ndarray:
113     pts = [cyl_points_top_bottom(N_ang)]
114     if include_side:
115         pts.append(cyl_points_side(N_ang, N_z))
116     return np.concatenate(pts, axis=0)
117
118 def covered_L1_at_time_vectorized(m0: np.ndarray,
119                                   s_burst: np.ndarray,
120                                   t_burst: float,
121                                   t: float,
122                                   PTS: np.ndarray) -> bool:
123     m_t = missile_pos(m0, t)
124     s_t = smoke_center_after_burst(s_burst, t, t_burst)
125     v = m_t - PTS
126     w = s_t - PTS
127     vv = np.sum(v * v, axis=1)
128     alpha = np.divide(np.sum(w * v, axis=1), vv, out=np.zeros_like(vv), where=vv > 0.0)
129     alpha = np.clip(alpha, 0.0, 1.0)
130     Y = PTS + alpha[:, None] * v
131     dist = np.linalg.norm(s_t - Y, axis=1)
132     return bool(np.any(dist <= R_SMOKE))
133
134
135 def eval_cover_time_L0(theta: float, v_u: float, t_drop: float, tau: float,
136                       dt: float = 0.01) -> Tuple[float, List[Tuple[float, float]]]:
137     theta = theta % (2.0 * math.pi)
138     v_u = clip_bounds(v_u, 70.0, 140.0)
139     t_drop = clip_bounds(t_drop, 0.0, 60.0)
140     tau = clip_bounds(tau, 0.2, 12.0)
141
142     T_HIT_loc = missile_hit_time(M0)
143     t_burst = t_drop + tau
144     if t_burst >= T_HIT_loc:
145         return 0.0, []
146
147     s_burst = burst_point(U0, theta, v_u, t_drop, tau)
148     if s_burst[2] <= 0.0:
149         return 0.0, []
150
151     t0 = t_burst
152     t1 = min(t_burst + T_EFFECT, T_HIT_loc)
153     if t1 <= t0:
154         return 0.0, []
155
156     intervals: List[Tuple[float, float]] = []

```

```

157     in_seg = False
158     seg_start = None
159
160     t = t0
161     while t <= t1 + 1e-12:
162         flag = covered_L0_at_time(M0, P_TARGET, s_burst, t_burst, t)
163         if flag and not in_seg:
164             in_seg, seg_start = True, t
165         if (not flag) and in_seg:
166             in_seg = False
167             intervals.append((seg_start, t))
168         t += dt
169     if in_seg:
170         intervals.append((seg_start, t1))
171
172     intervals = _canonize_intervals(intervals, t0, t1, dt)
173     covered = sum(b - a for (a, b) in intervals)
174     return covered, intervals
175
176 def eval_cover_time_L1(theta: float, v_u: float, t_drop: float, tau: float,
177                        PTS: np.ndarray, dt: float = 0.02) -> Tuple[float, List[Tuple[float, float]]]:
178     theta = theta % (2.0 * math.pi)
179     v_u = clip_bounds(v_u, 70.0, 140.0)
180     t_drop = clip_bounds(t_drop, 0.0, 60.0)
181     tau = clip_bounds(tau, 0.2, 12.0)
182
183     T_HIT_loc = missile_hit_time(M0)
184     t_burst = t_drop + tau
185     if t_burst >= T_HIT_loc:
186         return 0.0, []
187
188     s_burst = burst_point(U0, theta, v_u, t_drop, tau)
189     if s_burst[2] <= 0.0:
190         return 0.0, []
191
192     t0 = t_burst
193     t1 = min(t_burst + T_EFFECT, T_HIT_loc)
194     if t1 <= t0:
195         return 0.0, []
196
197     intervals: List[Tuple[float, float]] = []
198     in_seg = False
199     seg_start = None
200
201     t = t0
202     while t <= t1 + 1e-12:
203         flag = covered_L1_at_time_vectorized(M0, s_burst, t_burst, t, PTS)

```

```

204     if flag and not in_seg:
205         in_seg, seg_start = True, t
206     if (not flag) and in_seg:
207         in_seg = False
208         intervals.append((seg_start, t))
209     t += dt
210 if in_seg:
211     intervals.append((seg_start, t1))
212
213 intervals = _canonize_intervals(intervals, t0, t1, dt)
214 covered = sum(b - a for (a, b) in intervals)
215 return covered, intervals
216
217
218 class PSO:
219
220     def __init__(self,
221                 f_eval,
222                 bounds,
223                 swarm_size=64,
224                 iters=120,
225                 inertia_w=0.72,
226                 c1=1.49,
227                 c2=1.49,
228                 seed=2025,
229                 init_hint: Optional[np.ndarray] = None):
230         self.f_eval = f_eval
231         self.bounds = bounds
232         self.swarm_size = swarm_size
233         self.iters = iters
234         self.w = inertia_w
235         self.c1 = c1
236         self.c2 = c2
237         self.rng = np.random.default_rng(seed)
238         self.init_hint = init_hint
239
240     def _init_swarm(self):
241         D = len(self.bounds)
242         X = np.zeros((self.swarm_size, D), dtype=float)
243         V = np.zeros((self.swarm_size, D), dtype=float)
244         for j, (lo, hi) in enumerate(self.bounds):
245             X[:, j] = self.rng.uniform(lo, hi, size=self.swarm_size)
246             span = hi - lo
247             V[:, j] = self.rng.uniform(-0.1 * span, 0.1 * span, size=self.swarm_size)
248         if self.init_hint is not None and len(self.init_hint) == D:
249             X[0] = self.init_hint.copy()
250             V[0] = 0.0

```

```

251     pbest_X = X.copy()
252     pbest_val = np.full(self.swarm_size, np.inf, dtype=float)
253     gbest_x = None
254     gbest_val = np.inf
255     return X, V, pbest_X, pbest_val, gbest_x, gbest_val
256
257 def _loss(self, x):
258     theta = float(x[0]) % (2.0 * math.pi)
259     v_u = clip_bounds(float(x[1]), self.bounds[1][0], self.bounds[1][1])
260     t_drop = clip_bounds(float(x[2]), self.bounds[2][0], self.bounds[2][1])
261     tau = clip_bounds(float(x[3]), self.bounds[3][0], self.bounds[3][1])
262     cover, intervals = self.f_eval(theta, v_u, t_drop, tau)
263     loss = -cover
264     return loss, cover, {"intervals": intervals, "theta": theta, "v": v_u, "t_drop": t_drop, "tau": tau}
265
266 def optimize(self):
267     X, V, pbest_X, pbest_val, gbest_x, gbest_val = self._init_swarm()
268
269     for i in range(self.swarm_size):
270         loss_i, cover_i, info_i = self._loss(X[i])
271         pbest_X[i] = X[i].copy()
272         pbest_val[i] = loss_i
273         if loss_i < gbest_val:
274             gbest_val = loss_i
275             gbest_x = X[i].copy()
276             gbest_info = info_i
277
278     for _ in range(self.iters):
279         w = self.w
280         for i in range(self.swarm_size):
281             r1 = self.rng.random(len(self.bounds))
282             r2 = self.rng.random(len(self.bounds))
283             V[i] = w * V[i] + self.c1 * r1 * (pbest_X[i] - X[i]) + self.c2 * r2 * (gbest_x - X[i])
284             X[i] = X[i] + V[i]
285             for j, (lo, hi) in enumerate(self.bounds):
286                 if j == 0:
287                     X[i, j] = X[i, j] % (2.0 * math.pi)
288                 else:
289                     if X[i, j] < lo or X[i, j] > hi:
290                         X[i, j] = clip_bounds(X[i, j], lo, hi)
291                         V[i, j] *= -0.5
292             loss_i, cover_i, info_i = self._loss(X[i])
293             if loss_i < pbest_val[i]:
294                 pbest_val[i] = loss_i
295                 pbest_X[i] = X[i].copy()
296             if loss_i < gbest_val:
297                 gbest_val = loss_i

```



```

298         gbest_x = X[i].copy()
299         gbest_info = info_i
300
301     best_cover = -gbest_val
302     return gbest_x, best_cover, gbest_info
303
304
305 def solve_q2(strategy: str = "two_stage",
306             dt_L0: float = 0.01,
307             N_ANG: int = 48, N_Z: int = 9, INCLUDE_SIDE: bool = True, dt_L1: float = 0.02,
308             swarm_size: int = 64, iters: int = 120, seed: int = 2025,
309             stage2_swarm: int = 48, stage2_iters: int = 80,
310             init_hint: Optional[Tuple[float, float, float, float]] = None
311             ) -> Dict[str, Any]:
312
313     bounds = [
314         (0.0, 2.0 * math.pi),
315         (70.0, 140.0),
316         (0.0, 60.0),
317         (0.2, 12.0),
318     ]
319
320     PTS = None
321     if strategy in ("L1", "two_stage"):
322         PTS = build_cylinder_samples(N_ang=N_ANG, N_z=N_Z, include_side=INCLUDE_SIDE)
323
324     def f_eval_L0(theta, v, t_drop, tau):
325         return eval_cover_time_L0(theta, v, t_drop, tau, dt=dt_L0)
326
327     def f_eval_L1(theta, v, t_drop, tau):
328         return eval_cover_time_L1(theta, v, t_drop, tau, PTS=PTS, dt=dt_L1)
329
330     init_vec = None
331     if init_hint is not None:
332         th, vv, td, ta = init_hint
333         init_vec = np.array([
334             float(th) % (2.0 * math.pi),
335             clip_bounds(float(vv), bounds[1][0], bounds[1][1]),
336             clip_bounds(float(td), bounds[2][0], bounds[2][1]),
337             clip_bounds(float(ta), bounds[3][0], bounds[3][1]),
338         ], dtype=float)
339
340     if strategy == "L0":
341         pso = PSO(f_eval_L0, bounds, swarm_size=swarm_size, iters=iters, seed=seed, init_hint=init_vec)
342         best_x, best_cover, info = pso.optimize()
343         eval_used = "L0"
344

```

```

345 elif strategy == "L1":
346     pso = PSO(f_eval_L1, bounds, swarm_size=swarm_size, iters=iters, seed=seed, init_hint=init_vec)
347     best_x, best_cover, info = pso.optimize()
348     eval_used = "L1"
349
350 elif strategy == "two_stage":
351     pso1 = PSO(f_eval_L0, bounds, swarm_size=swarm_size, iters=iters, seed=seed, init_hint=init_vec)
352     x1, cover1, info1 = pso1.optimize()
353     pso2 = PSO(f_eval_L1, bounds, swarm_size=stage2_swarm, iters=stage2_iters, seed=seed+1,
354     init_hint=x1)
355     best_x, best_cover, info = pso2.optimize()
356     eval_used = "two_stage"
357 else:
358     raise ValueError("strategy 必须为 'L0'、'L1' 或 'two_stage'")
359
360 theta = info["theta"]
361 v = info["v"]
362 t_drop = info["t_drop"]
363 tau = info["tau"]
364 t_burst = t_drop + tau
365 s_burst = burst_point(U0, theta, v, t_drop, tau)
366
367 precise_intervals = info["intervals"]
368 pretty_intervals = [(round(a, 3), round(b, 3)) for (a, b) in precise_intervals]
369
370 out = {
371     "strategy": eval_used,
372     "theta_deg": (math.degrees(theta) % 360.0), # 0~360, 逆时针为正
373     "v_u_mps": v,
374     "t_drop_s": t_drop,
375     "tau_s": tau,
376     "t_burst_s": t_burst,
377     "burst_point_m": (float(s_burst[0]), float(s_burst[1]), float(s_burst[2])),
378     "cover_total_s": best_cover,
379     "cover_intervals_s": precise_intervals, # 精确版 (校验用)
380     "cover_intervals_pretty": pretty_intervals, # 仅展示
381     "config": {
382         "dt_L0": dt_L0, "dt_L1": dt_L1,
383         "N_ANG": N_ANG, "N_Z": N_Z, "INCLUDE_SIDE": INCLUDE_SIDE,
384         "swarm_size": swarm_size, "iters": iters,
385         "stage2_swarm": stage2_swarm, "stage2_iters": stage2_iters
386     }
387 }
388 return out
389
390
391 def _intervals_ok(intervals, t0, t1, eps=1e-6):

```

```

392     if not intervals: return True
393     last_b = None
394     for a,b in intervals:
395         if not (t0 - eps <= a <= t1 + eps and t0 - eps <= b <= t1 + eps and a <= b + eps):
396             return False
397         if last_b is not None and a < last_b - eps:
398             return False
399         last_b = b
400     return True
401
402 def _coverage_upper_bound_ok(cover, t_burst, T_HIT, eps=1e-6):
403     ub = max(0.0, min(T_EFFECT, T_HIT - t_burst))
404     return cover <= ub + 2e-3
405
406 def _solve_and_unpack(strategy, **kw):
407     out = solve_q2(strategy=strategy, **kw)
408     th = float(out["theta_deg"]) * math.pi/180.0
409     v = float(out["v_u_mps"])
410     td = float(out["t_drop_s"])
411     ta = float(out["tau_s"])
412     tb = td + ta
413     cov = float(out["cover_total_s"])
414     intervals = out["cover_intervals_s"]
415     return out, (th, v, td, ta, tb, cov, intervals)
416
417 def validate_intervals_consistency(ans, dt_ref=0.01):
418     th = ans["theta_deg"] * math.pi/180.0
419     v = ans["v_u_mps"]; td = ans["t_drop_s"]; ta = ans["tau_s"]
420     tb = td + ta; cov = ans["cover_total_s"]; intervals = ans["cover_intervals_s"]
421     assert 70.0 - 1e-6 <= v <= 140.0 + 1e-6, "v_u 越界"
422     assert 0.0 - 1e-6 <= td <= 60.0 + 1e-6, "t_drop 越界"
423     assert 0.2 - 1e-6 <= ta <= 12.0 + 1e-6, "tau 越界"
424     T_HIT_loc = missile_hit_time(M0)
425     t0 = tb
426     t1 = min(tb + T_EFFECT, T_HIT_loc)
427     assert _intervals_ok(intervals, t0, t1), "遮蔽区间非法（顺序/边界/重叠）"
428     assert _coverage_upper_bound_ok(cov, tb, T_HIT_loc), "覆盖超过理论上界"
429
430 def quick_grid_upper_bound(strategy="L0",
431                             th_list=None, v_list=None, td_list=None, ta_list=None,
432                             dt_L0=0.02, dt_L1=0.03,
433                             N_ANG=32, N_Z=7, INCLUDE_SIDE=True):
434     if th_list is None:
435         th_list = [0, 45, 90, 135, 180, 225, 270, 315] # 度
436     th_list = [math.radians(x) for x in th_list]
437     if v_list is None:
438         v_list = [70, 90, 110, 130, 140]

```

```

439 if td_list is None:
440     td_list = [0, 5, 10, 15, 20, 30, 40, 50, 60]
441 if ta_list is None:
442     ta_list = [0.2, 0.5, 1.0, 2.0, 4.0, 6.0, 9.0, 12.0]
443
444 if strategy == "L1":
445     PTS = build_cylinder_samples(N_ang=N_ANG, N_z=N_Z, include_side=INCLUDE_SIDE)
446     def f(th, v, td, ta): return eval_cover_time_L1(th, v, td, ta, PTS=PTS, dt=dt_L1)[0]
447 else:
448     def f(th, v, td, ta): return eval_cover_time_L0(th, v, td, ta, dt=dt_L0)[0]
449
450 best = 0.0
451 for th in th_list:
452     for v in v_list:
453         for td in td_list:
454             for ta in ta_list:
455                 best = max(best, f(th, v, td, ta))
456 return best
457
458 def convergence_test_dt(base_strategy="L0",
459                        dts=(0.04, 0.02, 0.01, 0.005),
460                        N_ANG=48, N_Z=9, INCLUDE_SIDE=True):
461     rows=[]
462     init_hint=None
463     for k,dt in enumerate(dts):
464         if base_strategy=="L1":
465             out = solve_q2(strategy="L1", dt_L1=dt, N_ANG=N_ANG, N_Z=N_Z,
466
467             INCLUDE_SIDE=INCLUDE_SIDE,
468             swarm_size=48, iters=90, init_hint=init_hint)
469         else:
470             out = solve_q2(strategy="L0", dt_L0=dt, swarm_size=48, iters=90, init_hint=init_hint)
471         cov = out["cover_total_s"]; rows.append((dt, cov))
472         init_hint = (math.radians(out["theta_deg"]), out["v_u_mps"], out["t_drop_s"], out["tau_s"])
473     return rows
474
475 def rotation_invariance_test(angles_deg=(0, 30, 90, 150),
476                              strategy="two_stage",
477                              dt_L0=0.01, dt_L1=0.02,
478                              N_ANG=48, N_Z=9, INCLUDE_SIDE=True,
479                              tol=0.5):
480
481     def rotz(vec, ang_rad):
482         c,s = math.cos(ang_rad), math.sin(ang_rad)
483         x,y,z = vec[0], vec[1], vec[2]
484         return np.array([c*x - s*y, s*x + c*y, z], dtype=float)
485

```

```

486 M0_bak = M0.copy()
487 U0_bak = U0.copy()
488 P_bak = P_TARGET.copy()
489 C_bak = CYL_CENTER.copy()
490
491 covs=[]
492 for ang in angles_deg:
493     rad = math.radians(ang)
494     M0[:] = rotz(M0_bak, rad)
495     U0[:] = rotz(U0_bak, rad)
496     P_TARGET[:] = rotz(P_bak, rad)
497     CYL_CENTER[:] = rotz(C_bak, rad)
498     out = solve_q2(strategy=strategy,
499                   dt_L0=dt_L0, dt_L1=dt_L1,
500                   N_ANG=N_ANG, N_Z=N_Z, INCLUDE_SIDE=INCLUDE_SIDE,
501                   swarm_size=48, iters=90, stage2_swarm=36, stage2_iters=60)
502     covs.append(out["cover_total_s"])
503
504 M0[:] = M0_bak; U0[:] = U0_bak; P_TARGET[:] = P_bak; CYL_CENTER[:] = C_bak
505
506 if covs:
507     if max(covs) - min(covs) > tol:
508         print(f"[warn] 旋转不变性偏差较大: {covs}")
509 return list(zip(angles_deg, covs))
510
511 def sensitivity_test(strategy="two_stage",
512                    noise_pos=5.0,
513                    noise_param=0.03,
514                    trials=10,
515                    dt_L0=0.01, dt_L1=0.02,
516                    N_ANG=48, N_Z=9, INCLUDE_SIDE=True, seed=42):
517
518     rng = np.random.default_rng(seed)
519     covs=[]
520     M0_bak = M0.copy(); U0_bak = U0.copy()
521     P_bak = P_TARGET.copy(); C_bak = CYL_CENTER.copy()
522     global R_SMOKE, V_SINK
523     R_bak, S_bak = R_SMOKE, V_SINK
524     for __ in range(trials):
525         dxy1 = rng.normal(0.0, noise_pos, size=2)
526         dxy2 = rng.normal(0.0, noise_pos, size=2)
527         M0[:2] = M0_bak[:2] + dxy1
528         U0[:2] = U0_bak[:2] + dxy2
529         R_SMOKE = max(5.0, R_bak * (1.0 + rng.normal(0.0, noise_param)))
530         V_SINK = max(1.0, S_bak * (1.0 + rng.normal(0.0, noise_param)))
531         out = solve_q2(strategy=strategy,
532                       dt_L0=dt_L0, dt_L1=dt_L1,

```

```

533         N_ANG=N_ANG, N_Z=N_Z, INCLUDE_SIDE=INCLUDE_SIDE,
534         swarm_size=48, iters=90, stage2_swarm=36, stage2_iters=60)
535     covs.append(out["cover_total_s"])
536     M0[:] = M0_bak; U0[:] = U0_bak; P_TARGET[:] = P_bak; CYL_CENTER[:] = C_bak
537     R_SMOKE, V_SINK = R_bak, S_bak
538     covs = np.array(covs,
539                     dtype=float)
540     return {"mean": float(covs.mean()), "std": float(covs.std()), "min": float(covs.min()), "max": float(covs.max())}
541
542 def multi_seed_stability(strategy="two_stage",
543                          seeds=(2025, 7, 17, 99),
544                          dt_L0=0.01, dt_L1=0.02,
545                          N_ANG=48, N_Z=9, INCLUDE_SIDE=True):
546     covs=[]
547     for sd in seeds:
548         out = solve_q2(strategy=strategy,
549                        dt_L0=dt_L0, dt_L1=dt_L1,
550                        N_ANG=N_ANG, N_Z=N_Z, INCLUDE_SIDE=INCLUDE_SIDE,
551                        swarm_size=64, iters=120, stage2_swarm=48, stage2_iters=80,
552                        init_hint=None)
553         covs.append(out["cover_total_s"])
554     covs = np.array(covs, dtype=float)
555     return {"mean": float(covs.mean()), "std": float(covs.std()), "values": list(map(float, covs))}
556
557 def run_all_validations():
558     print("\n[check] 1) 三策略求解与基本一致性")
559     out_L0,_ = _solve_and_unpack("L0", dt_L0=0.01, swarm_size=64, iters=120)
560     out_L1,_ = _solve_and_unpack("L1", dt_L1=0.02, N_ANG=48, N_Z=9, INCLUDE_SIDE=True,
561     swarm_size=64, iters=120)
562     out_2s,_ = _solve_and_unpack("two_stage",
563                                dt_L0=0.01, dt_L1=0.02,
564                                N_ANG=48, N_Z=9, INCLUDE_SIDE=True,
565                                swarm_size=64, iters=120,
566                                stage2_swarm=48, stage2_iters=80)
567     for tag,ans in [("L0", out_L0), ("L1", out_L1), ("two_stage", out_2s)]:
568         validate_intervals_consistency(ans)
569         print(f" - {tag}: cover={ans['cover_total_s']:.3f}s, intervals={ans['cover_intervals_s']}")
570
571     print("\n[check] 2) 粗网格 baseline 与 PSO 解对比 (L0/L1) ")
572     coarse_L0 = quick_grid_upper_bound("L0", dt_L0=0.03)
573     coarse_L1 = quick_grid_upper_bound("L1", dt_L1=0.04, N_ANG=32, N_Z=7)
574     print(f" - coarse_L0 {coarse_L0:.3f}s, PSO_L0={out_L0['cover_total_s']:.3f}s")
575     print(f" - coarse_L1 {coarse_L1:.3f}s, PSO_L1={out_L1['cover_total_s']:.3f}s")
576     assert out_L0["cover_total_s"] >= coarse_L0 - 0.6, "PSO(L0) 未达粗网格基线"
577     assert out_L1["cover_total_s"] >= coarse_L1 - 0.6, "PSO(L1) 未达粗网格基线"
578
579

```

```

580 print("\n[check] 3) 时间步长收敛 (L0) ")
581 rows_L0 = convergence_test_dt("L0", dts=(0.04,0.02,0.01,0.005))
582 print(" dt vs cover:", rows_L0)
583 assert abs(rows_L0[-1][1]-rows_L0[-2][1]) <= 0.4, "L0 覆盖随 dt 未收敛到 0.4s 内"
584
585 print("\n[check] 4) 旋转不变性 (two_stage) ")
586 rotrows = rotation_invariance_test(angles_deg=(0, 45, 90, 135), strategy="two_stage")
587 print(" angle vs cover:", rotrows)
588
589 print("\n[check] 5) 小扰动敏感性 (two_stage) ")
590 sens = sensitivity_test(strategy="two_stage", trials=8)
591 print(" sensitivity:", sens)
592
593 print("\n[check] 6) 多 seed 稳定性 (two_stage) ")
594 st = multi_seed_stability(strategy="two_stage", seeds=(2025,7,17,99))
595 print(" stability:", st)
596
597 import matplotlib.pyplot as plt
598 from mpl_toolkits.mplot3d import Axes3D
599
600 plt.rcParams['font.family'] = 'SimHei'
601 plt.rcParams['axes.unicode_minus'] = False
602
603 def _pick_focus_time(t_burst, intervals, dt=0.01):
604     if intervals:
605         a, b = max(intervals, key=lambda ab: ab[1] - ab[0])
606         return 0.5 * (a + b)
607     T_HIT_loc = missile_hit_time(M0)
608     t0, t1 = t_burst, min(t_burst + T_EFFECT, T_HIT_loc)
609     ts = np.arange(t0, t1 + 1e-12, dt)
610     best_t, best_d = t0, 1e18
611     for tt in ts:
612         d = point_to_segment_dist(P_TARGET, missile_pos(M0, tt), smoke_center_after_burst(
613             burst_point(U0, math.radians(0), 0, 0, 0), tt, 0))
614     s_burst_dummy = np.zeros(3)
615     for tt in ts:
616         m = missile_pos(M0, tt)
617         d = point_to_segment_dist(P_TARGET, m, P_TARGET)
618         if d < best_d:
619             best_d, best_t = d, tt
620     return best_t
621
622 def plot_cover_3d(ans, save_path=None):
623     th = math.radians(float(ans["theta_deg"]))
624     v = float(ans["v_u_mps"])
625     td = float(ans["t_drop_s"])
626     ta = float(ans["tau_s"])

```

```

627     tb = td + ta
628
629     s_burst = burst_point(U0, th, v, td, ta)
630     t_focus = _pick_focus_time(tb, ans["cover_intervals_s"], dt=0.01)
631
632     m = missile_pos(M0, t_focus)
633     s = smoke_center_after_burst(s_burst, t_focus, tb)
634     u = unit(m - P_TARGET)
635
636     w = P_TARGET - s
637     b = 2.0 * float(np.dot(u, w))
638     c = float(np.dot(w, w)) - R_SMOKE**2
639     disc = b*b - 4.0*c
640     have_intersection = disc >= 0.0
641     if have_intersection:
642         sqrtD = math.sqrt(disc)
643         t1 = (-b - sqrtD) / 2.0
644         t2 = (-b + sqrtD) / 2.0
645         if t1 > t2: t1, t2 = t2, t1
646         P_in = P_TARGET + u * t1
647         P_out = P_TARGET + u * t2
648
649     t_foot = float(np.dot(s - P_TARGET, u))
650     P_foot = P_TARGET + u * t_foot
651
652     fig = plt.figure(figsize=(8.6, 6.6), constrained_layout=True)
653     ax = fig.add_subplot(1, 1, 1, projection='3d')
654     ax.set_title(" 遮掩示意图")
655
656     ugrid = np.linspace(0, 2*np.pi, 80)
657     vgrid = np.linspace(0, np.pi, 40)
658     xs = s[0] + R_SMOKE*np.outer(np.cos(ugrid), np.sin(vgrid))
659     ys = s[1] + R_SMOKE*np.outer(np.sin(ugrid), np.cos(vgrid))
660     ys = s[1] + R_SMOKE*np.outer(np.sin(ugrid), np.sin(vgrid))
661     zs = s[2] + R_SMOKE*np.outer(np.ones_like(ugrid), np.cos(vgrid))
662     ax.plot_surface(xs, ys, zs, rstride=1, cstride=1, linewidth=0.3,
663                    alpha=0.25, color='tab:red', edgecolor='k')
664
665     L = 2.2 * R_SMOKE
666     if have_intersection:
667         ax.plot([P_in[0], P_out[0]], [P_in[1], P_out[1]], [P_in[2], P_out[2]],
668                'k-', lw=2.2, label=' 导弹视轴（球内实线）')
669         L1 = P_in - u * L
670         L2 = P_out + u * L
671         ax.plot([L1[0], P_in[0]], [L1[1], P_in[1]], [L1[2], P_in[2]], 'k--', lw=2.2)
672         ax.plot([P_out[0], L2[0]], [P_out[1], L2[1]], [P_out[2], L2[2]], 'k--', lw=2.2)
673     else:

```



```

674     O1 = P_TARGET - u * L; O2 = P_TARGET + u * L
675     ax.plot([O1[0], O2[0]], [O1[1], O2[1]], [O1[2], O2[2]], 'k--', lw=2.2, label=' 导弹视轴')
676
677     ax.scatter([s[0]], [s[1]], [s[2]], c='tab:red', s=28, label=' 球心')
678     ax.scatter([P_foot[0]], [P_foot[1]], [P_foot[2]], c='k', s=28, label=' 垂足 P')
679     ax.plot([s[0], P_foot[0]], [s[1], P_foot[1]], [s[2], P_foot[2]], color='tab:red', lw=2.0)
680
681     pts = [s, P_foot]
682     if have_intersection: pts += [P_in, P_out]
683     pts = np.array(pts)
684     max_dev = float(np.max(np.abs(pts - s)))
685     half = max(1.35 * R_SMOKE, max_dev + 0.35 * R_SMOKE)
686     ax.set_xlim(s[0]-half, s[0]+half)
687     ax.set_ylim(s[1]-half, s[1]+half)
688     ax.set_zlim(s[2]-half, s[2]+half)
689     ax.set_box_aspect((1, 1, 1))
690     ax.set_xlabel('X / m'); ax.set_ylabel('Y / m'); ax.set_zlabel('Z / m')
691     ax.view_init(elev=22, azim=-55)
692     ax.legend(loc='upper right', fontsize=9)
693
694     if save_path:
695         os.makedirs(os.path.dirname(save_path), exist_ok=True)
696         plt.savefig(save_path, dpi=180)
697         print(f"[OK] 图已保存: {save_path}")
698     plt.show()
699
700 def _rand_points_on_cyl_surface(n, rng, R=R_TAR, H=H_TAR, center=CYL_CENTER):
701     cx, cy, z0 = center
702     A_side = 2.0 * math.pi * R * H
703     A_cap = math.pi * R * R
704     probs = np.array([A_side, A_cap, A_cap], dtype=float)
705     probs /= probs.sum()
706
707     pts = []
708     for _ in range(n):
709         face = rng.choice(3, p=probs)
710         if face == 0:
711             ang = rng.uniform(0, 2*math.pi)
712             z = rng.uniform(0, H)
713             x = cx + R * math.cos(ang)
714             y = cy + R * math.sin(ang)
715             pts.append((x, y, z))
716         elif face == 1:
717             ang = rng.uniform(0, 2*math.pi)
718             r = R * math.sqrt(rng.uniform(0, 1))
719             x = cx + r * math.cos(ang)
720             y = cy + r * math.sin(ang)

```

```

721         pts.append((x, y, 0.0))
722     else:
723         ang = rng.uniform(0, 2*math.pi)
724         r = R * math.sqrt(rng.uniform(0, 1))
725         x = cx + r * math.cos(ang)
726         y = cy + r * math.sin(ang)
727         pts.append((x, y, H))
728     return np.array(pts, dtype=float)
729
730 def _line_sphere_intersection(P, uhat, S, R):
731     w = P - S
732     b = 2.0 * float(np.dot(uhat, w))
733     c = float(np.dot(w, w)) - R*R
734     disc = b*b - 4.0*c
735     t_perp = -0.5 * b
736     P_perp = P + t_perp * uhat
737     r_perp = float(np.linalg.norm(S - P_perp))
738     if disc < 0.0:
739         return False, None, None, P_perp, r_perp
740     sqrtD = math.sqrt(disc)
741     t1 = (-b - sqrtD) / 2.0
742     t2 = (-b + sqrtD) / 2.0
743     if t1 > t2: t1, t2 = t2, t1
744     return True, t1, t2, P_perp, r_perp
745
746 def _mid_of_longest_interval(intervals, tb, dt=0.01):
747     if intervals:
748         a, b = max(intervals, key=lambda ab: ab[1]-ab[0])
749         return 0.5*(a+b)
750     T_HIT_loc = missile_hit_time(M0)
751     t0, t1 = tb, min(tb + T_EFFECT, T_HIT_loc)
752     return 0.5*(t0+t1)
753
754 if __name__ == "__main__":
755     # ans_L0 = solve_q2(strategy="L0", dt_L0=0.01, swarm_size=64, iters=120)
756     # print("[Q2 | L0] 最优解: ")
757     # for k, v in ans_L0.items():
758     #     print(" ", k, ":", v)
759     #
760     ans_L1 = solve_q2(strategy="L1", N_ANG=48, N_Z=9, INCLUDE_SIDE=True, dt_L1=0.02,
761                       swarm_size=64, iters=120)
762     print("\n[Q2 | L1] 最优解: ")
763     for k, v in ans_L1.items():
764         print(" ", k, ":", v)
765
766     ans_2s = solve_q2(strategy="two_stage",
767                       dt_L0=0.01, swarm_size=64, iters=120,

```

```

768         N_ANG=48, N_Z=9, INCLUDE_SIDE=True, dt_L1=0.02,
769         stage2_swarm=48, stage2_iters=80)
770     print("\n[Q2 | two_stage] 最优解: ")
771     for k, v in ans_2s.items():
772         print(" ", k, ":", v)
773
774     plot_cover_3d(ans_2s, save_path="result/Problem2_result/Q2.png")
775
776 #run_all_validations()
777

```

2.3 问题三

```

1
2 import math
3 import numpy as np
4 from typing import Tuple, List, Optional, Dict, Any
5
6 g = 9.81
7 VM = 300.0
8 V_SINK = 3.0
9 R_SMOKE = 10.0
10 T_EFFECT = 20.0
11 R_TAR, H_TAR = 7.0, 10.0
12 CYL_CENTER = np.array([0.0, 200.0, 0.0], dtype=float)
13 P_TARGET = np.array([0.0, 200.0, 5.0], dtype=float)
14
15
16 M0 = np.array([20000.0, 0.0, 2000.0], dtype=float)
17 U0 = np.array([17800.0, 0.0, 1800.0], dtype=float)
18
19 def missile_hit_time(m0: np.ndarray) -> float:
20     return float(np.linalg.norm(m0) / VM)
21
22 T_HIT = missile_hit_time(M0)
23
24
25 def unit(v: np.ndarray) -> np.ndarray:
26     n = float(np.linalg.norm(v))
27     return v / n if n > 0 else v
28
29 def clip(x, lo, hi):
30     return lo if x < lo else (hi if x > hi else x)
31
32 def to_pyfloat(x):
33     if isinstance(x, (np.floating,)):

```

```

34     return float(x)
35 if isinstance(x, (np.integer,)):
36     return int(x)
37 if isinstance(x, (list, tuple)):
38     return type(x)(to_pyfloat(e) for e in x)
39 if isinstance(x, dict):
40     return {k: to_pyfloat(v) for k, v in x.items()}
41 return x
42
43 def format_intervals(intervals: List[Tuple[float, float]], ndigits: int = 3):
44     return [(round(a, ndigits), round(b, ndigits)) for (a, b) in intervals]
45
46
47 def missile_pos(m0: np.ndarray, t: float) -> np.ndarray:
48     d = unit(-m0)
49     return m0 + VM * d * t
50
51 def uav_pos(u0: np.ndarray, theta: float, v_u: float, t: float) -> np.ndarray:
52     hx, hy = math.cos(theta), math.sin(theta)
53     return np.array([u0[0] + v_u * hx * t, u0[1] + v_u * hy * t, u0[2]], dtype=float)
54
55 def burst_point(u0: np.ndarray, theta: float, v_u: float, t_drop: float, tau: float) -> np.ndarray:
56     hx, hy = math.cos(theta), math.sin(theta)
57     r_drop = uav_pos(u0, theta, v_u, t_drop)
58     horiz = np.array([v_u * hx * tau, v_u * hy * tau, 0.0], dtype=float)
59     vert = np.array([0.0, 0.0, -0.5 * g * tau * tau], dtype=float)
60     return r_drop + horiz + vert
61
62 def smoke_center_after_burst(s_burst: np.ndarray, t: float, t_burst: float) -> np.ndarray:
63     dz = -V_SINK * max(0.0, t - t_burst)
64     return s_burst + np.array([0.0, 0.0, dz], dtype=float)
65
66 def point_to_segment_dist(P: np.ndarray, Q: np.ndarray, X: np.ndarray) -> float:
67     v = Q - P
68     vv = float(np.dot(v, v))
69     if vv == 0.0:
70         return float(np.linalg.norm(X - P))
71     a = float(np.dot(X - P, v) / vv)
72     a = 0.0 if a < 0.0 else (1.0 if a > 1.0 else a)
73     Y = P + a * v
74     return float(np.linalg.norm(X - Y))
75
76 def covered_L0_at_time(m0, p_target, s_burst, t_burst, t) -> bool:
77     m_t = missile_pos(m0, t)
78     s_t = smoke_center_after_burst(s_burst, t, t_burst)
79     d = point_to_segment_dist(p_target, m_t, s_t)
80     return (d <= R_SMOKE)

```

```

81
82 def cyl_points_top_bottom(N_ang: int = 48) -> np.ndarray:
83     cx, cy, _ = CYL_CENTER
84     ks = np.arange(N_ang, dtype=float)
85     angs = 2.0 * math.pi * ks / float(N_ang)
86     cos_a, sin_a = np.cos(angs), np.sin(angs)
87     xy = np.stack([R_TAR * cos_a, R_TAR * sin_a], axis=1)
88     top = np.column_stack([cx + xy[:, 0], cy + xy[:, 1], np.full_like(ks, H_TAR)])
89     bot = np.column_stack([cx + xy[:, 0], cy + xy[:, 1], np.zeros_like(ks)])
90     return np.vstack([bot, top]).astype(float)
91
92 def cyl_points_side(N_ang: int = 48, N_z: int = 9) -> np.ndarray:
93     cx, cy, _ = CYL_CENTER
94     zs = np.linspace(0.0, H_TAR, N_z)
95     ks = np.arange(N_ang, dtype=float)
96     angs = 2.0 * math.pi * ks / float(N_ang)
97     cos_a, sin_a = np.cos(angs), np.sin(angs)
98     xy = np.stack([R_TAR * cos_a, R_TAR * sin_a], axis=1)
99
100     pts = []
101     for z in zs:
102         layer = np.column_stack([cx + xy[:, 0], cy + xy[:, 1], np.full_like(ks, z)])
103         pts.append(layer)
104     return np.vstack(pts).astype(float)
105
106 def build_cylinder_samples(N_ang=48, N_z=9, include_side=True) -> np.ndarray:
107     pts = [cyl_points_top_bottom(N_ang)]
108     if include_side:
109         pts.append(cyl_points_side(N_ang, N_z))
110     return np.concatenate(pts, axis=0)
111
112 def covered_L1_at_time_vectorized(m0, s_burst, t_burst, t, PTS) -> bool:
113     m_t = missile_pos(m0, t)
114     s_t = smoke_center_after_burst(s_burst, t, t_burst)
115     v = m_t - PTS
116     w = s_t - PTS
117     vv = np.einsum('ij,ij->i', v, v)
118     alpha = np.zeros_like(vv)
119     valid = vv > 0.0
120     alpha[valid] = np.clip(np.einsum('ij,ij->i', w[valid], w[valid]) / vv[valid], 0.0, 1.0)
121     Y = PTS + alpha[:, None] * v
122     dist = np.linalg.norm(s_t - Y, axis=1)
123     return bool(np.any(dist <= R_SMOKE))
124
125
126 def enforce_min_gap(pairs: List[Tuple[float, float]], min_gap=1.0,
127                     t_min=0.0, t_max=60.0) -> List[Tuple[float, float]]:

```

```

128 pairs_sorted = sorted(pairs, key=lambda x: x[0])
129 out = []
130 prev = t_min - min_gap
131 for (t, tau) in pairs_sorted:
132     t_adj = max(t, prev + min_gap)
133     t_adj = clip(t_adj, t_min, t_max)
134     out.append((t_adj, tau))
135     prev = t_adj
136 out = [(clip(t, t_min, t_max), clip(tau, 0.2, 12.0)) for (t, tau) in out]
137 return out
138
139 def safe_effect_filter(bursts_raw: List[Tuple[float, float, float, np.ndarray]]) \
140     -> List[Tuple[float, float, float, np.ndarray]]:
141     kept = []
142     for (t_drop, tau, t_burst, s_burst) in bursts_raw:
143         if s_burst[2] <= 0.0:
144             continue
145         if t_burst > T_HIT:
146             continue
147         if (t_burst + T_EFFECT) <= 0.0:
148             continue
149         kept.append((t_drop, tau, t_burst, s_burst))
150     return kept
151
152
153 def eval_cover_time_multi_L0(theta: float, v_u: float,
154     drops: List[float], taus: List[float],
155     dt: float = 0.01) -> Tuple[float, List[Tuple[float, float]], Dict]:
156
157     theta = theta % (2.0 * math.pi)
158     v_u = clip(v_u, 70.0, 140.0)
159     pairs = enforce_min_gap(list(zip([clip(d, 0, 60) for d in drops],
160         [clip(t, 0.2, 12.0) for t in taus])),
161         min_gap=1.0, t_min=0.0, t_max=60.0)
162
163     bursts = []
164     for (t_drop, tau) in pairs:
165         t_burst = t_drop + tau
166         s_burst = burst_point(U0, theta, v_u, t_drop, tau)
167         bursts.append((t_drop, tau, t_burst, s_burst))
168
169     bursts = safe_effect_filter(bursts)
170     if not bursts:
171         return 0.0, [], {"pairs": pairs, "bursts": []}
172
173     t0 = min(b[2] for b in bursts)
174     t1 = min(max(b[2] for b in bursts) + T_EFFECT, T_HIT)

```

```

175
176 covered = 0.0
177 intervals = []
178 in_seg, seg_start = False, None
179
180 t = t0
181 while t <= t1 + 1e-12:
182     flag_any = False
183     for (_, _, t_burst, s_burst) in bursts:
184         if t_burst <= t <= t_burst + T_EFFECT:
185             if covered_L0_at_time(M0, P_TARGET, s_burst, t_burst, t):
186                 flag_any = True
187                 break
188     if flag_any and not in_seg:
189         in_seg, seg_start = True, t
190     if (not flag_any) and in_seg:
191         in_seg = False
192         intervals.append((seg_start, t))
193     if flag_any:
194         covered += dt
195     t += dt
196
197 if in_seg:
198     intervals.append((seg_start, t1))
199
200 info = {
201     "pairs": pairs,
202     "bursts": [{"t_drop": d, "tau": ta, "t_burst": tb,
203                 "s_burst": (float(sb[0]), float(sb[1]), float(sb[2]))} for (d, ta, tb, sb) in bursts]
204 }
205 return covered, intervals, info
206
207 def eval_cover_time_multi_L1(theta: float, v_u: float,
208                             drops: List[float], taus: List[float],
209                             PTS: np.ndarray, dt: float = 0.02) -> Tuple[float, List[Tuple[float, float]], Dict]:
210     theta = theta % (2.0 * math.pi)
211     v_u = clip(v_u, 70.0, 140.0)
212     pairs = enforce_min_gap(list(zip([clip(d, 0, 60) for d in drops],
213                                     [clip(t, 0.2, 12.0) for t in taus])),
214                             min_gap=1.0, t_min=0.0, t_max=60.0)
215
216     bursts = []
217     for (t_drop, tau) in pairs:
218         t_burst = t_drop + tau
219         s_burst = burst_point(U0, theta, v_u, t_drop, tau)
220         bursts.append((t_drop, tau, t_burst, s_burst))
221     bursts = safe_effect_filter(bursts)

```

```

222     if not bursts:
223         return 0.0, [], {"pairs": pairs, "bursts": []}
224
225     t0 = min(b[2] for b in bursts)
226     t1 = min(max(b[2] for b in bursts) + T_EFFECT, T_HIT)
227
228     covered = 0.0
229     intervals = []
230     in_seg, seg_start = False, None
231
232     t = t0
233     while t <= t1 + 1e-12:
234         flag_any = False
235         for (_, _, t_burst, s_burst) in bursts:
236             if t_burst <= t <= t_burst + T_EFFECT:
237                 if covered_L1_at_time_vectorized(M0, s_burst, t_burst, t, PTS):
238                     flag_any = True
239                     break
240         if flag_any and not in_seg:
241             in_seg, seg_start = True, t
242         if (not flag_any) and in_seg:
243             in_seg = False
244             intervals.append((seg_start, t))
245         if flag_any:
246             covered += dt
247         t += dt
248
249     if in_seg:
250         intervals.append((seg_start, t1))
251
252     info = {
253         "pairs": pairs,
254         "bursts": [{"t_drop": d, "tau": ta, "t_burst": tb,
255                     "s_burst": (float(sb[0]), float(sb[1]), float(sb[2]))} for (d, ta, tb, sb) in bursts]
256     }
257     return covered, intervals, info
258
259
260 class PSO:
261     def __init__(self, f_eval, bounds, swarm_size=96, iters=180,
262                  inertia_w=0.72, c1=1.49, c2=1.49, seed=2025, init_hint: Optional[np.ndarray] = None):
263         self.f_eval = f_eval
264         self.bounds = bounds
265         self.swarm_size = swarm_size
266         self.iters = iters
267         self.w = inertia_w
268         self.c1 = c1

```



```

269     self.c2 = c2
270     self.rng = np.random.default_rng(seed)
271     self.init_hint = init_hint
272
273     def _init_swarm(self):
274         D = len(self.bounds)
275         X = np.zeros((self.swarm_size, D), dtype=float)
276         V = np.zeros((self.swarm_size, D), dtype=float)
277         for j, (lo, hi) in enumerate(self.bounds):
278             X[:, j] = self.rng.uniform(lo, hi, size=self.swarm_size)
279             span = hi - lo
280             V[:, j] = self.rng.uniform(-0.1 * span, 0.1 * span, size=self.swarm_size)
281         if self.init_hint is not None and len(self.init_hint) == D:
282             X[0] = self.init_hint.copy()
283             V[0] = 0.0
284         pbest_X = X.copy()
285         pbest_val = np.full(self.swarm_size, np.inf, dtype=float)
286         gbest_x = None
287         gbest_val = np.inf
288         return X, V, pbest_X, pbest_val, gbest_x, gbest_val
289
290     def _clip_vec(self, x):
291         for j, (lo, hi) in enumerate(self.bounds):
292             if j == 0: # theta 允许环绕
293                 x[j] = x[j] % (2.0 * math.pi)
294             else:
295                 x[j] = clip(x[j], lo, hi)
296         return x
297
298     def optimize(self):
299         X, V, pbest_X, pbest_val, gbest_x, gbest_val = self._init_swarm()
300         gbest_info = None
301         for i in range(self.swarm_size):
302             xi = self._clip_vec(X[i].copy())
303             score_i, _info = self.f_eval(xi)
304             loss_i = -score_i
305             pbest_X[i] = xi
306             pbest_val[i] = loss_i
307             if loss_i < gbest_val:
308                 gbest_val = loss_i
309                 gbest_x = xi.copy()
310                 gbest_info = _info
311         for _iter in range(self.iters):
312             w = self.w
313             for i in range(self.swarm_size):
314                 r1 = np.random.random(len(self.bounds))
315                 r2 = np.random.random(len(self.bounds))

```

```

316         V[i] = w * V[i] + self.c1 * r1 * (pbest_X[i] - X[i]) + self.c2 * r2 * (gbest_x - X[i])
317         X[i] = self._clip_vec(X[i] + V[i])
318         score_i, info_i = self.f_eval(X[i])
319         loss_i = -score_i
320         if loss_i < pbest_val[i]:
321             pbest_val[i] = loss_i
322             pbest_X[i] = X[i].copy()
323         if loss_i < gbest_val:
324             gbest_val = loss_i
325             gbest_x = X[i].copy()
326             gbest_info = info_i
327         best_cover = -gbest_val
328         return gbest_x, best_cover, gbest_info
329
330 def solve_q3(strategy: str = "L0",
331             dt_L0: float = 0.01,
332             N_ANG: int = 48, N_Z: int = 9, INCLUDE_SIDE: bool = True, dt_L1: float = 0.02,
333             swarm_size: int = 96, iters: int = 180, seed: int = 2025,
334             stage2_swarm: int = 64, stage2_iters: int = 90,
335             init_hint: Optional[Tuple[float, float, float, float, float, float, float, float]] = None
336             ) -> Dict[str, Any]:
337
338     bounds = [
339         (0.0, 2.0 * math.pi),
340         (70.0, 140.0),
341         (0.0, 60.0), (0.2, 12.0),
342         (0.0, 60.0), (0.2, 12.0),
343         (0.0, 60.0), (0.2, 12.0),
344     ]
345
346     PTS = None
347     if strategy in ("L1", "two_stage"):
348         PTS = build_cylinder_samples(N_ang=N_ANG, N_z=N_Z, include_side=INCLUDE_SIDE)
349
350     def f_eval_L0(vec):
351         th, v, t1, ta1, t2, ta2, t3, ta3 = vec
352         cover, intervals, info = eval_cover_time_multi_L0(th, v, [t1, t2, t3], [ta1, ta2, ta3], dt=dt_L0)
353         return cover, {"intervals": intervals, "info": info, "theta": th, "v": v}
354
355     def f_eval_L1(vec):
356         th, v, t1, ta1, t2, ta2, t3, ta3 = vec
357         cover, intervals, info = eval_cover_time_multi_L1(th, v, [t1, t2, t3], [ta1, ta2, ta3], PTS=PTS, dt=dt_L1)
358         return cover, {"intervals": intervals, "info": info, "theta": th, "v": v}
359
360     init_vec = None
361     if init_hint is not None:
362         init_vec = np.array([

```

```

363         float(init_hint[0]) % (2.0 * math.pi),
364         clip(float(init_hint[1]), bounds[1][0], bounds[1][1]),
365         clip(float(init_hint[2]), bounds[2][0], bounds[2][1]), clip(float(init_hint[3]), bounds[3][0], bounds[3][1]),
366         clip(float(init_hint[4]), bounds[4][0], bounds[4][1]), clip(float(init_hint[5]), bounds[5][0], bounds[5][1]),
367         clip(float(init_hint[6]), bounds[6][0], bounds[6][1]), clip(float(init_hint[7]), bounds[7][0], bounds[7][1]),
368     ], dtype=float)
369
370 if strategy == "L0":
371     pso = PSO(f_eval=f_eval_L0, bounds=bounds, swarm_size=swarm_size,
372
373     iters=iters, seed=seed, init_hint=init_vec)
374     best_x, best_cover, extra = pso.optimize()
375     eval_used = "L0"
376 elif strategy == "L1":
377     pso = PSO(f_eval=f_eval_L1, bounds=bounds, swarm_size=swarm_size, iters=iters, seed=seed,
378     init_hint=init_vec)
379     best_x, best_cover, extra = pso.optimize()
380     eval_used = "L1"
381 elif strategy == "two_stage":
382     pso1 = PSO(f_eval=f_eval_L0, bounds=bounds, swarm_size=swarm_size, iters=iters, seed=seed,
383     init_hint=init_vec)
384     x1, cover1, extra1 = pso1.optimize()
385     pso2 = PSO(f_eval=f_eval_L1, bounds=bounds, swarm_size=stage2_swarm, iters=stage2_iters,
386     seed=seed+1, init_hint=x1)
387     best_x, best_cover, extra = pso2.optimize()
388     eval_used = "two_stage"
389 else:
390     raise ValueError("strategy 必须为 'L0'、'L1' 或 'two_stage'")
391
392 th = float(extra["theta"]); v = float(extra["v"])
393 info = extra["info"]
394 intervals = extra["intervals"]
395 pairs = info["pairs"]
396 bursts = info["bursts"]
397 out = {
398     "strategy": eval_used,
399     "theta_deg": float(math.degrees(th)),
400     "v_u_mps": float(v),
401     "drops_s": [float(round(p[0], 3)) for p in pairs],
402     "taus_s": [float(round(p[1], 3)) for p in pairs],
403     "bursts": to_pyfloat(bursts),
404     "cover_total_s": float(best_cover),
405     "cover_intervals_s": format_intervals(intervals, 3),
406     "config": {
407         "dt_L0": float(dt_L0), "dt_L1": float(dt_L1),
408         "N_ANG": int(N_ANG), "N_Z": int(N_Z), "INCLUDE_SIDE": bool(INCLUDE_SIDE),
409         "swarm_size": int(swarm_size), "iters": int(iters),

```

```

410         "stage2_swarm": int(stage2_swarm), "stage2_iters": int(stage2_iters)
411     }
412 }
413 return out
414
415
416 def pretty_print(tag: str, ans: Dict[str, Any]):
417     print(f"[Q3 | {tag}] 最优: ")
418     print(f"  strategy: {ans['strategy']}")
419     print(f"  theta_deg: {round(ans['theta_deg'], 3)}")
420     print(f"  v_u_mps: {round(ans['v_u_mps'], 3)}")
421     print(f"  drops_s: {ans['drops_s']}")
422     print(f"  taus_s: {ans['taus_s']}")
423     print(f"  cover_total_s: {round(ans['cover_total_s'], 3)}")
424     print(f"  cover_intervals_s: {ans['cover_intervals_s']}")
425     if ans.get("bursts"):
426         print(f"  bursts: {ans['bursts'][:3]}")
427
428 def randomized_check(n_samples: int = 100,
429                     eval_strategy: str = "L1",
430                     around_solution: Optional[Dict[str, Any]] = None,
431                     seed: int = 2025,
432                     N_ANG: int = 48, N_Z: int = 9, INCLUDE_SIDE: bool = True,
433                     dt_eval: float = 0.02):
434
435     rng = np.random.default_rng(seed)
436     PTS = build_cylinder_samples(N_ang=N_ANG, N_z=N_Z, include_side=INCLUDE_SIDE)
437     if eval_strategy == "L1" else None
438
439     def sample_global():
440         theta = rng.uniform(0, 2*math.pi)
441         v = rng.uniform(70.0, 140.0)
442         t1 = rng.uniform(0.0, 0.3)
443         t2 = t1 + rng.uniform(1.0, 1.5)
444         t3 = rng.uniform(47.0, 53.0)
445         tau1 = rng.uniform(0.2, 0.8)
446         tau2 = rng.uniform(0.2, 0.8)
447         tau3 = rng.uniform(0.8, 3.0)
448         return theta, v, [t1, t2, t3], [tau1, tau2, tau3]
449
450     def sample_local(base):
451         theta = (math.radians(base["theta_deg"]) + rng.normal(0, math.radians(4.0))) % (2*math.pi)
452         v = clip(base["v_u_mps"] + rng.normal(0, 8.0), 70.0, 140.0)
453         t1, t2, t3 = [clip(x + rng.normal(0, 1.0), 0, 60) for x in base["drops_s"]]
454         tau1, tau2, tau3 = [clip(x + rng.normal(0, 0.6), 0.2, 12.0) for x in base["taus_s"]]
455         return theta, v, [t1, t2, t3], [tau1, tau2, tau3]
456

```

```

457 def eval_once(theta, v, drops, taus):
458     if eval_strategy == "L1":
459         cover, _, _ = eval_cover_time_multi_L1(theta, v, drops, taus, PTS=PTS, dt=dt_eval)
460     else:
461         cover, _, _ = eval_cover_time_multi_L0(theta, v, drops, taus, dt=dt_eval)
462     return float(cover), theta, v, list(map(float, drops)), list(map(float, taus))
463
464 mode = "local" if around_solution is not None else "global"
465 covers = []
466 records = []
467
468 for _ in range(n_samples):
469     if mode == "local":
470         theta, v, drops, taus = sample_local(around_solution)
471     else:
472         theta, v, drops, taus = sample_global()
473     c, th, vv, ds, ts = eval_once(theta, v, drops, taus)
474     covers.append(c)
475     records.append((c, th, vv, ds, ts))
476
477 covers = np.array(covers, dtype=float)
478 order = np.argsort(-covers)
479 print(f"\n[check] {mode} 随机 {n_samples} 组 ({eval_strategy} 评估) ")
480 print(f" mean_cover_s: {covers.mean():.3f}")
481 print(f" std_cover_s: {covers.std():.3f}")
482 print(f" max_cover_s: {covers.max():.3f}")
483 print(f" min_cover_s: {covers.min():.3f}")
484 print(" top5:")
485 topk = min(5, n_samples)
486 for i in range(topk):
487     c, th, vv, ds, ts = records[order[i]]
488     print(f"    #{i+1}: cover={c:.3f}s, theta={math.degrees(th):.2f}deg, v={vv:.1f}, "
489           f"pairs=[({ds[0]:.3f},{ts[0]:.3f}), ({ds[1]:.3f},{ts[1]:.3f}), ({ds[2]:.3f},{ts[2]:.3f})]")
490
491
492 def _single_mask_L0(theta: float, v: float, t_drop: float, tau: float, tgrid: np.ndarray) -> np.ndarray:
493     t_burst = t_drop + tau
494     s_burst = burst_point(U0, theta, v, t_drop, tau)
495     if t_burst >= T_HIT or s_burst[2] <= 0.0:
496         return np.zeros_like(tgrid, dtype=bool)
497     mask = np.zeros_like(tgrid, dtype=bool)
498     t_start, t_end = t_burst, min(t_burst + T_EFFECT, T_HIT)
499     if t_end <= t_start:
500         return mask
501     idx = np.where((tgrid >= t_start) & (tgrid <= t_end))[0]
502     for k in idx:
503         t = float(tgrid[k])

```

```

504         if covered_L0_at_time(M0, P_TARGET, s_burst, t_burst, t):
505             mask[k] = True
506     return mask
507
508 def _single_mask_L1(theta: float, v: float, t_drop: float, tau: float, tgrid: np.ndarray,
509 PTS: np.ndarray) -> np.ndarray:
510     t_burst = t_drop + tau
511     s_burst = burst_point(U0, theta, v, t_drop, tau)
512     if t_burst >= T_HIT or s_burst[2] <= 0.0:
513         return np.zeros_like(tgrid, dtype=bool)
514     mask = np.zeros_like(tgrid, dtype=bool)
515     t_start, t_end = t_burst, min(t_burst + T_EFFECT, T_HIT)
516     if t_end <= t_start:
517         return mask
518     idx = np.where((tgrid >= t_start) & (tgrid <= t_end))[0]
519     for k in idx:
520         t = float(tgrid[k])
521         if covered_L1_at_time_vectorized(M0, s_burst, t_burst, t, PTS):
522             mask[k] = True
523     return mask
524
525 def print_q3_report_table(ans: Dict[str, Any]):
526     theta_deg = float(ans["theta_deg"])
527     v = float(ans["v_u_mps"])
528     drops = [float(x) for x in ans["drops_s"]]
529     taus = [float(x) for x in ans["taus_s"]]
530
531     mode_L1 = (ans["strategy"] != "L0")
532     cfg = ans.get("config", {})
533     dt = float(cfg["dt_L1"]) if mode_L1 else cfg["dt_L0"]
534     tgrid = np.arange(0.0, T_HIT + 1e-12, dt)
535     PTS = None
536     if mode_L1:
537         PTS = build_cylinder_samples(N_ang=int(cfg.get("N_ANG", 48)),
538                                     N_z=int(cfg.get("N_Z", 9)),
539                                     include_side=bool(cfg.get("INCLUDE_SIDE", True)))
540
541     print(" 无人机运动方向\t 无人机运动速度 (m/s)\t 烟幕干扰弹编号\t"
542           " 烟幕干扰弹投放点的 x 坐标 (m)\t 烟幕干扰弹投放点的 y 坐标 (m)\t 烟幕干扰弹投放点的 z 坐标 (m)\t"
543           " 烟幕干扰弹起爆点的 x 坐标 (m)\t 烟幕干扰弹起爆点的 y 坐标 (m)\t 烟幕干扰弹起爆点的 z 坐标 (m)\t"
544           " 有效干扰时长 (s)")
545
546     th = math.radians(theta_deg)
547     for i, (td, ta) in enumerate(zip(drops, taus), 1):
548         r_drop = uav_pos(U0, th, v, td)
549         s_burst = burst_point(U0, th, v, td, ta)
550         if mode_L1:

```

```

551     mask = _single_mask_L1(th, v, td, ta, tgrid, PTS)
552 else:
553     mask = _single_mask_L0(th, v, td, ta, tgrid)
554     eff_time = float(mask.sum() * dt)
555     print(f"theta_deg:.3f\t{v:.3f}\t{i}\t"
556           f"r_drop[0]:.3f\t{r_drop[1]:.3f}\t{r_drop[2]:.3f}\t"
557           f"s_burst[0]:.3f\t{s_burst[1]:.3f}\t{s_burst[2]:.3f}\t"
558           f"eff_time:.3f")
559
560
561 if __name__ == "__main__":
562     ans_L0 = solve_q3(strategy="L0", dt_L0=0.01, swarm_size=96, iters=180)
563     pretty_print("L0", ans_L0)
564     print("\n[Q3 | L0] 明细表: ")
565     print_q3_report_table(ans_L0)
566
567     hint = (math.radians(190.0), 120.0, 2.0, 3.5, 4.0, 3.2, 7.0, 3.0)
568     ans_L1 = solve_q3(strategy="L1", N_ANG=48, N_Z=9, INCLUDE_SIDE=True, dt_L1=0.02,
569                       swarm_size=80, iters=140, init_hint=None)
570     print("[Q3 | L1] 最优: ");
571     [print(f" {k}: {v}") for k, v in ans_L1.items()]
572
573     ans_2s = solve_q3(strategy="two_stage",
574                       dt_L0=0.01, swarm_size=96, iters=180,
575                       N_ANG=48, N_Z=9, INCLUDE_SIDE=True, dt_L1=0.02,
576                       stage2_swarm=64, stage2_iters=80)
577     pretty_print("two_stage", ans_2s)
578     print("\n[Q3 | two_stage] 明细表: ")
579     print_q3_report_table(ans_2s)
580
581     randomized_check(n_samples=100, eval_strategy="L1", around_solution=None, seed=2025)
582     randomized_check(n_samples=100, eval_strategy="L1", around_solution=ans_2s, seed=2026)
583
584

```

2.4 问题四

```

1
2 import math
3 import random
4 import numpy as np
5 from typing import List, Dict, Any, Optional, Tuple
6 from itertools import permutations
7
8 g = 9.81
9 VM = 300.0

```

```

10 V_SINK = 3.0
11 R_SMOKE = 10.0
12 T_EFFECT = 20.0
13
14
15 R_TAR, H_TAR = 7.0, 10.0
16 CYL_CENTER = np.array([0.0, 200.0, 0.0], dtype=float)
17 P_TARGET = np.array([0.0, 200.0, 5.0], dtype=float)
18 M0 = np.array([20000.0, 0.0, 2000.0], dtype=float)
19
20 U0_FY1 = np.array([17800.0, 0.0, 1800.0], dtype=float)
21 U0_FY2 = np.array([12000.0, 1400.0, 1400.0], dtype=float)
22 U0_FY3 = np.array([ 6000.0, -3000.0, 700.0], dtype=float)
23 UAVS = [
24     {"name": "FY1", "U0": U0_FY1},
25     {"name": "FY2", "U0": U0_FY2},
26     {"name": "FY3", "U0": U0_FY3},
27 ]
28
29 def missile_hit_time(m0: np.ndarray) -> float:
30     return float(np.linalg.norm(m0) / VM)
31
32 T_HIT = missile_hit_time(M0)
33
34 def deg360(theta_rad: float) -> float:
35     d = math.degrees(theta_rad)
36     d = (d % 360.0 + 360.0) % 360.0
37     return 0.0 if abs(d-360.0) < 1e-9 else d
38
39
40 def unit(v: np.ndarray) -> np.ndarray:
41     n = float(np.linalg.norm(v))
42     return v / n if n > 0 else v
43
44 def missile_pos(m0: np.ndarray, t: float) -> np.ndarray:
45     d = unit(-m0)
46     return m0 + VM * d * t
47
48 def uav_pos(u0: np.ndarray, theta: float, v_u: float, t: float) -> np.ndarray:
49     hx, hy = math.cos(theta), math.sin(theta)
50     return np.array([u0[0] + v_u * hx * t, u0[1] + v_u * hy * t, u0[2]], dtype=float)
51
52 def burst_point(u0: np.ndarray, theta: float, v_u: float, t_drop: float, tau: float) -> np.ndarray:
53     hx, hy = math.cos(theta), math.sin(theta)
54     r_drop = uav_pos(u0, theta, v_u, t_drop)
55     horiz = np.array([v_u * hx * tau, v_u * hy * tau, 0.0], dtype=float)
56     vert = np.array([0.0, 0.0, -0.5 * g * tau * tau], dtype=float)

```



```

57     return r_drop + horiz + vert
58
59 def smoke_center_after_burst(s_burst: np.ndarray, t: float, t_burst: float) -> np.ndarray:
60     dz = -V_SINK * max(0.0, t - t_burst)
61     return s_burst + np.array([0.0, 0.0, dz], dtype=float)
62
63 def point_to_segment_dist(P: np.ndarray, Q: np.ndarray, X: np.ndarray) -> float:
64     v = Q - P
65     vv = float(np.dot(v, v))
66     if vv == 0.0:
67         return float(np.linalg.norm(X - P))
68     a = float(np.dot(X - P, v) / vv)
69     a = 0.0 if a < 0.0 else (1.0 if a > 1.0 else a)
70     Y = P + a * v
71     return float(np.linalg.norm(X - Y))
72
73 def covered_L0_at_time(m0, p_target, s_burst, t_burst, t) -> bool:
74     m_t = missile_pos(m0, t)
75     s_t = smoke_center_after_burst(s_burst, t, t_burst)
76     return (point_to_segment_dist(p_target, m_t, s_t) <= R_SMOKE)
77
78 def clip(x, lo, hi):
79     return lo if x < lo else (hi if x > hi else x)
80
81
82 def cyl_points_top_bottom(N_ang: int = 48) -> np.ndarray:
83     cx, cy, cz = CYL_CENTER
84     pts = []
85     for z in (cz, cz + H_TAR):
86         for k in range(N_ang):
87             ang = 2.0 * math.pi * k / N_ang
88             pts.append((cx + R_TAR * math.cos(ang), cy + R_TAR * math.sin(ang), z))
89     return np.array(pts, dtype=float)
90
91 def cyl_points_side(N_ang: int = 48, N_Z: int = 9) -> np.ndarray:
92     cx, cy, cz = CYL_CENTER
93     zs = np.linspace(cz, cz + H_TAR, N_Z)
94     pts = []
95     for z in zs:
96         for k in range(N_ang):
97             ang = 2.0 * math.pi * k / N_ang
98             pts.append((cx + R_TAR * math.cos(ang), cy + R_TAR * math.sin(ang), z))
99     return np.array(pts, dtype=float)
100
101 def build_cylinder_samples(N_ang=48, N_z=9, include_side=True) -> np.ndarray:
102     pts = [cyl_points_top_bottom(N_ang)]
103     if include_side:

```

```

104     pts.append(cyl_points_side(N_ang, N_z))
105     return np.concatenate(pts, axis=0)
106
107 def covered_L1_at_time_vectorized(m0, s_burst, t_burst, t, PTS) -> bool:
108     m_t = missile_pos(m0, t)
109     s_t = smoke_center_after_burst(s_burst, t, t_burst)
110     v = m_t - PTS
111     w = s_t - PTS
112     vv = np.sum(v * v, axis=1)
113     alpha = np.divide(np.sum(w * v, axis=1), vv, out=np.zeros_like(vv), where=vv > 0.0)
114     alpha = np.clip(alpha, 0.0, 1.0)
115     Y = PTS + alpha[:, None] * v
116     dist = np.linalg.norm(s_t - Y, axis=1)
117     return bool(np.any(dist <= R_SMOKE))
118
119 def _time_grid(dt: float = 0.02) -> np.ndarray:
120     return np.arange(0.0, T_HIT + 1e-12, dt)
121
122 def _candidate_from_anchor(u0: np.ndarray, frac: float, alpha: float, tau_mult: float, clamp_eps:
123 float = 0.10):
124     t_b = clip(frac * min(60.0, T_HIT - 2.0), 0.5, 59.5)
125     m_tb = missile_pos(M0, t_b)
126     Y = P_TARGET + alpha * (m_tb - P_TARGET)
127
128     dx, dy = Y[0] - u0[0], Y[1] - u0[1]
129     theta = math.atan2(dy, dx)
130     D_xy = math.hypot(dx, dy)
131     v = clip(D_xy / t_b, 70.0, 140.0)
132
133     u0z, Yz = float(u0[2]), float(Y[2])
134     if u0z > Yz:
135         tau_base = math.sqrt(max(0.0, 2.0 * (u0z - Yz)) / g)
136     else:
137         tau_base = 0.2
138
139     tau_max_by_t = max(0.2, t_b - clamp_eps)
140     tau = clip(tau_base * tau_mult, 0.2, min(12.0, tau_max_by_t))
141     t_drop = t_b - tau
142     return theta % (2.0 * math.pi), v, t_drop, tau
143
144 def _candidate_mask_L0(uav_idx: int, theta: float, v: float, t_drop: float, tau: float,
145 tgrid: np.ndarray) -> np.ndarray:
146     u0 = UAVS[uav_idx]["U0"]
147     t_burst = t_drop + tau
148     if t_burst >= T_HIT:
149         return np.zeros_like(tgrid, dtype=bool)
150     s_burst = burst_point(u0, theta, v, t_drop, tau)

```

```

151     if s_burst[2] <= 0.0:
152         return np.zeros_like(tgrid, dtype=bool)
153
154     mask = np.zeros_like(tgrid, dtype=bool)
155     t_start, t_end = t_burst, min(t_burst + T_EFFECT, T_HIT)
156     if t_end <= t_start:
157         return mask
158
159     idx = np.where((tgrid >= t_start) & (tgrid <= t_end))[0]
160     for k in idx:
161         t = float(tgrid[k])
162         if covered_L0_at_time(M0, P_TARGET, s_burst, t_burst, t):
163             mask[k] = True
164     return mask
165
166 def _candidate_mask_L1(uav_idx: int, theta: float, v: float, t_drop: float, tau: float,
167                       tgrid: np.ndarray, PTS: np.ndarray) -> np.ndarray:
168     u0 = UAVS[uav_idx]["U0"]
169     t_burst = t_drop + tau
170     if t_burst >= T_HIT:
171         return np.zeros_like(tgrid, dtype=bool)
172     s_burst = burst_point(u0, theta, v, t_drop, tau)
173     if s_burst[2] <= 0.0:
174         return np.zeros_like(tgrid, dtype=bool)
175
176     mask = np.zeros_like(tgrid, dtype=bool)
177     t_start, t_end = t_burst, min(t_burst + T_EFFECT, T_HIT)
178     if t_end <= t_start:
179         return mask
180
181     idx = np.where((tgrid >= t_start) & (tgrid <= t_end))[0]
182     for k in idx:
183         t = float(tgrid[k])
184         if covered_L1_at_time_vectorized(M0, s_burst, t_burst, t, PTS):
185             mask[k] = True
186     return mask
187
188 def build_candidates_q4(
189     fracs = (0.12, 0.25, 0.40, 0.55, 0.70, 0.85, 0.92, 0.96),
190     alphas = (0.60, 0.70, 0.80, 0.88, 0.92, 0.96, 0.985),
191     taus = (0.55, 0.70, 0.85, 1.00, 1.15, 1.30, 1.50),
192     per_uav_keep: int = 28,
193     dt_mask: float = 0.015,
194     mask_mode: str = 'L0',
195     PTS: Optional[np.ndarray] = None
196 ) -> Tuple[List[Dict[str, Any]], np.ndarray]:
197

```

```

198 tgrid = _time_grid(dt_mask)
199 all_cands = []
200 for uav_idx in range(3):
201     local = []
202     u0 = UAVS[uav_idx]["U0"]
203     for f in fracs:
204         for a in alphas:
205             for tau_mult in taus:
206                 th, v, td, ta = _candidate_from_anchor(u0, f, a, tau_mult)
207                 if mask_mode.upper() == 'L1':
208                     if PTS is None:
209                         raise ValueError("L1 评估需要提供采样点 PTS")
210                     mask = _candidate_mask_L1(uav_idx, th, v, td, ta, tgrid, PTS)
211                 else:
212                     mask = _candidate_mask_L0(uav_idx, th, v, td, ta, tgrid)
213                 score = float(mask.sum() * dt_mask)
214                 if score > 0.0:
215                     local.append({
216                         "uav": uav_idx,
217                         "theta": th, "v": v, "t_drop": td, "tau": ta,
218                         "mask": mask, "score": score
219                     })
220                 local.sort(key=lambda x: x["score"], reverse=True)
221                 all_cands += local[:per_uav_keep]
222 return all_cands, tgrid
223
224 def remask_candidates(
225     candidates: List[Dict[str, Any]],
226     tgrid: np.ndarray,
227     mask_mode: str,
228     dt_mask: float,
229     PTS: Optional[np.ndarray] = None
230 ) -> List[Dict[str, Any]]:
231     out = []
232     for c in candidates:
233         u = c["uav"]; th, v, td, ta = c["theta"], c["v"], c["t_drop"], c["tau"]
234         if mask_mode.upper() == 'L1':
235             if PTS is None:
236                 raise ValueError("L1 评估需要提供采样点 PTS")
237             mask = _candidate_mask_L1(u, th, v, td, ta, tgrid, PTS)
238         else:
239             mask = _candidate_mask_L0(u, th, v, td, ta, tgrid)
240         sc = float(mask.sum() * dt_mask)
241         if sc <= 0.0:
242             continue
243         c2 = dict(c)
244         c2["mask"] = mask

```

```

245     c2["score"] = sc
246     out.append(c2)
247     return out
248
249 def _largest_gap(mask: np.ndarray, tgrid: np.ndarray) -> Tuple[float, float]:
250     if len(tgrid) < 2:
251         return 0.0, 0.0
252     gaps = []
253     in_gap = False; a = None
254     for k in range(len(mask)):
255         if (not mask[k]) and (not in_gap):
256             in_gap = True; a = float(tgrid[k])
257         if in_gap and (k == len(mask)-1 or mask[k+1]):
258             b = float(tgrid[k])
259             gaps.append((a, b))
260             in_gap = False
261     if not gaps:
262         return float(tgrid[len(tgrid)//2]), 0.0
263     a, b = max(gaps, key=lambda it: it[1]-it[0])
264     return 0.5*(a+b), (b-a)
265
266 def _synthesize_and_polish_for_gap(uav_idx: int, gap_center_t: float,
267                                   tgrid: np.ndarray,
268                                   mask_mode: str = 'L0',
269                                   PTS: Optional[np.ndarray] = None) -> Optional[Dict[str, Any]]:
270     u0 = UAVS[uav_idx]["U0"]
271     t_b = clip(gap_center_t, 0.6, min(59.5, T_HIT-0.5))
272     frac = t_b / min(60.0, T_HIT-2.0)
273     alphas_try = (0.92, 0.96, 0.985)
274     tau_mult_try = (0.70, 0.85, 1.00, 1.15)
275     dt = float(tgrid[1]-tgrid[0])
276
277     def mk(theta, v, td, ta):
278         if mask_mode.upper() == 'L1':
279             if PTS is None:
280                 return None
281             mask = _candidate_mask_L1(uav_idx, theta, v, td, ta, tgrid, PTS)
282         else:
283             mask = _candidate_mask_L0(uav_idx, theta, v, td, ta, tgrid)
284         return {"uav": uav_idx, "theta": theta, "v": v, "t_drop": td, "tau": ta,
285               "mask": mask, "score": float(mask.sum()*dt)}
286
287     best = None
288     for a in alphas_try:
289         th, v, td, ta = _candidate_from_anchor(u0, frac, a, 1.0)
290         neigh_yaw = [th + math.radians(d) for d in (-8,-4,0,4,8)]
291         neigh_v = [clip(v + dv, 70.0, 140.0) for dv in (-12,-6,0,6,12)]

```

```

292     neigh_td = [clip(td + d, 0.0, 60.0) for d in (-1.5,-1.0,-0.5,0.0,0.5,1.0,1.5)]
293     neigh_tau = [clip(ta*m, 0.2, min(12.0, t_b-0.1)) for m in tau_mult_try]
294     for th1 in neigh_yaw:
295         for v1 in neigh_v:
296             for td1 in neigh_td:
297                 for ta1 in neigh_tau:
298                     c1 = mk(th1, v1, td1, ta1)
299                     if c1 is None:
300                         continue
301                     if best is None or c1["score"] > best["score"]:
302                         best = c1
303     return best
304
305 def select_exact_one_per_uav(
306     candidates: List[Dict[str, Any]],
307     tgrid: np.ndarray,
308     mask_mode: str = 'L0',
309     PTS: Optional[np.ndarray] = None
310 ) -> Tuple[List[Dict[str, Any]], float, np.ndarray]:
311     groups = {0: [], 1: [], 2: []}
312     for c in candidates:
313         groups[c["uav"]].append(c)
314     dt = float(tgrid[1] - tgrid[0])
315
316     best_score, best_sol, best_mask = -1.0, None, None
317     for order in permutations([0, 1, 2]):
318         union_mask = np.zeros_like(tgrid, dtype=bool)
319         picked = []
320         feasible = True
321         for u in order:
322             best_c, best_gain = None, -1.0
323             for c in groups[u]:
324                 new_cover = np.logical_or(union_mask, c["mask"])
325                 gain = float((new_cover.sum() - union_mask.sum()) * dt)
326                 if gain > best_gain:
327                     best_gain, best_c = gain, c
328             if best_c is None or best_gain <= 0.0:
329                 center_t, _ = _largest_gap(union_mask, tgrid)
330                 synth = _synthesize_and_polish_for_gap(u, center_t, tgrid, mask_mode=mask_mode,
331                 PTS=PTS)
332                 if synth is not None:
333                     new_cover = np.logical_or(union_mask, synth["mask"])
334                     gain = float((new_cover.sum() - union_mask.sum()) * dt)
335                     if gain > best_gain:
336                         best_gain, best_c = gain, synth
337             if best_c is None:
338                 feasible = False

```

```

339         break
340     picked.append(best_c)
341     union_mask = np.logical_or(union_mask, best_c["mask"])
342     if feasible:
343         total = float(union_mask.sum() * dt)
344         if total > best_score:
345             best_score, best_sol, best_mask = total, picked, union_mask
346
347     if best_sol is None:
348         return [], 0.0, np.zeros_like(tgrid, dtype=bool)
349     return best_sol, best_score, best_mask
350
351 def greedy_partition_matroid_max_coverage(
352     candidates: List[Dict[str, Any]],
353     tgrid: np.ndarray,
354     per_uav_limit: int = 1,
355     K_total: int = 3
356 ) -> Tuple[List[Dict[str, Any]], float, np.ndarray]:
357     chosen = []
358     union_mask = np.zeros_like(tgrid, dtype=bool)
359     counts = {0: 0, 1: 0, 2: 0}
360     dt = float(tgrid[1] - tgrid[0])
361
362     while len(chosen) < K_total:
363         best_gain, best_c = 0.0, None
364         for c in candidates:
365             u = c["uav"]
366             if counts[u] >= per_uav_limit:
367                 continue
368             new_cover = np.logical_or(union_mask, c["mask"])
369             gain = float((new_cover.sum() - union_mask.sum()) * dt)
370             if gain > best_gain:
371                 best_gain, best_c = gain, c
372         if best_c is None or best_gain <= 0.0:
373             break
374         chosen.append(best_c)
375         union_mask = np.logical_or(union_mask, best_c["mask"])
376         counts[best_c["uav"]] += 1
377         candidates = [x for x in candidates if x is not best_c]
378
379     total = float(union_mask.sum() * dt)
380     return chosen, total, union_mask
381
382 def _mask_to_intervals(mask: np.ndarray, tgrid: np.ndarray) -> List[Tuple[float, float]]:
383     intervals = []
384     if len(mask) == 0:
385         return intervals

```

```

386     in_seg = False
387     a = None
388     for k in range(len(mask)):
389         if mask[k] and not in_seg:
390             in_seg, a = True, float(tgrid[k])
391         if in_seg and (k == len(mask) - 1 or (not mask[k + 1]]):
392             b = float(tgrid[k])
393             intervals.append((round(a, 3), round(b, 3)))
394             in_seg = False
395     return intervals
396
397
398 def solve_q4_graph(
399     strategy: str = 'L0',
400     dt_mask: float = 0.015,
401     fracs=(0.12, 0.25, 0.40, 0.55, 0.70, 0.85, 0.92, 0.96, 0.985, 0.995),
402     alphas=(0.60, 0.70, 0.80, 0.88, 0.92, 0.96, 0.985),
403     taus=(0.55, 0.70, 0.85, 1.00, 1.15, 1.30, 1.50),
404     per_uav_keep=28,
405     N_ANG: int = 48, N_Z: int = 9, INCLUDE_SIDE: bool = True,
406     force_one_per_uav: bool = True,
407     dual_per_uav_keep_L1: Optional[int] = None,
408     debug: bool = False
409 ) -> Dict[str, Any]:
410
411     mode = strategy.upper()
412     if mode not in ('L0', 'L1', 'DUAL'):
413         raise ValueError("strategy 需为 'L0' / 'L1' / 'DUAL'")
414
415     PTS = None
416     if mode in ('L1', 'DUAL'):
417         PTS = build_cylinder_samples(N_ang=N_ANG, N_z=N_Z, include_side=INCLUDE_SIDE)
418
419     if mode in ('L0', 'L1'):
420         cands, tgrid = build_candidates_q4(
421             fracs=fracs, alphas=alphas, taus=taus,
422             per_uav_keep=per_uav_keep, dt_mask=dt_mask,
423             mask_mode=mode, PTS=PTS
424         )
425         if debug:
426             counts = {i: sum(1 for c in cands if c["uav"]==i) for i in range(3)}
427             print(f"[debug] mask_mode={mode}, per-UAV nonzero candidates:", counts)
428
429         if force_one_per_uav:
430             picked, cover_val, union_mask = select_exact_one_per_uav(cands, tgrid, mask_mode=mode,
431                 PTS=PTS)
432         else:

```



```

433     picked, cover_val, union_mask = greedy_partition_matroid_max_coverage(
434         candidates=cands, tgrid=tgrid, per_uav_limit=1, K_total=3
435     )
436
437     ths = [p["theta"] for p in picked]
438     vs = [p["v"] for p in picked]
439     tds = [p["t_drop"] for p in picked]
440     taus_ = [p["tau"] for p in picked]
441     uidx = [p["uav"] for p in picked]
442     unames = [UAVS[u]["name"] for u in uidx]
443
444     bursts = []
445     for p in picked:
446         u0 = UAVS[p["uav"]]["U0"]
447         t_burst = p["t_drop"] + p["tau"]
448         sb = burst_point(u0, p["theta"], p["v"], p["t_drop"], p["tau"])
449         bursts.append({
450             "uav": UAVS[p["uav"]]["name"],
451             "t_drop": float(p["t_drop"]),
452             "tau": float(p["tau"]),
453             "t_burst": float(t_burst),
454             "s_burst": (float(sb[0]), float(sb[1]), float(sb[2]))
455         })
456     bursts = sorted(bursts, key=lambda b: b["t_burst"])
457     intervals = _mask_to_intervals(union_mask, tgrid)
458
459     return {
460         "method": "graph_max_coverage",
461         "strategy": mode,
462         "theta_deg": [round(deg360(x), 3) for x in ths],
463         "v_u_mps": [round(x, 3) for x in vs],
464         "drops_s": [round(x, 3) for x in tds],
465         "taus_s": [round(x, 3) for x in taus_],
466         "uav_names": unames,
467         "uav_indices": uidx,
468         "bursts": bursts,
469         "cover_total_s": cover_val,
470         "cover_intervals_s": intervals,
471         "config": {
472             "dt_mask": dt_mask, "fracs": fracs, "alphas": alphas, "taus": taus,
473             "per_uav_keep": per_uav_keep, "force_one_per_uav": force_one_per_uav,
474             **({"N_ANG": N_ANG, "N_Z": N_Z, "INCLUDE_SIDE": INCLUDE_SIDE}
475                if mode!='L0' else {})
476         }
477     }
478
479     cands_L0, tgrid = build_candidates_q4(

```

```

480     frags=frags, alphas=alphas, taus=taus,
481     per_uav_keep=per_uav_keep, dt_mask=dt_mask,
482     mask_mode='L0', PTS=None
483 )
484 if debug:
485     counts0 = {i: sum(1 for c in cand_L0 if c["uav"]==i) for i in range(3)}
486     print("[debug][DUAL] L0 pool per-UAV:", counts0)
487
488 PTS = build_cylinder_samples(N_ang=N_ANG, N_z=N_Z, include_side=INCLUDE_SIDE)
489 cand_L1 = remask_candidates(cand_L0, tgrid, 'L1', dt_mask, PTS=PTS)
490 if dual_per_uav_keep_L1 is None:
491     dual_per_uav_keep_L1 = per_uav_keep
492 groups = {0: [], 1: [], 2: []}
493 for c in cand_L1: groups[c["uav"]].append(c)
494 cand_L1_trim = []
495 for u in [0,1,2]:
496     groups[u].sort(key=lambda x: x["score"], reverse=True)
497     cand_L1_trim += groups[u][:dual_per_uav_keep_L1]
498 if debug:
499     counts1 = {i: sum(1 for c in cand_L1_trim if c["uav"]==i) for i in range(3)}
500     print("[debug][DUAL] L1 re-score pool per-UAV:", counts1)
501
502 if force_one_per_uav:
503     picked, cover_val, union_mask = select_exact_one_per_uav(cand_L1_trim, tgrid, mask_mode='L1',
504     PTS=PTS)
505 else:
506     picked, cover_val, union_mask = greedy_partition_matroid_max_coverage(
507     candidates=cand_L1_trim, tgrid=tgrid, per_uav_limit=1, K_total=3
508     )
509
510 ths = [p["theta"] for p in picked]
511 vs = [p["v"] for p in picked]
512 tds = [p["t_drop"] for p in picked]
513 taus = [p["tau"] for p in picked]
514 uidx = [p["uav"] for p in picked]
515 unames = [UAVS[u]["name"] for u in uidx]
516
517 bursts = []
518 for p in picked:
519     u0 = UAVS[p["uav"]]["U0"]
520     t_burst = p["t_drop"] + p["tau"]
521     sb = burst_point(u0, p["theta"], p["v"], p["t_drop"], p["tau"])
522     bursts.append({
523         "uav": UAVS[p["uav"]]["name"],
524         "t_drop": float(p["t_drop"]),
525         "tau": float(p["tau"]),
526         "t_burst": float(t_burst),

```

```

527         "s_burst": (float(sb[0]), float(sb[1]), float(sb[2]))
528     })
529     bursts = sorted(bursts, key=lambda b: b["t_burst"])
530     intervals = _mask_to_intervals(union_mask, tgrid)
531
532     return {
533         "method": "graph_max_coverage",
534         "strategy": "DUAL(L0→L1)",
535         "theta_deg": [round(deg360(x), 3) for x in ths],
536         "v_u_mps": [round(x, 3) for x in vs],
537         "drops_s": [round(x, 3) for x in tds],
538         "taus_s": [round(x, 3) for x in taus_],
539         "uav_names": unames,
540         "uav_indices": uidx,
541         "cover_total_s": cover_val,
542         "cover_intervals_s": intervals,
543         "config": {
544             "dt_mask": dt_mask, "fracs": fracs, "alphas": alphas, "taus": taus,
545             "per_uav_keep_L0": per_uav_keep, "per_uav_keep_L1": dual_per_uav_keep_L1,
546             "force_one_per_uav": True,
547             "N_ANG": N_ANG, "N_Z": N_Z, "INCLUDE_SIDE": INCLUDE_SIDE
548         }
549     }
550
551 def _reconstruct_union_from_ans(ans: Dict[str, Any]) -> Tuple[np.ndarray, np.ndarray]:
552     cfg = ans.get("config", {})
553     dt = float(cfg.get("dt_mask", 0.015))
554     tgrid = _time_grid(dt)
555     strategy = str(ans.get("strategy", "L0")).upper()
556     mode = 'L1' if ('L1' in strategy) else 'L0'
557
558     PTS = None
559     if mode == 'L1':
560         PTS = build_cylinder_samples(
561             N_ang=int(cfg.get("N_ANG", 48)),
562             N_z=int(cfg.get("N_Z", 9)),
563             include_side=bool(cfg.get("INCLUDE_SIDE", True))
564         )
565
566     uidx = ans.get("uav_indices", None)
567     if uidx is None:
568         uidx = [0, 1, 2]
569
570     ths_deg = ans["theta_deg"]
571     vs = ans["v_u_mps"]
572     tds = ans["drops_s"]
573     taus_ = ans["taus_s"]

```

```

574
575 if not (len(ths_deg)==len(vs)==len(tds)==len(taus_)==len(uidx)):
576     raise RuntimeError(" 返回数组长度不一致，无法重建 union 掩码")
577
578 union = np.zeros_like(tgrid, dtype=bool)
579 for i in range(len(uidx)):
580     u = int(uidx[i])
581     th = math.radians(float(ths_deg[i]))
582     v = float(vs[i])
583     td = float(tds[i])
584     ta = float(taus_[i])
585
586     if mode == 'L1':
587         mask = _candidate_mask_L1(u, th, v, td, ta, tgrid, PTS)
588     else:
589         mask = _candidate_mask_L0(u, th, v, td, ta, tgrid)
590     union = np.logical_or(union, mask)
591 return union, tgrid
592
593 def _intervals_from_mask(mask: np.ndarray, tgrid: np.ndarray) -> List[Tuple[float, float]]:
594     return _mask_to_intervals(mask, tgrid)
595
596 def validate_q4_solution(ans: Dict[str, Any], verbose: bool = True, tol_s: float = 0.45):
597     union, tgrid = _reconstruct_union_from_ans(ans)
598     dt = float(tgrid[1]-tgrid[0])
599     cover = float(union.sum()*dt)
600     cover_ans = float(ans["cover_total_s"])
601     if verbose:
602         print(f"[check] reconstruct cover={cover:.3f}s vs ans={cover_ans:.3f}s
603             (|Δ|= {abs(cover-cover_ans):.3f}s)")
604     assert abs(cover - cover_ans) <= max(tol_s, 2.5*dt), " 覆盖时长与返回值不一致"
605
606     inter = _intervals_from_mask(union, tgrid)
607     inter_ans = ans["cover_intervals_s"]
608     if verbose:
609         print(f"[check] intervals reconstructed: {inter}")
610         print(f"[check] intervals in ans      : {inter_ans}")
611     assert len(inter) == len(inter_ans), " 区间数量不一致"
612     for (a,b), (A,B) in zip(inter, inter_ans):
613         assert abs(a-A) <= 2.5*dt and abs(b-B) <= 2.5*dt, " 区间端点不一致（离散误差之外）"
614
615
616 def _mode_from_strategy_str(strategy_str: str) -> str:
617     s = (strategy_str or "").upper()
618     return 'L1' if ('L1' in s) else 'L0'
619
620 def summarize_solution_rows(ans: Dict[str, Any]) -> List[Dict[str, Any]]:

```

```

621     cfg = ans.get("config", {})
622     dt = float(cfg.get("dt_mask", 0.02))
623     tgrid = _time_grid(dt)
624     mode = _mode_from_strategy_str(str(ans.get("strategy", "L0")))
625
626     PTS = None
627     if mode == 'L1':
628         N_ANG = int(cfg.get("N_ANG", 48))
629         N_Z = int(cfg.get("N_Z", 9))
630         INCLUDE_SIDE = bool(cfg.get("INCLUDE_SIDE", True))
631         PTS = build_cylinder_samples(N_ang=N_ANG, N_z=N_Z, include_side=INCLUDE_SIDE)
632
633     uidx_list = ans.get("uav_indices", [0,1,2])
634     ths_deg = ans["theta_deg"]
635     vs = ans["v_u_mps"]
636     tds = ans["drops_s"]
637     taus_ = ans["taus_s"]
638     if not (len(uidx_list)==len(ths_deg)==len(vs)==len(tds)==len(taus_)):
639         raise RuntimeError(" 返回数组长度不一致，无法生成明细表。")
640
641     rows = []
642     for i in range(len(uidx_list)):
643         u = int(uidx_list[i])
644         name = UAVS[u]["name"]
645         th_deg = float(ths_deg[i])
646         th = math.radians(th_deg)
647         v = float(vs[i])
648         td = float(tds[i])
649         ta = float(taus_[i])
650
651         r_drop = uav_pos(UAVS[u]["U0"], th, v, td)
652         s_burst = burst_point(UAVS[u]["U0"], th, v, td, ta)
653
654         if mode == 'L1':
655             mask = _candidate_mask_L1(u, th, v, td, ta, tgrid, PTS)
656         else:
657             mask = _candidate_mask_L0(u, th, v, td, ta, tgrid)
658         eff_time = float(mask.sum() * dt)
659
660         rows.append({
661             "uav": name,
662             "theta_deg": th_deg,
663             "v": v,
664             "x_drop": float(r_drop[0]), "y_drop": float(r_drop[1]), "z_drop": float(r_drop[2]),
665             "x_burst": float(s_burst[0]), "y_burst": float(s_burst[1]), "z_burst": float(s_burst[2]),
666             "eff_time_s": eff_time
667         })

```

```

668     return rows
669
670 def print_q4_report_table(ans: Dict[str, Any]):
671     rows = summarize_solution_rows(ans)
672     header = (
673         " 无人机编号\t"
674         " 无人机运动方向\t"
675         " 无人机运动速度 (m/s)\t"
676         " 烟幕干扰弹投放点的 x 坐标 (m)\t"
677         " 烟幕干扰弹投放点的 y 坐标 (m)\t"
678         " 烟幕干扰弹投放点的 z 坐标 (m)\t"
679         " 烟幕干扰弹起爆点的 x 坐标 (m)\t"
680         " 烟幕干扰弹起爆点的 y 坐标 (m)\t"
681         " 烟幕干扰弹起爆点的 z 坐标 (m)\t"
682         " 有效干扰时长 (s)"
683     )
684     print(header)
685     for r in rows:
686         print(
687             f"{r['uav']}\t"
688             f"{r['theta_deg']:.3f}\t"
689             f"{r['v']:.3f}\t"
690             f"{r['x_drop']:.3f}\t"
691             f"{r['y_drop']:.3f}\t"
692             f"{r['z_drop']:.3f}\t"
693             f"{r['x_burst']:.3f}\t"
694             f"{r['y_burst']:.3f}\t"
695             f"{r['z_burst']:.3f}\t"
696             f"{r['eff_time_s']:.3f}"
697         )
698
699
700 def _intervals_to_mask(intervals, tgrid):
701     mask = np.zeros_like(tgrid, dtype=bool)
702     if not intervals:
703         return mask
704     for (a, b) in intervals:
705         idx = np.where((tgrid >= a - 1e-12) & (tgrid <= b + 1e-12))[0]
706         mask[idx] = True
707     return mask
708
709 def _mask_by_mode_for_params(uav_idx, theta_deg, v, t_drop, tau, tgrid, mode, PTS=None):
710     th = math.radians(theta_deg)
711     if mode.upper() == 'L1':
712         if PTS is None:
713             raise ValueError("L1 模式需要 PTS")
714         return _candidate_mask_L1(uav_idx, th, v, t_drop, tau, tgrid, PTS)

```

```

715     else:
716         return _candidate_mask_L0(uav_idx, th, v, t_drop, tau, tgrid)
717
718 def _pick_best_pair_given_fixed_union(
719     groups,
720     tgrid,
721     union_fixed
722 ):
723     dt = float(tgrid[1] - tgrid[0])
724     u_left = sorted(groups.keys())
725     assert len(u_left) == 2, " 应仅剩两架 UAV"
726     U, V = u_left[0], u_left[1]
727     best = None
728     best_union = None
729     best_cover = -1.0
730
731     for cU in groups[U]:
732         for cV in groups[V]:
733             uni = union_fixed
734             uni = np.logical_or(uni, cU["mask"])
735             uni = np.logical_or(uni, cV["mask"])
736             cover = float(uni.sum()) * dt
737             if cover > best_cover:
738                 best_cover = cover
739                 best = (cU, cV)
740                 best_union = uni
741     return best, best_cover, best_union
742
743 def solve_q4_graph_fixed_fy1(
744     strategy: str = 'DUAL',
745     dt_mask: float = 0.02,
746     fracs=(0.12, 0.25, 0.40, 0.55, 0.70, 0.85, 0.92, 0.96, 0.985, 0.995),
747     alphas=(0.60, 0.70, 0.80, 0.88, 0.92, 0.96, 0.985),
748     taus=(0.55, 0.70, 0.85, 1.00, 1.15, 1.30, 1.50),
749     per_uav_keep=28,
750     dual_per_uav_keep_L1: Optional[int] = None,
751     N_ANG: int = 48, N_Z: int = 9, INCLUDE_SIDE: bool = True,
752     debug: bool = False,
753     fy1_theta_deg: Optional[float] = None,
754     fy1_v: Optional[float] = None,
755     fy1_t_drop: Optional[float] = None,
756     fy1_tau: Optional[float] = None,
757     fy1_intervals: Optional[List[Tuple[float, float]]] = None
758 ) -> Dict[str, Any]:
759     mode = strategy.upper()
760     if mode not in ('L0', 'L1', 'DUAL'):
761         raise ValueError("strategy 需为 'L0' / 'L1' / 'DUAL'")

```

```

762
763 tgrid = _time_grid(dt_mask)
764 PTS = None
765 if mode in ('L1','DUAL'):
766     PTS = build_cylinder_samples(N_ang=N_ANG, N_z=N_Z, include_side=INCLUDE_SIDE)
767
768 have_params = (fy1_theta_deg is not None and fy1_v is not None and
769               fy1_t_drop is not None and fy1_tau is not None)
770 union_fixed = np.zeros_like(tgrid, dtype=bool)
771 fy1_row_for_report = None
772
773 if have_params:
774     eval_mode = 'L1' if (mode in ('L1','DUAL')) else 'L0'
775     mask_fy1 = _mask_by_mode_for_params(
776         uav_idx=0, theta_deg=fy1_theta_deg, v=fy1_v,
777         t_drop=fy1_t_drop, tau=fy1_tau,
778         tgrid=tgrid, mode=eval_mode, PTS=PTS
779     )
780     union_fixed = np.logical_or(union_fixed, mask_fy1)
781     r_drop = uav_pos(UAVS[0]["U0"], math.radians(fy1_theta_deg), fy1_v, fy1_t_drop)
782     s_burst = burst_point(UAVS[0]["U0"], math.radians(fy1_theta_deg), fy1_v, fy1_t_drop, fy1_tau)
783     fy1_row_for_report = {
784         "uav": "FY1",
785         "theta_deg": float(fy1_theta_deg),
786         "v": float(fy1_v),
787         "x_drop": float(r_drop[0]), "y_drop": float(r_drop[1]), "z_drop": float(r_drop[2]),
788         "x_burst": float(s_burst[0]), "y_burst": float(s_burst[1]), "z_burst": float(s_burst[2]),
789         "eff_time_s": float(mask_fy1.sum()*dt_mask),
790         "mask": mask_fy1
791     }
792 elif fy1_intervals is not None:
793     mask_fy1 = _intervals_to_mask(fy1_intervals, tgrid)
794     union_fixed = np.logical_or(union_fixed, mask_fy1)
795     fy1_row_for_report = {
796         "uav": "FY1", "theta_deg": float('nan'), "v": float('nan'),
797         "x_drop": float('nan'), "y_drop": float('nan'), "z_drop": float('nan'),
798         "x_burst": float('nan'), "y_burst": float('nan'), "z_burst": float('nan'),
799         "eff_time_s": float(mask_fy1.sum()*dt_mask),
800         "mask": mask_fy1
801     }
802 else:
803     raise ValueError(" 请至少提供 FY1 的参数 (theta/v/t_drop/tau) 或遮蔽区间 intervals")
804
805 if mode in ('L0','L1'):
806     cands, __ = build_candidates_q4(
807         fracs=fracs, alphas=alphas, taus=taus,
808         per_uav_keep=per_uav_keep, dt_mask=dt_mask,

```



```

809     mask_mode=mode, PTS=PTS
810 )
811 if debug:
812     counts_all = {i: sum(1 for cc in cands if cc["uav"]==i) for i in range(3)}
813     counts_23 = {i: sum(1 for cc in cands if cc["uav"]==i) for i in (1,2)}
814     print(f"[debug][fixed FY1] mask_mode={mode}, per-UAV nonzero candidates (all): {counts_all}")
815     print(f"[debug][fixed FY1] using FY2/FY3: {counts_23}")
816 else:
817     cands_L0, _ = build_candidates_q4(
818         fracs=fracs, alphas=alphas, taus=taus,
819         per_uav_keep=per_uav_keep, dt_mask=dt_mask,
820         mask_mode='L0', PTS=None
821     )
822     if debug:
823         counts0 = {i: sum(1 for cc in cands_L0 if cc["uav"]==i) for i in range(3)}
824         print(f"[debug][fixed FY1][DUAL] L0 pool per-UAV:", counts0)
825     cands = remask_candidates(cands_L0, tgrid, 'L1', dt_mask, PTS=PTS)
826     if dual_per_uav_keep_L1 is None:
827         dual_per_uav_keep_L1 = per_uav_keep
828     groups_tmp = {0:[], 1:[], 2:[]}
829     for c in cands: groups_tmp[c["uav"]].append(c)
830     cands_trim = []
831     for u in [0,1,2]:
832         groups_tmp[u].sort(key=lambda x:x["score"], reverse=True)
833         cands_trim += groups_tmp[u][:dual_per_uav_keep_L1]
834     cands = cands_trim
835     if debug:
836         counts1 = {i: sum(1 for cc in cands if cc["uav"]==i) for i in range(3)}
837         print(f"[debug][fixed FY1][DUAL] L1 re-score pool per-UAV:", counts1)
838
839     groups = {1:[], 2:[]}
840     for c in cands:
841         if c["uav"] in (1,2):
842             groups[c["uav"]].append(c)
843
844     if debug:
845         print(f"[debug][fixed FY1] pool sizes (FY2/FY3):", {u:len(groups[u]) for u in (1,2)})
846
847     (c2, c3), cover_total, union_mask = _pick_best_pair_given_fixed_union(groups, tgrid, union_fixed)
848
849     ths = [fy1_row_for_report["theta_deg"], deg360(c2["theta"]), deg360(c3["theta"])]
850     vs = [fy1_row_for_report["v"], c2["v"], c3["v"]]
851     tds = [fy1_t_drop if have_params else float('nan'), c2["t_drop"], c3["t_drop"]]
852     taus_ = [fy1_tau if have_params else float('nan'), c2["tau"], c3["tau"]]
853     uidx = [0, 1, 2]
854     unames = [UAVS[u]["name"] for u in uidx]
855

```

```

856 bursts = []
857 if have_params:
858     sb1 = burst_point(UAVS[0]["U0"],
859
860     math.radians(fy1_row_for_report["theta_deg"]), fy1_row_for_report["v"], fy1_t_drop, fy1_tau)
861     bursts.append({
862         "uav": "FY1",
863         "t_drop": float(fy1_t_drop), "tau": float(fy1_tau),
864         "t_burst": float(fy1_t_drop + fy1_tau),
865         "s_burst": (float(sb1[0]), float(sb1[1]), float(sb1[2]))
866     })
867 for p in (c2, c3):
868     u0 = UAVS[p["uav"]]["U0"]
869     t_burst = p["t_drop"] + p["tau"]
870     sb = burst_point(u0, p["theta"], p["v"], p["t_drop"], p["tau"])
871     bursts.append({
872         "uav": UAVS[p["uav"]]["name"],
873         "t_drop": float(p["t_drop"]),
874         "tau": float(p["tau"]),
875         "t_burst": float(t_burst),
876         "s_burst": (float(sb[0]), float(sb[1]), float(sb[2]))
877     })
878 bursts = sorted(bursts, key=lambda b: b["t_burst"])
879 intervals = _mask_to_intervals(union_mask, tgrid)
880
881 ans = {
882     "method": "graph_max_coverage (FY1 fixed)",
883     "strategy": ( "DUAL(L0→L1)" if mode=="DUAL" else mode ),
884     "theta_deg": [round(x, 3) for x in ths],
885     "v_u_mps": [round(x, 3) for x in vs],
886     "drops_s": [round(x, 3) for x in tds],
887     "taus_s": [round(x, 3) for x in taus_],
888     "uav_names": unames,
889     "uav_indices": uidx,
890     "bursts": bursts,
891     "cover_total_s": float(cover_total),
892     "cover_intervals_s": intervals,
893     "config": {
894         "dt_mask": dt_mask, "fracs": fracs, "alphas": alphas, "taus": taus,
895         "per_uav_keep_base": per_uav_keep,
896         "dual_per_uav_keep_L1": dual_per_uav_keep_L1,
897         "N_ANG": N_ANG, "N_Z": N_Z, "INCLUDE_SIDE": INCLUDE_SIDE
898     }
899 }
900 return ans
901
902 def convergence_test_dt_fixed(fy1_theta_deg, fy1_v, fy1_t_drop, fy1_tau,

```

```

903         strategy='DUAL', dts=(0.04, 0.02, 0.015, 0.01),
904         per_uav_keep=24, dual_per_uav_keep_L1=24,
905         N_ANG=48, N_Z=9, INCLUDE_SIDE=True):
906     rows = []
907     for dt in dts:
908         ans = solve_q4_graph_fixed_fy1(
909             strategy=strategy, dt_mask=dt,
910             per_uav_keep=per_uav_keep, dual_per_uav_keep_L1=dual_per_uav_keep_L1,
911             N_ANG=N_ANG, N_Z=N_Z, INCLUDE_SIDE=INCLUDE_SIDE, debug=False,
912             fy1_theta_deg=fy1_theta_deg, fy1_v=fy1_v, fy1_t_drop=fy1_t_drop, fy1_tau=fy1_tau
913         )
914         rows.append((dt, float(ans["cover_total_s"])))
915     return rows
916
917 def rotation_invariance_test_fixed(fy1_theta_deg, fy1_v, fy1_t_drop, fy1_tau,
918     angles_deg=(0,45,90,135),
919     strategy='DUAL', dt_mask=0.02,
920     per_uav_keep=24, dual_per_uav_keep_L1=24,
921     N_ANG=48, N_Z=9, INCLUDE_SIDE=True):
922     def rotz(vec, ang_rad):
923         c,s = math.cos(ang_rad), math.sin(ang_rad)
924         x,y,z = vec[0], vec[1], vec[2]
925         return np.array([c*x - s*y, s*x + c*y, z], dtype=float)
926
927     M0_bak = M0.copy()
928     U_bak = [u["U0"].copy() for u in UAVS]
929     P_bak = P_TARGET.copy()
930     C_bak = CYL_CENTER.copy()
931
932     covs=[]
933     for ang in angles_deg:
934         rad = math.radians(ang)
935         M0[:] = rotz(M0_bak, rad)
936         for i in range(3):
937             UAVS[i]["U0"][:] = rotz(U_bak[i], rad)
938             P_TARGET[:] = rotz(P_bak, rad)
939             CYL_CENTER[:] = rotz(C_bak, rad)
940             ans = solve_q4_graph_fixed_fy1(
941                 strategy=strategy, dt_mask=dt_mask,
942                 per_uav_keep=per_uav_keep, dual_per_uav_keep_L1=dual_per_uav_keep_L1,
943                 N_ANG=N_ANG, N_Z=N_Z, INCLUDE_SIDE=INCLUDE_SIDE, debug=False,
944                 fy1_theta_deg=(fy1_theta_deg + ang), # 增加旋转角
945                 fy1_v=fy1_v, fy1_t_drop=fy1_t_drop, fy1_tau=fy1_tau
946             )
947             covs.append(ans["cover_total_s"])
948
949     M0[:] = M0_bak

```

```

950     for i in range(3):
951         UAVS[i]["U0"][:] = U_bak[i]
952     P_TARGET[:] = P_bak
953     CYL_CENTER[:] = C_bak
954
955     return list(zip(angles_deg, covs))
956
957 def run_all_q4_checks_fixed(
958     fy1_theta_deg=7.374506365477594,
959     fy1_v=98.85283623880619,
960     fy1_t_drop=0.025903266051691094,
961     fy1_tau=0.8431654639833148,
962     fy1_intervals=[(0.869068730035006, 5.929068730035007)]
963 ):
964
965     print("\n[check] 基本求解 + 一致性 (FY1 固定, L0) ")
966     ans_L0_fix = solve_q4_graph_fixed_fy1(
967         strategy='L0', dt_mask=0.015,
968         per_uav_keep=28, debug=True,
969         fy1_theta_deg=fy1_theta_deg, fy1_v=fy1_v, fy1_t_drop=fy1_t_drop, fy1_tau=fy1_tau,
970         fy1_intervals=fy1_intervals
971     )
972     validate_q4_solution(ans_L0_fix, verbose=True)
973
974     print("\n[check] DUAL 一致性 (FY1 固定) ")
975     ans_DUAL_fix = solve_q4_graph_fixed_fy1(
976         strategy='DUAL', dt_mask=0.02,
977         per_uav_keep=28, dual_per_uav_keep_L1=24, debug=True,
978         fy1_theta_deg=fy1_theta_deg, fy1_v=fy1_v, fy1_t_drop=fy1_t_drop, fy1_tau=fy1_tau,
979         fy1_intervals=fy1_intervals
980     )
981     validate_q4_solution(ans_DUAL_fix, verbose=True)
982
983     print("\n[check] 时间步长收敛 (DUAL, FY1 固定) ")
984     rows = convergence_test_dt_fixed(
985         fy1_theta_deg, fy1_v, fy1_t_drop, fy1_tau,
986         strategy='DUAL', dts=(0.04, 0.02, 0.015, 0.01),
987         per_uav_keep=24, dual_per_uav_keep_L1=24
988     )
989     print(" dt vs cover:", [(round(dt,3), round(cov,3)) for dt,cov in rows])
990
991     print("\n[check] 旋转不变性 (DUAL, FY1 固定) ")
992     rotrows = rotation_invariance_test_fixed(
993         fy1_theta_deg, fy1_v, fy1_t_drop, fy1_tau,
994         angles_deg=(0, 45, 90, 135), strategy='DUAL', dt_mask=0.02,
995         per_uav_keep=24, dual_per_uav_keep_L1=24
996     )

```

```

997 print(" angle vs cover:", [(ang, round(cov,3)) for ang,cov in rotrows])
998
999 return {
1000     "ans_L0_fix": ans_L0_fix,
1001     "ans_DUAL_fix": ans_DUAL_fix,
1002     "convergence_rows": rows,
1003     "rotation_rows": rotrows
1004 }
1005
1006
1007 if __name__ == "__main__":
1008     ans_L0 = solve_q4_graph(
1009         strategy='L0',
1010         dt_mask=0.015,
1011         per_uav_keep=28,
1012         debug=True
1013     )
1014     print("\n[Q4 | Graph-L0] 最优: ")
1015     for k, v in ans_L0.items():
1016         print(" ", k, ":", v)
1017     print("\n[Q4 | Graph-L0] 结果明细表: ")
1018     print_q4_report_table(ans_L0)
1019
1020     ans_L1 = solve_q4_graph(
1021         strategy='L1',
1022         dt_mask=0.02,
1023         N_ANG=48, N_Z=9, INCLUDE_SIDE=True,
1024         per_uav_keep=24,
1025         debug=True
1026     )
1027     print("\n[Q4 | Graph-L1] 最优: ")
1028     for k, v in ans_L1.items():
1029         print(" ", k, ":", v)
1030     print("\n[Q4 | Graph-L1] 结果明细表: ")
1031     print_q4_report_table(ans_L1)
1032
1033     ans_DUAL = solve_q4_graph(
1034         strategy='DUAL',
1035         dt_mask=0.02,
1036         per_uav_keep=28,
1037         dual_per_uav_keep_L1=24,
1038         N_ANG=48, N_Z=9, INCLUDE_SIDE=True,
1039         debug=True
1040     )
1041     print("\n[Q4 | Graph-DUAL] 最优: ")
1042     for k, v in ans_DUAL.items():
1043         print(" ", k, ":", v)

```

```

1044 print("\n[Q4 | Graph-DUAL] 结果明细表: ")
1045 print_q4_report_table(ans_DUAL)
1046
1047 ans_fixed = solve_q4_graph_fixed_fy1(
1048     strategy='DUAL',
1049     dt_mask=0.02,
1050     per_uav_keep=28,
1051     dual_per_uav_keep_L1=24,
1052     N_ANG=48, N_Z=9, INCLUDE_SIDE=True,
1053     debug=True,
1054     fy1_theta_deg=7.374506365477594,
1055     fy1_v=98.85283623880619,
1056     fy1_t_drop=0.025903266051691094,
1057     fy1_tau=0.8431654639833148,
1058     fy1_intervals=[(0.869068730035006, 5.929068730035007)]
1059 )
1060 print("\n[Q4 | Graph-DUAL | FY1 fixed] 最优: ")
1061 for k, v in ans_fixed.items():
1062     print(" ", k, ":", v)
1063 print("\n[Q4 | Graph-DUAL | FY1 fixed] 结果明细表: ")
1064 print_q4_report_table(ans_fixed)
1065
1066
1067 # 一致性校验
1068 print("\n[Q4 | 校验 | FY1 fixed]")
1069 validate_q4_solution(ans_fixed, verbose=True)
1070
1071 __ = run_all_q4_checks_fixed(
1072     fy1_theta_deg=7.374506365477594,
1073     fy1_v=98.85283623880619,
1074     fy1_t_drop=0.025903266051691094,
1075     fy1_tau=0.8431654639833148,
1076     fy1_intervals=[(0.869068730035006, 5.929068730035007)]
1077 )
1078

```

2.5 问题五

```

1
2 import math
3 import copy
4 import statistics as stats
5 import numpy as np
6 from typing import List, Dict, Any, Optional, Tuple
7 rng = np.random.default_rng(2024)
8

```

```

9  g = 9.81
10 VM = 300.0
11 V_SINK = 3.0
12 R_SMOKE = 10.0
13 T_EFFECT = 20.0
14
15 DT_STEP = 0.015
16
17 R_TAR, H_TAR = 7.0, 10.0
18 CYL_CENTER = np.array([0.0, 200.0, 0.0], dtype=float)
19 P_TARGET = np.array([0.0, 200.0, 5.0], dtype=float)
20
21 MISSILES = [
22     {"name": "M1", "M0": np.array([20000.0, 0.0, 2000.0], dtype=float)},
23     {"name": "M2", "M0": np.array([19000.0, 600.0, 2100.0], dtype=float)},
24     {"name": "M3", "M0": np.array([18000.0, -600.0, 1900.0], dtype=float)},
25 ]
26
27 UAVS = [
28     {"name": "FY1", "U0": np.array([17800.0, 0.0, 1800.0], dtype=float)},
29     {"name": "FY2", "U0": np.array([12000.0, 1400.0, 1400.0], dtype=float)},
30     {"name": "FY3", "U0": np.array([6000.0, -3000.0, 700.0], dtype=float)},
31     {"name": "FY4", "U0": np.array([11000.0, 2000.0, 1800.0], dtype=float)},
32     {"name": "FY5", "U0": np.array([13000.0, -2000.0, 1300.0], dtype=float)},
33 ]
34
35 BUDGETS = (3, 3, 3, 3, 3)
36 MIN_DROP_GAP = 1.0
37
38 HEADING_TOL_DEG = 2.0
39 SPEED_TOL = 1.5
40
41 FRACS = (0.10, 0.18, 0.25, 0.40, 0.55, 0.70, 0.85, 0.92, 0.96, 0.985)
42 ALPHAS = (0.60, 0.70, 0.80, 0.88, 0.92, 0.96, 0.985)
43 TAUS_MUL = (0.55, 0.70, 0.85, 1.00, 1.15, 1.30)
44 PER_UAV_KEEP = 36
45 DEDUP_EPS = 0.12
46
47 N_ANG, N_Z, INCLUDE_SIDE = 48, 9, True
48 POLISH_ROUNDS = 1
49
50 GLOBAL_SA_ON = True
51 SA_ITERS = 2000
52 SA_T0 = 0.5
53 SA_TEND = 1e-3
54 SA_SIG_T = 0.6
55 SA_SIG_A = 0.30

```

```

56
57
58 def unit(v: np.ndarray) -> np.ndarray:
59     n = float(np.linalg.norm(v))
60     return v / n if n > 0 else v
61
62 def missile_hit_time(m0: np.ndarray) -> float:
63     return float(np.linalg.norm(m0) / VM)
64
65 def missile_pos(m0: np.ndarray, t: float) -> np.ndarray:
66     d = unit(-m0)
67     return m0 + VM * d * t
68
69 def uav_pos(u0: np.ndarray, theta: float, v_u: float, t: float) -> np.ndarray:
70     hx, hy = math.cos(theta), math.sin(theta)
71     return np.array([u0[0] + v_u*hx*t, u0[1] + v_u*hy*t, u0[2]], dtype=float)
72
73 def burst_point(u0: np.ndarray, theta: float, v_u: float, t_drop: float, tau: float) -> np.ndarray:
74     hx, hy = math.cos(theta), math.sin(theta)
75     r_drop = uav_pos(u0, theta, v_u, t_drop)
76     horiz = np.array([v_u*hx*tau, v_u*hy*tau, 0.0], dtype=float)
77     vert = np.array([0.0, 0.0, -0.5*g*tau*tau], dtype=float)
78     return r_drop + horiz + vert
79
80 def smoke_center_after_burst(s_burst: np.ndarray, t: float, t_burst: float) -> np.ndarray:
81     dz = -V_SINK * max(0.0, t - t_burst)
82     return s_burst + np.array([0.0, 0.0, dz], dtype=float)
83
84 def point_to_segment_dist(P: np.ndarray, Q: np.ndarray, X: np.ndarray) -> float:
85     v = Q - P
86     vv = float(np.dot(v, v))
87     if vv == 0.0:
88         return float(np.linalg.norm(X - P))
89     a = float(np.dot(X - P, v) / vv)
90     a = 0.0 if a < 0 else (1.0 if a > 1.0 else a)
91     Y = P + a * v
92     return float(np.linalg.norm(X - Y))
93
94 def clip(x, lo, hi): return lo if x < lo else (hi if x > hi else x)
95
96 def covered_L0_at_time(m0, p_target, s_burst, t_burst, t) -> bool:
97     m_t = missile_pos(m0, t)
98     s_t = smoke_center_after_burst(s_burst, t, t_burst)
99     return (point_to_segment_dist(p_target, m_t, s_t) <= R_SMOKE)
100
101 def cyl_points_top_bottom(N_ang: int = 48) -> np.ndarray:
102     cx, cy, _ = CYL_CENTER; out=[]

```



```

103     for z in (0.0, H_TAR):
104         for k in range(N_ang):
105             ang = 2.0*math.pi*k/N_ang
106             out.append((cx+R_TAR*math.cos(ang), cy+R_TAR*math.sin(ang), z))
107     return np.array(out, dtype=float)
108
109 def cyl_points_side(N_ang: int = 48, N_Z: int = 9) -> np.ndarray:
110     cx, cy, _ = CYL_CENTER; zs = np.linspace(0.0, H_TAR, N_Z); out=[]
111     for z in zs:
112         for k in range(N_ang):
113             ang = 2.0*math.pi*k/N_ang
114             out.append((cx+R_TAR*math.cos(ang), cy+R_TAR*math.sin(ang), z))
115     return np.array(out, dtype=float)
116
117 def build_cylinder_samples(N_ang=48, N_Z=9, include_side=True) -> np.ndarray:
118     pts=[cyl_points_top_bottom(N_ang)]
119     if include_side: pts.append(cyl_points_side(N_ang, N_Z))
120     return np.concatenate(pts, axis=0)
121
122 def covered_L1_at_time_vectorized(m0: np.ndarray, s_burst: np.ndarray,
123 t_burst: float, t: float, PTS: np.ndarray) -> bool:
124     m_t = missile_pos(m0, t); s_t = smoke_center_after_burst(s_burst, t, t_burst)
125     v = m_t - PTS; w = s_t - PTS
126     vv = np.sum(v*v, axis=1)
127     alpha = np.divide(np.sum(w*v, axis=1), vv, out=np.zeros_like(vv), where=vv>0.0)
128     alpha = np.clip(alpha, 0.0, 1.0)
129     Y = PTS + alpha[:, None]*v
130     dist = np.linalg.norm(s_t - Y, axis=1)
131     return bool(np.any(dist <= R_SMOKE))
132
133 def _time_grid(T_hit: float, dt: float) -> np.ndarray:
134     return np.arange(0.0, T_hit + 1e-12, dt)
135
136 def _mask_for_candidate_L0(uav_idx: int, cand, tgrids: Dict[str, np.ndarray]) -> Dict[str, np.ndarray]:
137     masks={}
138     u0 = UAVS[uav_idx]["U0"]
139     th, v, td, ta = cand["theta"], cand["v"], cand["t_drop"], cand["tau"]
140     t_burst = td + ta
141     for m in MISSILES:
142         name, m0 = m["name"], m["M0"]
143         T_HIT = missile_hit_time(m0)
144         tgrid = tgrids[name]
145         mask = np.zeros_like(tgrid, dtype=bool)
146         if t_burst < T_HIT:
147             sb = burst_point(u0, th, v, td, ta)
148             if sb[2] > 0.0:
149                 t_start, t_end = t_burst, min(t_burst + T_EFFECT, T_HIT)

```

```

150         idx = np.where((tgrid >= t_start) & (tgrid <= t_end))[0]
151         for k in idx:
152             t = float(tgrid[k])
153             if covered_L0_at_time(m0, P_TARGET, sb, t_burst, t):
154                 mask[k] = True
155         masks[name]=mask
156     return masks
157
158 def _mask_for_candidate_L1(uav_idx: int, cand, tgrids: Dict[str, np.ndarray], PTS: np.ndarray) ->
159 Dict[str, np.ndarray]:
160     masks={}
161     u0 = UAVS[uav_idx]["U0"]
162     th, v, td, ta = cand["theta"], cand["v"], cand["t_drop"], cand["tau"]
163     t_burst = td + ta
164     for m in MISSILES:
165         name, m0 = m["name"], m["M0"]
166         T_HIT = missile_hit_time(m0)
167         tgrid = tgrids[name]
168         mask = np.zeros_like(tgrid, dtype=bool)
169         if t_burst < T_HIT:
170             sb = burst_point(u0, th, v, td, ta)
171             if sb[2] > 0.0:
172                 t_start, t_end = t_burst, min(t_burst + T_EFFECT, T_HIT)
173                 idx = np.where((tgrid >= t_start) & (tgrid <= t_end))[0]
174                 for k in idx:
175                     t = float(tgrid[k])
176                     if covered_L1_at_time_vectorized(m0, sb, t_burst, t, PTS):
177                         mask[k] = True
178         masks[name]=mask
179     return masks
180
181 def _cover_sum(masks: Dict[str, np.ndarray], dt: float) -> float:
182     return float(sum(mask.sum() for mask in masks.values()) * dt)
183
184 def _candidate_from_anchor(u0: np.ndarray, m0: np.ndarray,
185                             frac: float, alpha: float, tau_mult: float,
186                             clamp_eps: float = 0.10):
187     T_HIT = missile_hit_time(m0)
188     t_b = clip(frac * min(60.0, T_HIT-2.0), 0.5, 59.5)
189     m_tb = missile_pos(m0, t_b)
190     Y = P_TARGET + alpha*(m_tb - P_TARGET)
191     dx, dy = Y[0]-u0[0], Y[1]-u0[1]
192     theta = math.atan2(dy, dx)
193     D_xy = math.hypot(dx, dy)
194     v = clip(D_xy / t_b, 70.0, 140.0)
195     u0z, Yz = float(u0[2]), float(Y[2])
196     tau_base = math.sqrt(max(0.0, 2.0*(u0z - Yz))/g) if u0z > Yz else 0.2

```

```

197 tau_max_by_t = max(0.2, t_b - clamp_eps)
198 tau = clip(tau_base * tau_mult, 0.2, min(12.0, tau_max_by_t))
199 t_drop = t_b - tau
200 return theta%(2.0*math.pi), v, t_drop, tau
201
202 def build_candidates_L0(
203     dt: float = DT_STEP,
204     fracs=FRACS, alphas=ALPHAS, taus=TAUS_MUL,
205     per_uav_keep: int = PER_UAV_KEEP, dedup_eps: float = DEDUP_EPS
206 ) -> Tuple[List[Dict[str, Any]], Dict[str, np.ndarray]]:
207     tgrids = {m["name"]: _time_grid(missile_hit_time(m["M0"]), dt) for m in MISSILES}
208     all_cands=[]
209     for u in range(len(UAVS)):
210         u0 = UAVS[u]["U0"]; local=[]
211         for m in MISSILES:
212             m0 = m["M0"]
213             for f in fracs:
214                 for a in alphas:
215                     for mul in taus:
216                         th, v, td, ta = _candidate_from_anchor(u0, m0, f, a, mul)
217                         cand = {"uav": u, "theta": th, "v": v, "t_drop": td, "tau": ta}
218                         masks = _mask_for_candidate_L0(u, cand, tgrids)
219                         score = _cover_sum(masks, dt)
220                         if score <= 0.0:
221                             continue
222                         cand.update({"mask_by_missile": masks, "score_cover": score, "t_burst": td+ta})
223                         local.append(cand)
224                     local.sort(key=lambda c:c["t_burst"])
225                 filtered=[]
226                 for c in local:
227                     if not filtered or abs(c["t_burst"]-filtered[-1]["t_burst"])>dedup_eps:
228                         filtered.append(c)
229                     else:
230                         if c["score_cover"] > filtered[-1]["score_cover"]:
231                             filtered[-1]=c
232                 filtered.sort(key=lambda c:c["score_cover"], reverse=True)
233                 all_cands += filtered[:per_uav_keep]
234     return all_cands, tgrids
235
236 def _angdiff(a: float, b: float) -> float:
237     d = abs((a - b + math.pi) % (2*math.pi) - math.pi)
238     return d
239
240 def _feasible_with(cand, chosen, budgets, min_drop_gap,
241     locks: Dict[int, Optional[Tuple[float,float]]],
242     heading_tol_rad: float, speed_tol: float) -> bool:
243     u = cand["uav"]

```

```

244     if budgets[u] <= 0: return False
245     for c in chosen:
246         if c["uav"]==u and abs(float(c["t_drop"]) - float(cand["t_drop"])) < min_drop_gap:
247             return False
248     thv = locks.get(u)
249     if thv is not None:
250         th_ref, v_ref = thv
251         if _angdiff(cand["theta"], th_ref) > heading_tol_rad: return False
252         if abs(cand["v"] - v_ref) > speed_tol: return False
253     return True
254
255 def _marginal_gain_cover(cand, union_masks, tgrids, dt):
256     gain = 0.0
257     for name in tgrids:
258         old = union_masks[name]
259         new = np.logical_or(old, cand["mask_by_missile"][name])
260         gain += float((new.sum() - old.sum()) * dt)
261     return gain
262
263
264 def seed_cover_all_missiles(candidates: List[Dict[str,Any]],
265                             tgrids: Dict[str,np.ndarray],
266                             dt: float,
267                             budgets_per_uav=BUDGETS,
268                             min_drop_gap: float = MIN_DROP_GAP,
269                             heading_tol_deg: float = HEADING_TOL_DEG,
270                             speed_tol: float = SPEED_TOL):
271     budgets = {i: budgets_per_uav[i] for i in range(len(UAVS))}
272     locks: Dict[int, Optional[Tuple[float,float]]] = {i: None for i in range(len(UAVS))}
273     union_masks = {name: np.zeros_like(tgrids[name], dtype=bool) for name in tgrids}
274     chosen: List[Dict[str,Any]] = []
275     tol_rad = math.radians(heading_tol_deg)
276
277     for m in MISSILES:
278         name = m["name"]
279         best=None; best_gain=-1e18
280         for c in candidates:
281             if c["mask_by_missile"][name].sum() == 0:
282                 continue
283             if not _feasible_with(c, chosen, budgets, min_drop_gap, locks, tol_rad, speed_tol):
284                 continue
285             old = union_masks[name]
286             new = np.logical_or(old, c["mask_by_missile"][name])
287             gain = float((new.sum() - old.sum()) * dt)
288             if gain > best_gain:
289                 best, best_gain = c, gain
290         if best is not None and best_gain > 0.0:

```

```

291     chosen.append(best)
292     u = best["uav"]
293     budgets[u] -= 1
294     if locks[u] is None:
295         locks[u] = (best["theta"], best["v"])
296     for nm in tgrids:
297         union_masks[nm] = np.logical_or(union_masks[nm], best["mask_by_missile"][nm])
298     return chosen, union_masks, budgets, locks
299
300 def augment_locked_same_course(uav_idx: int, theta_ref: float, v_ref: float,
301                                tgrids: Dict[str, np.ndarray], dt: float,
302                                t_step: float = 1.0,
303                                tau_list = (0.3, 0.5, 0.8, 1.2, 2.0, 3.0, 6.0, 9.0, 12.0),
304                                dedup_eps: float = 0.10):
305     u0 = UAVS[uav_idx]["U0"]
306     T_hit_min = min(missile_hit_time(m["M0"]) for m in MISSILES)
307     t_end = max(3.0, min(59.0, T_hit_min - 0.5))
308     t_bursts = np.arange(3.0, t_end + 1e-9, t_step)
309
310     cands = []
311     for t_b in t_bursts:
312         for tau in tau_list:
313             t_drop = t_b - tau
314             if t_drop < 0.0 or t_drop > 60.0:
315                 continue
316             cand = {"uav": uav_idx, "theta": theta_ref, "v": v_ref, "t_drop": t_drop, "tau": tau}
317             sb = burst_point(u0, theta_ref, v_ref, t_drop, tau)
318             if sb[2] <= 0.0:
319                 continue
320             masks = _mask_for_candidate_L0(uav_idx, cand, tgrids)
321             score = _cover_sum(masks, dt)
322             cand.update({"mask_by_missile": masks, "score_cover": score, "t_burst": t_b})
323             cands.append(cand)
324
325     cands.sort(key=lambda c: c["t_burst"])
326     filtered = []
327     for c in cands:
328         if not filtered or abs(c["t_burst"] - filtered[-1]["t_burst"]) > dedup_eps:
329             filtered.append(c)
330         else:
331             if c["score_cover"] > filtered[-1]["score_cover"]:
332                 filtered[-1] = c
333     filtered.sort(key=lambda c: c["score_cover"], reverse=True)
334     return filtered
335
336 def greedy_fill_after_seed(candidates, tgrids, dt,
337                            seed_chosen, seed_union, seed_budgets, seed_locks,

```

```

338         budgets_per_uav=BUDGETS,
339         min_drop_gap: float = MIN_DROP_GAP,
340         heading_tol_deg: float = HEADING_TOL_DEG,
341         speed_tol: float = SPEED_TOL):
342     chosen = seed_chosen[:]
343     union_masks = {nm: seed_union[nm].copy() for nm in tgrids}
344     budgets = seed_budgets.copy()
345     locks = seed_locks.copy()
346     tol_rad = math.radians(heading_tol_deg)
347
348     pool = candidates[:]
349     for u in range(len(UAVS)):
350         if locks[u] is not None:
351             th_ref, v_ref = locks[u]
352             extra = augment_locked_same_course(u, th_ref, v_ref, tgrids, dt, t_step=1.0)
353             pool.extend(extra)
354
355     while sum(max(0,b) for b in budgets.values()) > 0:
356         best=None; best_gain=-1e18
357         for c in pool:
358             if not _feasible_with(c, chosen, budgets, min_drop_gap, locks, tol_rad, speed_tol):
359                 continue
360             gain = _marginal_gain_cover(c, union_masks, tgrids, dt)
361             if gain > best_gain:
362                 best, best_gain = c, gain
363         if best is None:
364             break
365         chosen.append(best)
366         u = best["uav"]
367         budgets[u] -= 1
368         if locks[u] is None:
369             locks[u] = (best["theta"], best["v"])
370             pool.extend(augment_locked_same_course(u, locks[u][0], locks[u][1], tgrids, dt, t_step=1.0))
371         for nm in tgrids:
372             union_masks[nm] = np.logical_or(union_masks[nm], best["mask_by_missile"][nm])
373         pool = [x for x in pool if x is not best]
374         if sum(max(0,b) for b in budgets.values()) == 0:
375             break
376     return chosen, union_masks, budgets, locks
377
378 def _force_fill_to_15(chosen_list, tgrids_loc, dt_loc):
379     locks_tmp = {}
380     for c in chosen_list:
381         u = c["uav"]
382         if u not in locks_tmp:
383             locks_tmp[u] = (c["theta"], c["v"])
384     for u in range(len(UAVS)):

```

```

385     if u not in locks_tmp:
386         u0 = UAVS[u]["U0"]
387         th = math.atan2(P_TARGET[1]-u0[1], P_TARGET[0]-u0[0])
388         locks_tmp[u] = (th, 100.0)
389
390     used_per_uav = {i:0 for i in range(len(UAVS))}
391     for c in chosen_list:
392         used_per_uav[c["uav"]] += 1
393
394     def ok_tdrop(u, td):
395         for x in chosen_list:
396             if x["uav"] == u and abs(float(x["t_drop"]) - float(td)) < MIN_DROP_GAP:
397                 return False
398         return True
399
400     for u in range(len(UAVS)):
401         th, v = locks_tmp[u]
402         pool_u = augment_locked_same_course(u, th, v, tgrids_loc, dt_loc, t_step=1.0)
403         k = 0
404         while used_per_uav[u] < BUDGETS[u]:
405             pick = None
406             while k < len(pool_u):
407                 if ok_tdrop(u, pool_u[k]["t_drop"]):
408                     pick = pool_u[k]; k += 1; break
409             k += 1
410             if pick is None:
411                 td = 0.0
412                 while td <= 60.0 and (not ok_tdrop(u, td)):
413                     td += 0.25
414                 tau = 0.3
415                 c = {"uav": u, "theta": th, "v": v, "t_drop": td, "tau": tau, "t_burst": td+tau}
416                 c["mask_by_missile"] = _mask_for_candidate_L0(u, c, tgrids_loc)
417                 c["score_cover"] = _cover_sum(c["mask_by_missile"], dt_loc)
418                 pick = c
419             chosen_list.append(pick)
420             used_per_uav[u] += 1
421     return chosen_list
422
423 def polish_L1_keep_course(selected: List[Dict[str,Any]],
424                           tgrids: Dict[str,np.ndarray],
425                           dt_mask: float = DT_STEP,
426                           rounds: int = POLISH_ROUNDS,
427                           N_ang:int=48, N_Z:int=9, INCLUDE_SIDE:bool=True):
428     PTS = build_cylinder_samples(N_ang, N_Z, INCLUDE_SIDE)
429
430     def rebuild_masks_L1(c):
431         c2 = dict(c)

```

```

432     c2["mask_by_missile"] = _mask_for_candidate_L1(c["uav"], c, tgrids, PTS)
433     c2["score_cover"] = _cover_sum(c2["mask_by_missile"], dt_mask)
434     return c2
435
436 cur = [rebuild_masks_L1(c) for c in selected]
437
438 def score_of(sol):
439     union = {nm: np.zeros_like(tgrids[nm], dtype=bool) for nm in tgrids}
440     for c in sol:
441         for nm in tgrids:
442             union[nm] = np.logical_or(union[nm], c["mask_by_missile"][nm])
443     sc = sum(float(union[nm].sum()) * dt_mask for nm in tgrids)
444     return sc, union
445
446 def ok_gap_with_others(u, td, others):
447     for x in others:
448         if x["uav"]==u and abs(float(x["t_drop"]) - float(td)) < MIN_DROP_GAP:
449             return False
450     return True
451
452 best = cur[:]
453 best_score, best_union = score_of(best)
454 for _ in range(rounds):
455     changed=False
456     for i in range(len(best)):
457         base = best[:i] + best[i+1:]
458         c = best[i]
459         tds = [clip(c["t_drop"] + d, 0.0, 60.0) for d in (-1.2,-0.8,-0.4,0.0,0.4,0.8,1.2)]
460         taus = [clip(c["tau"] * r, 0.2, 12.0) for r in (0.85,1.0,1.15)]
461         best_local=c; best_local_score=best_score
462         for td in tds:
463             if not ok_gap_with_others(c["uav"], td, base):
464                 continue
465             for ta in taus:
466                 cand = dict(c)
467                 cand["t_drop"]=td; cand["tau"]=ta; cand["t_burst"]=td+ta
468                 cand = rebuild_masks_L1(cand)
469                 sc,_ = score_of(base + [cand])
470                 if sc > best_local_score:
471                     best_local, best_local_score = cand, sc
472             if best_local_score > best_score:
473                 best[i]=best_local; best_score=best_local_score; changed=True
474         if not changed: break
475 final_union = {nm: np.zeros_like(tgrids[nm], dtype=bool) for nm in tgrids}
476 for c in best:
477     for nm in tgrids:
478         final_union[nm] = np.logical_or(final_union[nm], c["mask_by_missile"][nm])

```



```

479     return best, final_union
480
481 def global_sa_optimize_L1(selected: List[Dict[str,Any]],
482                          tgrids: Dict[str,np.ndarray],
483                          dt_mask: float = DT_STEP,
484                          iters: int = SA_ITERS,
485                          T0: float = SA_T0,
486                          Tend: float = SA_TEND,
487                          sig_t: float = SA_SIG_T,
488                          sig_tau_rel: float = SA_SIG_A,
489                          N_ang:int=48, N_Z:int=9, INCLUDE_SIDE:bool=True):
490
491     PTS = build_cylinder_samples(N_ang, N_Z, INCLUDE_SIDE)
492
493     def rebuild(c):
494         c2 = dict(c)
495         c2["mask_by_missile"] = _mask_for_candidate_L1(c["uav"], c2, tgrids, PTS)
496         c2["score_cover"] = _cover_sum(c2["mask_by_missile"], dt_mask)
497         return c2
498
499     def score_of(sol):
500         union = {nm: np.zeros_like(tgrids[nm], dtype=bool) for nm in tgrids}
501         for c in sol:
502             for nm in tgrids:
503                 union[nm] = np.logical_or(union[nm], c["mask_by_missile"][nm])
504         sc = sum(float(union[nm].sum()) * dt_mask for nm in tgrids)
505         return sc, union
506
507     def ok_gap_for_idx(sol, idx, td_new):
508         u = sol[idx]["uav"]
509         for j,c in enumerate(sol):
510             if j==idx: continue
511             if c["uav"]==u and abs(float(c["t_drop"]) - float(td_new)) < MIN_DROP_GAP:
512                 return False
513         return True
514
515     cur = [rebuild(c) for c in selected]
516     cur_score, cur_union = score_of(cur)
517     best = [dict(c) for c in cur]
518     best_score = cur_score
519
520     def snap(x):
521         k = round(x / DT_STEP)
522         return float(max(0.0, min(60.0, k * DT_STEP)))
523
524     for k in range(iters):
525         i = rng.integers(0, len(cur))

```

```

526     c0 = cur[i]
527     td0, ta0 = float(c0["t_drop"]), float(c0["tau"])
528
529     trial = dict(c0)
530     for _attempt in range(12):
531         td = snap(td0 + rng.normal(0.0, sig_t))
532         if not ok_gap_for_idx(cur, i, td):
533             continue
534         ta = float(clip(ta0 * (1.0 + rng.normal(0.0, sig_tau_rel)), 0.2, 12.0))
535         trial["t_drop"] = td
536         trial["tau"] = ta
537         trial["t_burst"] = td + ta
538         trial = rebuild(trial)
539         break
540     else:
541         continue
542
543     new_sol = cur[:i] + [trial] + cur[i+1:]
544     new_score, _ = score_of(new_sol)
545
546     T = T0 * (Tend / T0) ** (k / max(1, iters-1))
547     if new_score >= cur_score or rng.random() < math.exp((new_score - cur_score) / max(1e-12, T)):
548         cur = new_sol
549         cur_score = new_score
550         if cur_score > best_score:
551             best = [dict(x) for x in cur]
552             best_score = cur_score
553
554     final_union = {nm: np.zeros_like(tgrids[nm], dtype=bool) for nm in tgrids}
555     for c in best:
556         for nm in tgrids:
557             final_union[nm] = np.logical_or(final_union[nm], c["mask_by_missile"][nm])
558     return best, final_union
559
560
561 def __drop_point(u0: np.ndarray, theta: float, v_u: float, t_drop: float) -> np.ndarray:
562     return uav_pos(u0, theta, v_u, t_drop)
563
564 def build_report_rows(selected_raw: List[Dict[str, Any]],
565                       tgrids: Dict[str, np.ndarray],
566                       dt: float) -> List[Dict[str, Any]]:
567
568     by_uav={}
569     for c in selected_raw:
570         by_uav.setdefault(c["uav"], []).append(c)
571     for u, lst in by_uav.items():
572         lst.sort(key=lambda x: x["t_drop"])

```

```

573     for idx,c in enumerate(lst, start=1):
574         c["_seq_in_uav"] = idx
575
576     rows=[]
577     selected_sorted = sorted(selected_raw, key=lambda c:(c["uav"], c["t_drop"]))
578     for c in selected_sorted:
579         u_idx=c["uav"]; u_name=UAVS[u_idx]["name"]; u0=UAVS[u_idx]["U0"]
580         theta_deg=(math.degrees(c["theta"])%360.0); v_u=float(c["v"])
581         t_drop=float(c["t_drop"]); tau=float(c["tau"])
582         r_drop = _drop_point(u0, c["theta"], v_u, t_drop)
583         s_burst = burst_point(u0, c["theta"], v_u, t_drop, tau)
584         smoke_id=f"{u_name}-{c.get('_seq_in_uav',1)}"
585         had_positive=False
586         for m in MISSILES:
587             name=m["name"]; mask=c["mask_by_missile"][name]
588             eff_s=float(mask.sum()*dt)
589             if eff_s<=0.0: continue
590             had_positive=True
591             rows.append({
592                 " 无人机编号": u_name,
593                 " 无人机运动方向": round(theta_deg,3),
594                 " 无人机运动速度 (m/s)": round(v_u,3),
595                 " 烟幕干扰弹编号": smoke_id,
596                 " 烟幕干扰弹投放点的 x 坐标 (m)": round(float(r_drop[0]),3),
597                 " 烟幕干扰弹投放点的 y 坐标 (m)": round(float(r_drop[1]),3),
598                 " 烟幕干扰弹投放点的 z 坐标 (m)": round(float(r_drop[2]),3),
599                 " 烟幕干扰弹起爆点的 x 坐标 (m)": round(float(s_burst[0]),3),
600                 " 烟幕干扰弹起爆点的 y 坐标 (m)": round(float(s_burst[1]),3),
601                 " 烟幕干扰弹起爆点的 z 坐标 (m)": round(float(s_burst[2]),3),
602                 " 有效干扰时长 (s)": round(eff_s,3),
603                 " 干扰的导弹编号": name
604             })
605         if not had_positive:
606             rows.append({
607                 " 无人机编号": u_name,
608                 " 无人机运动方向": round(theta_deg,3),
609                 " 无人机运动速度 (m/s)": round(v_u,3),
610                 " 烟幕干扰弹编号": smoke_id,
611                 " 烟幕干扰弹投放点的 x 坐标 (m)": round(float(r_drop[0]),3),
612                 " 烟幕干扰弹投放点的 y 坐标 (m)": round(float(r_drop[1]),3),
613                 " 烟幕干扰弹投放点的 z 坐标 (m)": round(float(r_drop[2]),3),
614                 " 烟幕干扰弹起爆点的 x 坐标 (m)": round(float(s_burst[0]),3),
615                 " 烟幕干扰弹起爆点的 y 坐标 (m)": round(float(s_burst[1]),3),
616                 " 烟幕干扰弹起爆点的 z 坐标 (m)": round(float(s_burst[2]),3),
617                 " 有效干扰时长 (s)": 0.0,
618                 " 干扰的导弹编号": "-"
619             })

```

```

620     return rows
621
622 def print_report_rows(rows: List[Dict[str, Any]]):
623     headers = [
624         " 无人机编号", " 无人机运动方向", " 无人机运动速度 (m/s)", " 烟幕干扰弹编号",
625         " 烟幕干扰弹投放点的 x 坐标 (m)", " 烟幕干扰弹投放点的 y 坐标 (m)", " 烟幕干扰弹投放点的 z 坐标 (m)",
626         " 烟幕干扰弹起爆点的 x 坐标 (m)", " 烟幕干扰弹起爆点的 y 坐标 (m)", " 烟幕干扰弹起爆点的 z 坐标 (m)",
627         " 有效干扰时长 (s)", " 干扰的导弹编号"
628     ]
629     print("\t".join(headers))
630     for r in rows:
631         print("\t".join(str(r[h]) for h in headers))
632
633 def solve_q5(
634     dt_step: float = DT_STEP,
635     do_polish_L1: bool = True,
636     do_global_SA: bool = GLOBAL_SA_ON,
637     heading_tol_deg: float = HEADING_TOL_DEG,
638     speed_tol: float = SPEED_TOL
639 ) -> Dict[str, Any]:
640     candidates_L0, tgrids_L0 = build_candidates_L0(dt=dt_step)
641     counts = {i: sum(1 for c in candidates_L0 if c["uav"]==i) for i in range(len(UAVS))}
642     print("[debug] L0 per-UAV nonzero candidates:", counts)
643
644     seed_chosen, seed_union, budgets, locks = seed_cover_all_missiles(
645         candidates_L0, tgrids_L0, dt_step,
646         budgets_per_uav=BUDGETS,
647         min_drop_gap=MIN_DROP_GAP,
648         heading_tol_deg=heading_tol_deg, speed_tol=speed_tol
649     )
650     chosen, union, budgets, locks = greedy_fill_after_seed(
651         candidates_L0, tgrids_L0, dt_step,
652         seed_chosen, seed_union, budgets, locks,
653         budgets_per_uav=BUDGETS,
654         min_drop_gap=MIN_DROP_GAP,
655         heading_tol_deg=heading_tol_deg, speed_tol=speed_tol
656     )
657     chosen = _force_fill_to_15(chosen, tgrids_L0, dt_step)
658     assert len(chosen) == sum(BUDGETS), f"[fatal] 只选了 {len(chosen)}/{sum(BUDGETS)}"
659
660     mode = "L0 only"
661     tgrids = {m["name"]: _time_grid(missile_hit_time(m["M0"]), dt_step) for m in MISSILES}
662     if do_polish_L1:
663         chosen, union = polish_L1_keep_course(chosen, tgrids, dt_mask=dt_step,
664             rounds=POLISH_ROUNDS, N_ang=N_ANG,
665
666             N_Z=N_Z, INCLUDE_SIDE=INCLUDE_SIDE)

```

```

667     mode = "L0 → L1(polish)"
668
669 if do_global_SA:
670     before_score = sum(float(union[nm].sum()*dt_step) for nm in union)
671     chosen, union = global_sa_optimize_L1(
672         selected=chosen, tgrids=tgrids, dt_mask=dt_step,
673         iters=SA_ITERS, T0=SA_T0, Tend=SA_TEND,
674         sig_t=SA_SIG_T, sig_tau_rel=SA_SIG_A,
675         N_ang=N_ANG, N_Z=N_Z, INCLUDE_SIDE=INCLUDE_SIDE
676     )
677     after_score = sum(float(union[nm].sum()*dt_step) for nm in union)
678     print(f"[info] SA 提升覆盖: {before_score:.3f} → {after_score:.3f} 秒")
679     mode += " → L1(SA)"
680
681 per_missile=[]
682 for m in MISSILES:
683     name=m["name"]; tgrid=tgrids[name]; mask=union[name]
684     cover=float(mask.sum()*dt_step)
685     intervals=[]; in_seg=False; a=None
686     for k in range(len(mask)):
687         if mask[k] and not in_seg: in_seg=True; a=float(tgrid[k])
688         if in_seg and (k==len(mask)-1 or (not mask[k+1])):
689             b=float(tgrid[k]); intervals.append((round(a,3), round(b,3))); in_seg=False
690     per_missile.append({
691         "missile": name,
692         "T_hit": round(missile_hit_time(m["M0"]),3),
693         "cover_s": round(cover,3),
694         "intervals": intervals
695     })
696 cover_sum = round(sum(x["cover_s"] for x in per_missile), 3)
697
698 out_sel=[]
699 for c in chosen:
700     u0 = UAVS[c["uav"]]["U0"]
701     sb = burst_point(u0, c["theta"], c["v"], c["t_drop"], c["tau"])
702     out_sel.append({
703         "uav": UAVS[c["uav"]]["name"],
704         "theta_deg": round(math.degrees(c["theta"]),3),
705         "v_u_mps": round(float(c["v"]),3),
706         "t_drop": round(float(c["t_drop"]),3),
707         "tau": round(float(c["tau"]),3),
708         "t_burst": round(float(c["t_drop"])+c["tau"]),3),
709         "s_burst": (round(float(sb[0]),3), round(float(sb[1]),3), round(float(sb[2]),3))
710     })
711
712 rows = build_report_rows(chosen, tgrids, dt_step)
713

```

```

714     return {
715         "method": "graph_Q5",
716         "mode": mode,
717         "selected": out_sel,
718         "per_missile": per_missile,
719         "cover_sum_s": cover_sum,
720         "used_total": len(chosen),
721         "config": {
722             "dt_step": dt_step,
723             "budgets": BUDGETS,
724             "min_drop_gap": MIN_DROP_GAP,
725             "heading_tol_deg": HEADING_TOL_DEG, "speed_tol": SPEED_TOL,
726             "L1_polish": do_polish_L1,
727             "L1_SA": do_global_SA,
728             "SA_iters": SA_ITERS,
729             "N_ANG": N_ANG, "N_Z": N_Z, "INCLUDE_SIDE": INCLUDE_SIDE
730         },
731         "rows": rows
732     }
733
734 def _name_to_idx():
735     return {UAVS[i]["name"]: i for i in range(len(UAVS))}
736
737 def _name2idx(): # 兼容旧名
738     return _name_to_idx()
739
740 def _ans_to_internal_selected(ans_selected):
741     n2i = _name2idx()
742     internal=[]
743     for r in ans_selected:
744         internal.append({
745             "uav": n2i[str(r["uav"])],
746             "theta": math.radians(float(r["theta_deg"])),
747             "v": float(r["v_u_mps"]),
748             "t_drop": float(r["t_drop"]),
749             "tau": float(r["tau"]),
750             "t_burst": float(r["t_drop"])+float(r["tau"])
751         })
752     return internal
753
754 def _recompute_cover_L1(selected_internal, dt: float, N_ANG:int=48, N_Z:int=9, INCLUDE_SIDE:
755 bool=True):
756     tgrids = {m["name"]: _time_grid(missile_hit_time(m["M0"]), dt) for m in MISSILES}
757     union = {nm: np.zeros_like(tgrids[nm], dtype=bool) for nm in tgrids}
758     PTS = build_cylinder_samples(N_ang=N_ANG, N_Z=N_Z, include_side=INCLUDE_SIDE)
759
760     def rebuild_masks_L1(c):

```

```

761     c2 = dict(c)
762     c2["mask_by_missile"] = _mask_for_candidate_L1(c2["uav"], c2, tgrids, PTS)
763     return c2
764
765     for c in selected_internal:
766         cc = rebuild_masks_L1(c)
767         for nm in tgrids:
768             union[nm] = np.logical_or(union[nm], cc["mask_by_missile"][nm])
769
770     cover_sum = float(sum(union[nm].sum() * dt for nm in union))
771     return cover_sum, union, tgrids
772
773 def _repair_min_gap_per_uav(selected_internal, min_gap: float = MIN_DROP_GAP):
774     by_uav = {}
775     for c in selected_internal:
776         by_uav.setdefault(c["uav"], []).append(c)
777     for u, lst in by_uav.items():
778         lst.sort(key=lambda x: x["t_drop"])
779         last = -1e9
780         for c in lst:
781             td = float(c["t_drop"])
782             if td < last + min_gap:
783                 td = last + min_gap
784                 td = clip(td, 0.0, 60.0)
785                 c["t_drop"] = td
786                 c["t_burst"] = td + float(c["tau"])
787             last = float(c["t_drop"])
788     return selected_internal
789
790 def _perturb_solution(selected_internal, sigma_t: float = 0.6, sigma_tau_rel: float = 0.3, seed: int = None):
791     rng_loc = np.random.default_rng(seed)
792     out=[]
793     for c in selected_internal:
794         td = float(c["t_drop"]) + float(rng_loc.normal(0.0, sigma_t))
795         ta = float(c["tau"]) * (1.0 + float(rng_loc.normal(0.0, sigma_tau_rel)))
796         td = clip(td, 0.0, 60.0)
797         ta = clip(ta, 0.2, 12.0)
798         out.append({
799             "uav": c["uav"], "theta": float(c["theta"]), "v": float(c["v"]),
800             "t_drop": td, "tau": ta, "t_burst": td+ta
801         })
802     return _repair_min_gap_per_uav(out, MIN_DROP_GAP)
803
804 def _random_solution_global_like(selected_internal, seed: int = None):
805     rng_loc = np.random.default_rng(seed)
806     thv = {}
807     for c in selected_internal:

```

```

808     thv[c["uav"]] = (float(c["theta"]), float(c["v"]))
809 cnt_per_uav = {i:0 for i in range(len(UAVS))}
810 out=[]
811 for u in range(len(UAVS)):
812     th, v = thv[u]
813     K = BUDGETS[u]
814     tds = list(rng_loc.uniform(0.0, 60.0, size=K))
815     tds.sort()
816     for i in range(1, K):
817         if tds[i] < tds[i-1] + MIN_DROP_GAP:
818             tds[i] = min(60.0, tds[i-1] + MIN_DROP_GAP)
819     for i in range(K):
820         td = float(tds[i])
821         tau = float(rng_loc.uniform(0.2, 12.0))
822         out.append({"uav": u, "theta": th, "v": v, "t_drop": td, "tau": tau, "t_burst": td+tau})
823 return out
824
825
826 def monte_carlo_compare(ans, n_trials: int = 100,
827                         sigma_t: float = 0.6, sigma_tau_rel: float = 0.3,
828                         dt: float = DT_STEP, N_ANG: int = 48, N_Z: int = 9, INCLUDE_SIDE: bool = True,
829                         also_global_random: bool = True):
830     base_sel = _ans_to_internal_selected(ans["selected"])
831     base_cov, _, _ = _recompute_cover_L1(base_sel, dt, N_ANG, N_Z, INCLUDE_SIDE)
832
833     better_local = 0
834     cov_local = []
835     for i in range(n_trials):
836         cand = _perturb_solution(base_sel, sigma_t=sigma_t, sigma_tau_rel=sigma_tau_rel, seed=2024+i)
837         cov, _, _ = _recompute_cover_L1(cand, dt, N_ANG, N_Z, INCLUDE_SIDE)
838         cov_local.append(cov)
839         if cov > base_cov + 1e-9:
840             better_local += 1
841
842     print("\n[MC-Local] 局部扰动对比:")
843     print(f" 基线覆盖 (L1): {base_cov:.3f} s")
844     print(f" {n_trials} 组扰动 覆盖均值/中位/最大/最小: {stats.mean(cov_local):.3f}
845
846     / {stats.median(cov_local):.3f} / {max(cov_local):.3f} / {min(cov_local):.3f}")
847     print(f" 超过基线的比例: {better_local}/{n_trials} = {better_local/n_trials:.2%}")
848
849     cov_glob = None
850     if also_global_random:
851         better_glob = 0
852         cov_glob = []
853         for i in range(n_trials):
854             cand = _random_solution_global_like(base_sel, seed=4096+i)

```



```

855     cov, __, __ = _recompute_cover_L1(cand, dt, N_ANG, N_Z, INCLUDE_SIDE)
856     cov_glob.append(cov)
857     if cov > base_cov + 1e-9:
858         better_glob += 1
859     print("\n[MC-Global] 全局随机对比 (锁 (,v)) :")
860     print(f" {n_trials} 组随机 覆盖均值/中位/最大/最小: {stats.mean(cov_glob):.3f} /
861
862     {stats.median(cov_glob):.3f} / {max(cov_glob):.3f} / {min(cov_glob):.3f}")
863     print(f" 超过基线的比例: {better_glob}/{n_trials} = {better_glob/n_trials:.2%}")
864
865     return {
866         "base": base_cov,
867         "local": cov_local,
868         "global": cov_glob
869     }
870
871
872 def sweep_dt(ans, dt_list=(0.010, 0.015, 0.020, 0.030), N_ANG:int=48, N_Z:int=9, INCLUDE_SIDE
873 :bool=True):
874     sel = _ans_to_internal_selected(ans["selected"])
875     out=[]
876     for dt in dt_list:
877         cov, __, __ = _recompute_cover_L1(sel, dt, N_ANG, N_Z, INCLUDE_SIDE)
878         out.append((dt, cov))
879     base = out[0][1]
880     print("\n[稳健性] 步长灵敏度 (L1): ")
881     for dt, cov in out:
882         print(f" dt={dt:.3f} cover={cov:.3f} 相对偏差 = {(cov-base)/max(1e-12,base):+.2%}")
883     return out
884
885 def sweep_sampling(ans, NANG_list=(24,48,96), NZ_list=(5,9,13), dt: float = DT_STEP):
886     sel = _ans_to_internal_selected(ans["selected"])
887     base_cov, __, __ = _recompute_cover_L1(sel, dt, N_ANG=48, N_Z=9, INCLUDE_SIDE=True)
888     print("\n[稳健性] 采样密度灵敏度 (L1): ")
889     print(f" 基线 (N_ANG=48,N_Z=9): {base_cov:.3f} s")
890     out=[]
891     for na in NANG_list:
892         for nz in NZ_list:
893             cov, __, __ = _recompute_cover_L1(sel, dt, N_ANG=na, N_Z=nz, INCLUDE_SIDE=True)
894             out.append((na, nz, cov))
895             print(f" N_ANG={na:>3}, N_Z={nz:>2} -> cover={cov:.3f}
896             偏差 = {(cov-base_cov)/max(1e-12,base_cov):+.2%}")
897     return out
898
899 def perturbation_curve(ans, sig_t_grid=(0.0,0.2,0.4,0.6,0.8,1.0), sig_tau_rel: float = 0.30,
900     reps: int = 30, dt: float = DT_STEP, N_ANG:int=48, N_Z:int=9):
901     sel = _ans_to_internal_selected(ans["selected"])

```

```

902 base_cov, __, _ = _recompute_cover_L1(sel, dt, N_ANG, N_Z, True)
903 print("\n[扰动曲线] 覆盖-噪声关系 ( _t, 固定 _tau_rel=%.2f) : " % sig_tau_rel)
904 print(f" 基线覆盖: {base_cov:.3f} s")
905 table=[]
906 for st in sig_t_grid:
907     vals=[]
908     for r in range(reps):
909         cand = _perturb_solution(sel, sigma_t=st, sigma_tau_rel=sig_tau_rel, seed=7000+r)
910         cov, __, _ = _recompute_cover_L1(cand, dt, N_ANG, N_Z, True)
911         vals.append(cov)
912     mean_v = float(np.mean(vals))
913     p10 = float(np.percentile(vals, 10))
914     p50 = float(np.percentile(vals, 50))
915     p90 = float(np.percentile(vals, 90))
916     print(f" _t={st:.2f} -> mean={mean_v:.3f}, P10={p10:.3f}, P50={p50:.3f}, P90={p90:.3f}")
917     table.append((st, mean_v, p10, p50, p90))
918 return table
919
920 def validate_q5(ans,
921     n_trials: int = 100,
922     local_sigma_t: float = 0.6,
923     local_sigma_tau_rel: float = 0.30,
924     dt_list=(0.010,0.015,0.020,0.030),
925     NANG_list=(24,48,96),
926     NZ_list=(5,9,13),
927     curve_sig_t=(0.0,0.2,0.4,0.6,0.8,1.0),
928     curve_reps: int = 30):
929     __ = monte_carlo_compare(ans, n_trials=n_trials,
930         sigma_t=local_sigma_t, sigma_tau_rel=local_sigma_tau_rel,
931         dt=DT_STEP, N_ANG=48, N_Z=9, INCLUDE_SIDE=True,
932         also_global_random=True)
933     __ = sweep_dt(ans, dt_list=dt_list, N_ANG=48, N_Z=9, INCLUDE_SIDE=True)
934     __ = sweep_sampling(ans, NANG_list=NANG_list, NZ_list=NZ_list, dt=DT_STEP)
935     __ = perturbation_curve(ans, sig_t_grid=curve_sig_t, sig_tau_rel=local_sigma_tau_rel,
936         reps=curve_reps, dt=DT_STEP, N_ANG=48, N_Z=9)
937
938 # 邻域检验
939 def _group_by_uav(selected_internal):
940     by={}
941     for c in selected_internal:
942         by.setdefault(c["uav"], []).append(c)
943     for u,lst in by.items():
944         lst.sort(key=lambda x: x["t_drop"])
945     return by
946
947 def _apply_uniform_shift_per_uav(base_sel, eps_t: float, eps_tau: float, rng_loc):
948

```

```

949 cand = copy.deepcopy(base_sel)
950 by = _group_by_uav(cand)
951 for u,lst in by.items():
952     dt_u = float(rng_loc.uniform(-eps_t, eps_t))
953     da_u = float(rng_loc.uniform(-eps_tau, eps_tau))
954     for c in lst:
955         td = clip(float(c["t_drop"]) + dt_u, 0.0, 60.0)
956         ta = clip(float(c["tau"]) + da_u, 0.2, 12.0)
957         c["t_drop"] = td
958         c["tau"] = ta
959         c["t_burst"] = td + ta
960     return cand
961
962 def _apply_small_jitter_per_shot(base_sel, eps_t: float, eps_tau: float, rng_loc):
963     cand = copy.deepcopy(base_sel)
964     by = _group_by_uav(cand)
965     for u,lst in by.items():
966         for c in lst:
967             td = clip(float(c["t_drop"]) + float(rng_loc.uniform(-eps_t, eps_t)), 0.0, 60.0)
968             ta = clip(float(c["tau"]) + float(rng_loc.uniform(-eps_tau, eps_tau)), 0.2, 12.0)
969             c["t_drop"] = td
970             c["tau"] = ta
971             c["t_burst"] = td + ta
972         lst.sort(key=lambda x:x["t_drop"])
973         for i in range(1, len(lst)):
974             if lst[i]["t_drop"] < lst[i-1]["t_drop"] + MIN_DROP_GAP:
975                 lst[i]["t_drop"] = min(60.0, lst[i-1]["t_drop"] + MIN_DROP_GAP)
976         if lst and lst[-1]["t_drop"] > 60.0:
977             lst[-1]["t_drop"] = 60.0
978         for i in reversed(range(len(lst)-1)):
979             hi = lst[i+1]["t_drop"] - MIN_DROP_GAP
980             if lst[i]["t_drop"] > hi:
981                 lst[i]["t_drop"] = max(0.0, hi)
982         for c in lst:
983             c["t_burst"] = float(c["t_drop"]) + float(c["tau"])
984     return cand
985
986 def neighborhood_validate_best(ans,
987     n_samples: int = 100,
988     eps_t: float = 0.30,
989     eps_tau: float = 0.20,
990     ratio_per_shot_jitter: float = 0.25,
991     dt: float = DT_STEP,
992     N_ANG: int = 48, N_Z: int = 9, INCLUDE_SIDE: bool = True,
993     seed: int = 20250):
994     rng_loc = np.random.default_rng(seed)
995     base_sel = _ans_to_internal_selected(ans["selected"])

```

```

996 base_cov, __, __ = _recompute_cover_L1(base_sel, dt, N_ANG, N_Z, INCLUDE_SIDE)
997
998 covers = []
999 better = 0
1000 best_improve = (-1e9, None)
1001 for k in range(n_samples):
1002     if rng_loc.random() < ratio_per_shot_jitter:
1003         cand = _apply_small_jitter_per_shot(base_sel, eps_t, eps_tau, rng_loc)
1004     else:
1005         cand = _apply_uniform_shift_per_uav(base_sel, eps_t, eps_tau, rng_loc)
1006     cov, __, __ = _recompute_cover_L1(cand, dt, N_ANG, N_Z, INCLUDE_SIDE)
1007     covers.append(cov)
1008     if cov > base_cov + 1e-9:
1009         better += 1
1010     if cov > best_improve[0]:
1011         best_improve = (cov, k)
1012
1013 print("\n[邻域检验 (最优解周围)]")
1014 print(f" 基线覆盖 (L1): {base_cov:.3f} s")
1015 print(f" 邻域半径: eps_t={eps_t:.2f}s, eps_tau={eps_tau:.2f}s, 样本数={n_samples}")
1016 print(f" 覆盖统计: mean={np.mean(covers):.3f}, median={np.median(covers):.3f}, "
1017       f"max={np.max(covers):.3f}, min={np.min(covers):.3f}")
1018 print(f" 超过基线的比例: {better}/{n_samples} = {better/n_samples:.2%}")
1019 if best_improve[0] > base_cov:
1020     print(f" 最佳样本提升: +{(best_improve[0]-base_cov):.3f} s (绝对覆盖 {best_improve[0]:.3f})")
1021 else:
1022     print(" 邻域内未发现比基线更好的样本。")
1023 return {
1024     "base": base_cov,
1025     "covers": covers,
1026     "better_count": better,
1027     "best_cov": best_improve[0]
1028 }
1029
1030 if __name__ == "__main__":
1031     ans = solve_q5(
1032         dt_step=DT_STEP,
1033         do_polish_L1=True,
1034         do_global_SA=True,
1035         heading_tol_deg=HEADING_TOL_DEG,
1036         speed_tol=SPEED_TOL
1037     )
1038 print("\n[Q5 | 结果概览]")
1039 for k_, v in ans.items():
1040     if k_ != "rows":
1041         print(" ", k_, ":", v)
1042

```

```

1043 print("\n[Q5 | 报表] 无人机 × 烟幕投放 × 导弹干扰明细 (12 列, 含零遮蔽行): ")
1044 print_report_rows(ans["rows"])
1045
1046 validate_q5(ans,
1047             n_trials=100,
1048             local_sigma_t=0.6,
1049             local_sigma_tau_rel=0.30,
1050             dt_list=(0.010,0.015,0.020,0.030),
1051             NANG_list=(24,48,96),
1052             NZ_list=(5,9,13),
1053             curve_sig_t=(0.0,0.2,0.4,0.6,0.8,1.0),
1054             curve_reps=30)
1055
1056 neighborhood_validate_best(ans,
1057                             n_samples=100,
1058                             eps_t=0.30,
1059                             eps_tau=0.20,
1060                             ratio_per_shot_jitter=0.25,
1061                             dt=DT_STEP, N_ANG=48, N_Z=9, INCLUDE_SIDE=True,
1062                             seed=20250)
1063

```